

Experimentelle Bearbeitung von Möglichkeiten und Grenzen für Webentwickler durch aktuelle chatbasierte KI-Modelle

Studiengang Medieninformatik

Bachelorarbeit

vorgelegt von

Robin Richard Prause

geb. in Starnberg

durchgeführt an der

Technischen Hochschule Mittelhessen, Friedberg

Referent der Arbeit:

Korreferent der Arbeit:

Betreuer bei Randstad Digital Germany AG:

Dipl. Ing. Benjamin Einert

Prof. Dr.-Ing. Nicolas Stein

Nicolas Wombacher

Friedberg, 2023

Selbstständigkeitserklärung

Ich erkläre, dass ich die eingereichte Bachelorarbeit selbstständig und ohne fremde Hilfe verfasst, andere als die von mir angegebenen Quellen und Hilfsmittel nicht benutzt und die den benutzten Werken wörtlich oder inhaltlich entnommenen Stellen als solche kenntlich gemacht habe.

Friedberg, November 2023

Robin Richard Prause

Inhaltsverzeichnis

Selbstständigkeitserklärung	i
Inhaltsverzeichnis	iii
Abbildungsverzeichnis	v
1 Einleitung	1
1.1 Motivation	1
1.2 Problemstellung	2
1.3 Zielsetzung	3
1.4 Aufbau der Arbeit	4
2 Stand der Technik	7
2.1 Mangel an Daten	7
2.1.1 Datenqualität	7
2.1.2 Datenquantität	7
2.1.3 Schlussfolgerung	8
2.2 Datenschutz, Sicherheit und Ethik	8
2.2.1 Datenschutz und Ethik in KI-Modellen	8
2.2.2 Sicherheit und Verantwortung in der KI-Entwicklung	8
2.2.3 Methoden zur Reduzierung von Vorurteilen und Risiken	8
2.2.4 Schlussfolgerung	9
2.3 Komplexität von KI-Modellen	9
2.4 Neue Möglichkeiten von KI	9
2.4.1 Inhaltserstellung	10
2.4.2 Empfehlungsmodelle	10
2.4.3 Prozessautomatisierung und -optimierung	11
2.4.4 Softwareentwicklung	11
Automatisierung von Tests	12
2.5 Aktuelle Tools	12
2.5.1 ChatGPT und Google Bard	12
2.5.2 Generative KI für Bilder	13
2.5.3 UIZard und Autodesigner	13
2.5.4 Github Copilot	14

2.5.5	Der GPT-Engineer	14
2.6	Verwandte Arbeiten	14
2.6.1	Über die Schulter geschaut	14
2.6.2	Yes, You Can Make an App Too: A Systematic Study of Prompt Engineering in the Automatic Generation of Mobile Applications from User Queries	15
2.6.3	Künstliche Intelligenz in der Softwareentwicklung	16
2.6.4	How Artificial Intelligence Can Improve Web Development and Testing	17
2.6.5	AI for Agile development: a Meta-Analysis	17
2.6.6	A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT	18
2.6.7	Interactive and Visual Prompt Engineering for Ad-hoc Task Adaptation with Large Language Models	19
2.7	Warum es weiterer Behandlung bedarf	20
3	Grundlagen und Begriffsdefinitionen	23
3.1	Künstliche Intelligenz und Artificial Intelligence	23
3.1.1	Starke und Schwache KI	24
	Schwache KI (Narrow AI)	24
	Starke KI (General AI)	24
3.2	Machine Learning und deep Learning	25
3.2.1	Machine Learning (ML)	25
3.2.2	Deep Learning (DL)	25
3.2.3	Fazit	26
3.3	Supervised, unsupervised und reinforcement Learning	26
3.4	Neuronale Netzwerke	27
3.4.1	Struktur und Aufbau	27
3.4.2	Bias, Overfitting und Underfitting	27
3.4.3	Typen und Funktionen von neuronalen Netzwerken	28
3.5	Generative KI	28
3.5.1	Foundation Models	28
3.5.2	Large Language Models (LLMs)	29
3.5.3	Natural Language Processing (NLP)	30
3.5.4	Tuning und CodeX	31
3.6	Wichtigkeit von Daten	32
3.6.1	Knowledge Graphs	32
3.6.2	Schlechte Daten	32
4	Verfahrensauswahl	33
4.1	Arbeiten mit Chatbots	33
4.1.1	Warum ChatGPT?	33
4.1.2	Allgemeines zu ChatGPT	34
4.1.3	Aufbau eines Chats	35
4.1.4	Funktionsweise und Verarbeitung	35

4.1.5	Vorgehensweise mit ChatGPT	36
4.2	Konzeptionierung der Webanwendung	37
4.2.1	Warum eine Webanwendung?	37
4.2.2	Problemstellung	37
4.2.3	Erwartungshaltung an Backend und Frontend	38
4.2.4	Ziel der Webanwendung	38
4.2.5	Architektur & Softwaredesign	38
4.3	Prompt Based Entwicklung	40
4.3.1	Was ist ein Prompt?	40
4.3.2	Ansätze von Prompt Engineering	40
5	Generierung und Analyse	43
5.1	Umgesetzte Features	43
5.1.1	Login und Logout	43
5.1.2	Events	43
5.1.3	Staff	45
5.1.4	Lines und Business Units	47
5.1.5	Feedback	47
5.1.6	Votes	48
5.1.7	Dashboard	49
5.2	Leitfaden der Generierung	50
5.2.1	Definition of Coding	50
5.2.2	Vorgehen	50
5.3	Generierung der Features	52
5.4	Bewertung der Codequalität	53
5.4.1	Benennung im Code	53
5.4.2	Errorhandling	54
5.4.3	Weiteres	54
6	Auswertung	57
6.1	Datenaufbereitung	57
6.1.1	Der Effizienzfaktor	58
6.1.2	Übertrag in Airtable	59
6.2	KPIs und Charts	60
6.2.1	Prompts je Kategorie	60
6.2.2	Lines of Code je Prompt	61
6.2.3	Effizienz Faktor je Kategorie	61
6.2.4	Effizienzfaktorverlauf bei gestellten Rückfragen	62
6.2.5	Verwendung von Prompt Bricks	63
6.2.6	Verwendung von Prompt Bricks	64
6.2.7	Rückfragen bei Prompt Bricks	65
6.2.8	Wurde ein Prompt Brick verwendet	66
6.3	Aufgetretene Grenzen	66
6.3.1	Zeitersparnis	66

6.3.2	Kontextspeicher	67
6.3.3	Integration in die industrielle Softwareentwicklung	67
6.3.4	Veraltete Technologieversionen	68
6.3.5	Fortbildung der Entwickler	68
6.3.6	Fazit	68
7	Ergebnisse	69
7.1	Ausgangssituation	69
7.2	Lösungsansatz	69
7.3	Evaluation	70
7.4	Vergleich zum traditionellen Entwicklungsansatz	71
7.4.1	Zeitfaktor	71
7.4.2	Coding Richtlinien	71
7.4.3	Komplexität der Features	72
7.4.4	Updates und Skalierbarkeit	72
8	Zusammenfassung	73
8.1	Retrospektive	73
8.2	Ausblick	74
8.2.1	Technologieperspektive	74
8.2.2	Entwicklerperspektive	75

Abbildungsverzeichnis

3.1	Schematische Darstellung eines 3-Schichtigen NN	28
4.1	Größenvergleich zwischen GPT-3 und GPT-4 auf Basis der Anzahl der Parameter. In Anlehnung an Quelle: <i>The Ultimate Guide to GPT-4 Parameters</i> [41]	34
4.2	Schematische Darstellung der Architektur in Frontend und Backend	39
5.1	Frontend View zur Erstellung eines Events (Screenshot aus der Webapp)	44
5.2	Datenbankobjekt eines Event in der Datenbank	44
5.3	Frontend View zur Erstellung eines Staff (Screenshot aus der Webapp)	45
5.4	Frontend View zum Importieren von Staff (Screenshot aus der Webapp)	46
5.5	CSV Datei zum Importieren von Staff	46
5.6	Frontend View für die Übersicht aller Lines (Screenshot aus der Webapp)	47
5.7	Frontend View für das Einsehen von Bewertungen zu einem Event (Screenshot aus der Webapp)	48
5.8	Frontend View für das Abstimmen für Events (Screenshot aus der Webapp)	48
5.9	Frontend View für das Dashboard (Screenshot aus der Webapp)	49
5.10	Backend Methode aus dem Event Service	53
5.11	Frontend Skript aus der Komponente UpcomingEvents.vue	54
5.12	Ausschnitt aus der Datei lineStore.js im Frontend	55
6.1	Ausschnitt aus der Tabelle, die zur Auswertung der Daten verwendet wird (zur besseren Darstellung wurden nicht alle Felder abgebildet).	59
6.2	Darstellung, wie viele Prompts je Kategorie benötigt wurden	60
6.3	Darstellung, wie viele Zeilen Code mit jedem Prompt im Durchschnitt generiert wurden	61
6.4	Darstellung des Effizienzfaktors je Kategorie	62
6.5	Darstellung des Effizienzfaktorverlaufs bei gestellten Rückfragen	63
6.6	Darstellung der Nutzung von Prompt Bricks	64
6.7	Darstellung des Effizienzfaktors je Brick	64
6.8	Darstellung der Rückfragen je Prompt Brick	65
6.9	Darstellung des Effizienzfaktors, wenn ein Prompt Brick verwendet wurde	66
8.1	Darstellung des Effizienzfaktors über das gesamte Projekt	73

Kapitel 1

Einleitung

1.1 Motivation

Künstliche Intelligenzen sind schon länger auf dem Markt und unterstützen bereits in hohen Umfängen Konzerne wie Amazon, Microsoft oder Google [77] [32] [3]. Der Endkunde hatte bisher kaum Zugang zu solchen Tools. In den vergangenen Monaten wurde insbesondere das von OpenAI entwickelte chatbasierte KI-Modell GPT einer breiten Öffentlichkeit bekannt [13]. Durch den frei verfügbaren Chatbot, ChatGPT [16], welcher auf dem gleichnamigen KI-Modell basiert, wird die künstliche Intelligenz einfach und kostenfrei für jeden zugänglich. Die Veröffentlichung von ChatGPT 2022 [42] löste eine Reihe von Entwicklungen aus, die viele Unternehmen in „Zugzwang“ brachten und dazu anregten, die eigenen Geschäftsprozesse mit künstlicher Intelligenz zu unterstützen. Neben der Unternehmenswelt müssen auch in der Lehre neue Tools entwickelt werden, um etwa die Eigenständigkeit von Hausaufgaben bis hin zu Abschlussarbeiten überprüfen zu können. Durch die große Zeitersparnis, die Schüler und Studenten in den Einsatzmöglichkeiten und dem unendlichen Wissen einer KI sehen, wird daher auch die Echtheit und Authentizität von Schriftstücken infrage gestellt.

Zu den vielen Einsatzzwecken gehört auch das Generieren von Werbetexten oder von Code. In der Entwicklung ermöglichen daher immer mehr Unternehmen ihren Entwicklern, anhand von Fragen, einfache Code-Teile durch solche chatbasierten Tools generieren zu lassen. Im Vordergrund steht dabei vorwiegend die hohe Zeitersparnis, da Entwickler bei konkreten Aufgaben nicht länger selbst den Code schreiben müssen, sondern ihn innerhalb von Sekunden generieren lassen können. Entwickler prüfen den Code anschließend nur noch einmal manuell, um sicherzustellen, dass sich keine Fehler im Code befinden. Neben ChatGPT existieren mittlerweile auch auf GPT-basierte Chatbots, die noch einmal speziell für das Schreiben von Code trainiert wurden. Zu diesen Chatbots für Code gehört unter anderem GitHub Copilot [28]. Dieser lässt sich nahtlos in Entwicklungsumgebungen integrieren und ermöglicht es dadurch, dass der Entwickler noch einfacher nach Lösungen für ein gegebenes Problem suchen kann. Weiterhin gibt es Tools, die auf KI setzen und so mit einer erhöhten Produktivität für den Anwender werben. Dazu gehören im Webumfeld das Designtool „UIZard“, welches anhand einer kurzen Beschreibung vollständige UX/UI-Designs in

einem bestimmten Stil erzeugen kann. Im Projektmanagement Umfeld lässt sich Notion als Beispiel nennen. Hier kann der Nutzer die fortschrittliche Technologie durch einen einfachen Mausklick verwenden. So kann eine Seite mit Text befüllt oder eine Beschreibung ausformuliert werden, mit vergleichsweise wenig Aufwand.

Die breite Masse an Tools, die sich bereits auf dem freien Markt befinden, zeigt, dass sich die Nutzer und Unternehmen wünschen und darum bemühen, Arbeitsprozesse kosten- und zeiteffizienter durch den Einsatz von KIs zu gestalten. Nahezu jeder Bereich eines Webentwicklers kann bereits in einer Form unterstützt werden. Hierbei gilt es abzuwägen, ob der Output durch die KI größer ist als der Input, also die Kommunikation mit dem Modell. Lässt sich der Webdesigner mithilfe des Tools UIZard ein Design erstellen, muss anschließend betrachtet werden, wie viel hier noch manuell bearbeitet oder umstrukturiert werden muss, um zu einem professionellen Ergebnis zu gelangen. Das heißt, es stellt sich bei all diesen Tools die Frage, bis zu welchem Grad KI-Modelle den Endanwender bei der Arbeit unterstützen können oder wo die Resultate noch nicht dem Maßstab des professionellen Entwicklers oder Designers entsprechen. Fakt ist aber, dass die aktuelle Entwicklung aufzeigt, dass die Bestrebungen groß sind, den Anwender bei der Umsetzung eines Konzepts maximal zu unterstützen.

Konkret soll im Rahmen dieser Bachelorarbeit genauer untersucht werden, ob es nicht auch bereits möglich ist, auf das Schreiben von Code im Kontext des Webentwicklers gänzlich zu verzichten. Das heißt, in einem Praxisversuch soll ein reiner „prompt-based“^[64] Ansatz gewählt und der Code nur durch Rückfragen und Anmerkungen an das KI-Modell modifiziert werden. Dieser Versuch soll zeigen, was bereits mithilfe künstlicher Intelligenz möglich ist und wo die Forschung noch stärker vorangetrieben werden muss, um Ergebnisse industriellen Maßstabs zu erreichen.

1.2 Problemstellung

Im Zuge rasanter Entwicklungen in vielen Bereichen der modernen Gesellschaft steigen auch die Anforderungen an die Informationstechnologie. Insbesondere die Softwarebranche ist durch großen Fachkräftemangel geprägt und gleichzeitig mit stetig wachsender Nachfrage und Anforderungen konfrontiert. In diesem Zuge werden die Entwicklungsabläufe, Tools und Projektmanagement Frameworks immer agiler, um ein maximales Maß an Effizienz hervorzurufen. Neben neuen Programmiersprachen, Frameworks, Projektmanagementtools, Versionsverwaltungstools und täglich neu erscheinenden Technologien, arbeiten größere Unternehmen auch an der sogenannten künstlichen Intelligenz. Damit soll das Ziel verfolgt werden, langfristig die Aufgabenbereiche von Entwicklern, Business Analysten, UX/UI-Designern und vielen weiteren auf den rein kreativen Prozess zu fokussieren und neue Herangehensweisen zu entdecken. Gleichzeitig bedeutet dies, dass die automatisierbaren Aufgaben ausschließlich von künstlichen Intelligenzen übernommen werden sollen. Repetitive Tasks, wie das Anlegen einer User Story, das Entwickeln einer kleinen Komponente oder das Design eines bestimmten Buttons soll zukünftig nicht mehr durch die Arbeitskraft umgesetzt werden. Das würde

neben einer großen Zeitersparnis und der erwähnten Verschiebung des Fokus auf den kreativen Prozess viele weitere Vorteile mit sich bringen.

Heute existieren bereits viele Sprachmodelle, die auf riesigen Datensätzen trainiert werden, um dabei zu unterstützen, Texte oder auch Code zu generieren. Diese Sprachmodelle sind bereits in der Lage, längere Codeabschnitte durch eine einfache Frage oder kurze Spezifikation zu generieren. Neben textbasierten Modellen wurden auch weitere Modelle entwickelt, die gleichermaßen Bilder analysieren und neue Bilder erstellen können [19]. In diesem Kontext existieren erste Tools, die dabei unterstützen können, UX-Designs per Prompt automatisch zu generieren [5]. Dabei kann bereits ein ganzes Layout durch eine Beschreibung des Inhalts, Themas und der groben Stil-Vorstellungen generiert werden.

Auch wenn dutzende Vorteile entstehen, wächst in den genannten Berufsfeldern die Frage nach der eigenen „Ersetzbarkeit“ stetig. Im Verlauf dieser Bachelorarbeit soll genau dieses Phänomen untersucht werden, indem die Frage aufgeworfen wird, ob ein Entwickler zukünftig durch eine KI vollkommen ersetzt werden kann. Es soll darüber hinaus analysiert werden, inwieweit schon heute eine Verbindung verschiedenster Tools als Ergebnis eine Webanwendung ohne das Schreiben von Code erzeugen kann. Um diese Ergebnisse zu generieren, etabliert sich aktuell der Begriff „prompt-based“ Development. Durch dieses „prompt-based“ Development wird es notwendig, Entwickler stärker in der Architektur zu schulen als im eigentlichen „Coden“. Damit wird der Ansatz verfolgt, mit der chatbasierten KI zu kommunizieren, um mit jedem weiteren Prompt einen besseren Kontext zu gestalten, aus der anschließend der benötigte Code generiert werden kann.

Wenn bereits Tools in den meisten Berufsfeldern existieren, die den Anwender bei seiner Arbeit unterstützen können, stellt sich die Frage, wie viel Aufwand betrieben werden muss, um bereits heute durch den Einsatz dieser Tools, Ergebnisse industriellen Maßstabs zu erzielen. Im Rahmen dieser Arbeit soll daher durch experimentelle Bearbeitung die Grenzen und Möglichkeiten für Webentwickler durch aktuelle chatbasierte KI-Modelle untersucht werden.

1.3 Zielsetzung

Zunächst soll ein gutes Verständnis für allgemeine Begriffe erzeugt werden, die im Sprachgebrauch häufig miteinander vermischt werden. Dazu zählt machine Learning, deep Learning und künstliche neuronale Netzwerke sowie die Einordnung in den Kontext von künstlicher Intelligenz. Außerdem muss eine wissenschaftliche Grundlage geschaffen werden, auf Basis dieser dann fundierte Ergebnisse bewertet werden können. Neben Supervised und unsupervised learning ist es auch wichtig zu klären, worum es sich bei starker und schwacher künstlicher Intelligenz handelt. Das Prompt-based Engineering steht ebenfalls im Fokus dieser Arbeit, da hiermit das Zentrum des experimentellen Ansatzes gebildet wird. In dem Kontext dieser Begriffe muss immer auch eine technische Erläuterung mit einhergehen.

KI-Modelle werden immer auf bereits existierenden Datensätzen trainiert und können daher sensible oder auch falsche Informationen wiedergeben. Daher sollen auch ethische und Datenschutzbedenken kurz angerissen werden. Anhand einer direkten Untersuchung soll herausgefunden werden, bis zu welchem Grad eine Webanwendung bereits durch eine intelligent eingesetzte Toolchain entwickelt werden kann. Dabei soll möglichst auf das Eingreifen eines

Entwicklers verzichtet und der Automatisierungsgrad maximal ausgeschöpft werden. Diese Untersuchung soll im Vordergrund dieser Arbeit stehen. Technische Begriffe und Grundlagen sollen dahingehend erklärt werden, dass der Leser dieser Arbeit, mit einem grundlegenden technischen Verständnis, dem Aufbau und den Denkansätzen des Versuchs folgen kann. Im weiteren Verlauf könnten für Unternehmen einige Hilfestellungen anhand der Erfahrungen, die gesammelt wurden, gestellt werden. Dadurch können Unternehmen die Toolchain in ihrem eigenen Setup intern anwenden. Zusätzlich lässt sich eine Untersuchung durchführen, die die Zeitersparnis durch die Verwendung einer intelligenten Toolchain quantifiziert. Ein Vergleich mit herkömmlichen Entwicklungsansätzen dient dabei zur Veranschaulichung der erzielten Ergebnisse.

1.4 Aufbau der Arbeit

Die vorliegende Bachelorarbeit widmet sich dem Thema „experimentelle Bearbeitung von Möglichkeiten und Grenzen für Webentwickler durch aktuelle chatbasierte KI-Modelle“. In der Einleitung (1) wird der Leser in die Thematik eingeführt und über die Motivation, Problemstellung sowie Zielsetzung der Arbeit informiert. Die rasante Entwicklung der Künstlichen Intelligenz und insbesondere des deep Learning hat in den letzten Jahren zu bedeutenden Fortschritten im Bereich der automatisierten Codegenerierung geführt. Die Möglichkeit, durch KI-Systeme wie ChatGPT, Google Bard, GitHub Copilot und andere chatbasierte Systeme, Code automatisch zu erzeugen, eröffnet vielfältige Chancen und Potenziale für die Softwareentwicklung. Unternehmen können von enormen Zeitersparnissen profitieren, indem routinemäßige und wiederkehrende Aufgaben automatisiert werden. Dennoch bestehen Herausforderungen, die es zu bewältigen gilt, um die Codegenerierung in einem industriellen Maßstab einzusetzen.

Im Kapitel „Stand der Technik“(2.6) wird ein Überblick über bereits vorhandene Forschungsarbeiten und Lösungsansätze zur automatisierten Codegenerierung mittels künstlicher Intelligenz gegeben. Dabei werden verschiedene Modelle wie ChatGPT, Google Bard, Dall-E, Uizard oder GitHub Copilot betrachtet und analysiert. Des Weiteren wird erläutert, warum bisherige Ansätze das Problem der Codegenerierung nicht vollständig lösen konnten.

Im Kapitel „Grundlagen und Begriffsdefinitionen“(3) werden relevante Begriffe wie künstliche Intelligenz, machine Learning, deep Learning, supervised und unsupervised Learning, künstliche neuronale Netzwerke sowie foundation und large language Models erklärt, um das Verständnis für die technischen Grundlagen der automatisierten Codegenerierung zu schaffen.

Im Kapitel „Verfahrensauswahl“ 4 wird zunächst ChatGPT und der Aufbau von Chatbots näher betrachtet. Im zweiten Schritt soll die Konzeptionierung einer Webanwendung als Beispiel für die Codegenerierung mittels KI im Fokus stehen. Es wird untersucht, wie eine feingranulare komponentenbasierte Entwicklung und die Verwendung von Prompts in Verbindung mit einem KI-Framework zu einer effizienten Entwicklung führen können. Zuletzt soll das Prompt-Engineering im Fokus stehen.

Im Kapitel „Generierung“(5) erfolgt die praktische Umsetzung des zuvor konzipierten Modells. Es wird ein klares Vorgehensmodell für die Generierung der Webanwendung erstellt

und erläutert, wie die einzelnen Komponenten zusammengefügt wurden. Dieses Kapitel soll dem Leser einen Überblick über die Webanwendung geben und aufzeigen, welche Features umgesetzt wurden.

In der „Auswertung“ (6) befasst sich die Arbeit mit einer umfassenden Analyse der Herausforderungen und Anforderungen im Zusammenhang mit der automatisierten Codegenerierung. Dabei werden Aspekte wie der industrielle Maßstab von Code, die Integration in bestehende Toolchains, die Zeitersparnis und die Schaffung eines wiederkehrenden Kontexts für das Sprachmodell beleuchtet.

Im Kapitel „Ergebnisse“ (7) werden die Ergebnisse der Implementierung detailliert dargestellt und evaluiert. Eine Ausgangssituation wird beschrieben, und der eigene Lösungsansatz wird aufbereitet. Zudem wird ein Vergleich zum traditionellen Entwicklungsansatz gezogen.

In der „Zusammenfassung“ (8) erfolgt abschließend eine Retrospektive über die Arbeit, in der aufgezeigt wird, was gut lief und was nicht gut lief. Zudem werden mögliche zukünftige Lösungsansätze für noch nicht behandelte Teile des Problems aufgezeigt.

Mit diesem Aufbau wird die Bachelorarbeit einen ganzheitlichen Einblick in die Thematik der automatisierten Codegenerierung durch künstliche Intelligenz bieten und wertvolle Erkenntnisse für die weitere Forschung und Praxis liefern.

Kapitel 2

Stand der Technik

Künstliche Intelligenz (KI) hat sich als revolutionäre Technologie erwiesen, die zahlreiche Anwendungen in verschiedenen Bereichen ermöglicht. Trotz der beeindruckenden Fortschritte gibt es jedoch Grenzen und Herausforderungen, die das volle Potenzial von KI einschränken können. Dieses Kapitel untersucht diese Aspekte, einschließlich der Verfügbarkeit von Daten, der Komplexität von Modellen, Bias, Datenschutz, Trainingsdaten und ethischen Bedenken.

2.1 Mangel an Daten

2.1.1 Datenqualität

Die Qualität der Daten spielt eine entscheidende Rolle bei der Entwicklung effektiver KI-Modelle. In einer Welt, in der Daten in beispielloser Geschwindigkeit generiert werden, ist die Gewährleistung der Genauigkeit, Relevanz und Integrität dieser Daten eine Herausforderung. Die Qualität der Daten kann durch verschiedene Faktoren beeinträchtigt werden, einschließlich unvollständiger Datensätze, Inkonsistenzen, Verzerrungen und menschlichen Fehlern. Wenn solche Probleme nicht erkannt und behoben werden, können sie zu fehlerhaften Analysen und Vorhersagen führen, die die Effektivität von KI-Modellen beeinträchtigen.

2.1.2 Datenquantität

Neben der Qualität ist auch die Quantität der Daten von entscheidender Bedeutung. Bernard Marr hebt in seinem Dokument [\[12\]](#) hervor, dass wir in den letzten zwei Jahren mehr Daten geschaffen haben als in der gesamten vorherigen Geschichte der Menschheit. Bis 2020 sollten etwa 1,7 Megabyte neue Daten pro Sekunde für jeden Menschen auf dem Planeten erstellt werden. Diese Daten stammen nicht nur aus Kommunikationsmitteln wie E-Mails und sozialen Medien, sondern auch aus einer wachsenden Anzahl von Sensoren in intelligenten Geräten. Trotz dieser enormen Datenmenge kann ein Mangel an relevanten und qualitativ hochwertigen Daten die Entwicklung von KI-Modellen behindern. Die Sammlung von Daten, die für ein bestimmtes Forschungs- oder Geschäftsziel relevant sind, kann eine komplexe und zeitaufwändige Aufgabe sein.

2.1.3 Schlussfolgerung

Der Mangel an hochwertigen, geprüften Daten ist ein kritisches Hindernis für die Entwicklung robuster KI-Modelle. Die Gewährleistung der Datenqualität erfordert sorgfältige Überwachung und Bereinigung, um Ungenauigkeiten und Verzerrungen zu vermeiden. Gleichzeitig muss die Datenquantität ausreichend sein, um repräsentative Muster und Beziehungen innerhalb der Daten zu erfassen. Die Herausforderungen in Bezug auf Qualität und Quantität erfordern eine sorgfältige Planung, Ressourcenallokation und Expertise, um effektive KI-Modelle zu entwickeln, die in der Lage sind, genaue und zuverlässige Erkenntnisse zu liefern [12].

2.2 Datenschutz, Sicherheit und Ethik

2.2.1 Datenschutz und Ethik in KI-Modellen

Die Verwendung großer Datenmengen zur Entwicklung von künstlicher Intelligenz (KI) wirft auch wichtige Fragen zu Datenschutz und Ethik auf. Ein Aspekt, der in der Forschung hervorgehoben wurde, ist, dass Computerprogramme, die für das maschinelle Lernen verwendet werden, in der Lage sind, menschenähnliche Vorurteile aus den Daten zu „lernen“, mit denen sie gefüttert werden. Diese Vorurteile können harmlos sein, wie die Verknüpfung von Blumen mit Insekten, oder sie können ernstere Probleme darstellen, wie Vorurteile gegenüber bestimmten Rassen oder Geschlechtern. Wenn die Datenaufbereitung nicht ausreichend überwacht wird, könnten diese Vorurteile in den Technologien, die wir entwickeln, verstärkt und verbreitet werden. Dies unterstreicht die Bedeutung von Kontrollen und Überprüfungen, um sicherzustellen, dass unsere Technologien ethisch und verantwortungsbewusst entwickelt werden.

2.2.2 Sicherheit und Verantwortung in der KI-Entwicklung

Die Sicherheit in der KI-Entwicklung bezieht sich nicht nur auf den Schutz der Daten selbst, sondern auch auf die Verantwortung, sicherzustellen, dass die Modelle, die aus diesen Daten abgeleitet werden, keine schädlichen oder diskriminierenden Muster aufweisen. Die Arbeit von Caliskan et al. zeigt, dass selbst weit verbreitete, off-the-shelf KI-Komponenten kulturelle Stereotypen in die KI-Technologien übertragen können. Dies unterstreicht die Notwendigkeit einer sorgfältigen Überprüfung und Kontrolle der Daten und der daraus resultierenden Modelle.

2.2.3 Methoden zur Reduzierung von Vorurteilen und Risiken

In der Welt der KI gibt es bereits Forschung und Techniken, die darauf abzielen, die Vorurteile in den Daten zu reduzieren, die für das Training von Modellen verwendet werden. Zum Beispiel haben einige Wissenschaftler eine Methode entwickelt, um die sogenannten „Wort-Einbettungen“ zu korrigieren, die in KI-Systemen verwendet werden, um die Bedeutung von Wörtern zu verstehen. Diese Korrektur hilft, die Vorurteile, die in der Sprache vorhanden sein könnten, zu entfernen. Darüber hinaus gibt es Bemühungen, die Beziehung zwischen

den in der Sprache vorhandenen Vorurteilen und den tatsächlichen Daten und Fakten über die Gesellschaft zu untersuchen. Dies hilft, ein klareres Bild davon zu bekommen, wie Vorurteile in KI-Systemen entstehen und wie sie möglicherweise reduziert oder eliminiert werden können.

Insgesamt bedarf es Methoden und Strategien, die entwickelt werden, um die Risiken zu minimieren, die mit der Verwendung von großen Datenmengen in der KI verbunden sind. Dies beinhaltet das Verständnis und die Korrektur von Vorurteilen in den Daten sowie die Untersuchung, wie diese Vorurteile mit der realen Welt in Beziehung stehen.

2.2.4 Schlussfolgerung

Die Verwendung großer Datenmengen zur Entwicklung von KI-Modellen birgt sowohl Chancen als auch Risiken. Die Erkenntnisse aus der Studie von Caliskan et al. bieten Einblicke in die potenziellen Fallstricke und ethischen Bedenken, die berücksichtigt werden müssen. Durch ein besseres Verständnis der Mechanismen, durch die Vorurteile in KI-Modellen verankert werden können, und durch die Entwicklung von Methoden zur Minimierung dieser Risiken, kann die KI-Entwicklung in eine Richtung gelenkt werden, die sowohl sicher als auch ethisch verantwortungsbewusst ist [15].

2.3 Komplexität von KI-Modellen

Tiefe neuronale Netzwerke haben sich als leistungsstarke Werkzeuge in Bereichen wie Sprach- und Bilderkennung erwiesen. Ihre Hochleistungsfähigkeit beruht jedoch auf einer Komplexität, die eine sorgfältige Analyse erfordert. Die Komplexität in der Entwicklung von KI-Modellen hat ein Niveau erreicht, das spezialisierte Kenntnisse und Fähigkeiten erfordert. Da diese Modelle nicht zu interpretieren sind und Fähigkeiten besitzen, gegenintuitive Eigenschaften zu lernen, wird die Einstellung von Fachleuten, die diese Nuancen verstehen und steuern können, unerlässlich.

Viele Unternehmen beginnen bereits und müssen auch zukünftig in die Einstellung von technischen Experten investieren, die diese komplexen Modelle verstehen und effektiv einsetzen können. Die Fähigkeit, die zugrunde liegenden Mechanismen und Eigenschaften von neuronalen Netzwerken zu verstehen, wird nicht nur ein Bereich der akademischen Forschung bleiben, sondern zu einer geschäftlichen Notwendigkeit werden. Die Investition in qualifiziertes Personal wird somit zu einem entscheidenden Faktor für den Erfolg in einer zunehmend datengetriebenen Welt [63].

2.4 Neue Möglichkeiten von KI

Während Abschnitt 2.3 die Grenzen und Herausforderungen von KI untersuchte, bietet dieses Kapitel einen Einblick in die vielfältigen Möglichkeiten, die KI in verschiedenen Bereichen bietet. Von der Personalisierung von Inhalten bis zur Schaffung neuer Berufsfelder – die Anwendungen von KI sind weitreichend und vielfältig.

2.4.1 Inhaltserstellung

Die Unterstützung in der Inhaltserstellung durch künstliche Intelligenz (KI) hat sich als revolutionärer Fortschritt in der Content-Marketing-Branche erwiesen. KI-Tools wie ChatGPT und andere generative KIs werden zunehmend eingesetzt, um den Prozess der Ideenfindung, Inhaltsstrukturierung, Erstellung von Erstentwürfen und sogar der Bearbeitung und Korrektur zu beschleunigen. Zum Beispiel ermöglichen KI-Tools die Generierung neuer Ideen für Blogartikel, soziale Medien und andere Inhaltsformate, wenn ein kreativer Stillstand eintritt. Sie können auch potenzielle Strukturen mit Überschriften und Gesprächspunkten entwickeln, um den Schreibprozess zu erleichtern.

Die Verwendung von KI in der Inhaltserstellung spart nicht nur Zeit, sondern ermöglicht auch eine Skalierung der Inhaltsproduktion, wobei menschliche Eingriffe auf ein Minimum reduziert werden. Es ist jedoch wichtig zu betonen, dass KI-Tools die emotionale Intelligenz, Kreativität und das Verständnis von Nuancen, die den Inhalt hervorheben, noch vermissen lassen, und daher eine sorgfältige Überprüfung durch menschliche Redakteure erforderlich ist [2].

2.4.2 Empfehlungsmodelle

Die Integration von künstlicher Intelligenz in die wirtschaftliche Landschaft verändert bereits in großem Maße die Industrie. Unter anderem nutzen KI-Modelle, komplexe Algorithmen, um Benutzerdaten zu analysieren und personalisierte Inhalte oder Produkte bereitzustellen. Diese Empfehlungsmodelle haben sich als entscheidend für die Verbesserung der Kundenbindung und die Steigerung des Umsatzes erwiesen. Durch dieses Empfehlungsmodell wird das „Long Tail“-Problem ausgelöst. Modelle sind demnach in der Lage, seltene oder nicht auf den ersten Blick erkennbare Zusammenhänge zu identifizieren und zu empfehlen, die zu den spezifischen Interessen eines Benutzers passen könnten. Da diese Zusammenhänge aber durch den Menschen oft gar nicht erkannt werden können, wird eine Komplexität eingeführt, die nur schwer nachvollzogen werden kann. Durch die Verwendung von maschinellem Lernen und deep Learning können diese Systeme kontinuierlich aus den Daten lernen und ihre Empfehlungen im Laufe der Zeit verfeinern.

Deep Learning, spielt eine besonders wichtige Rolle in modernen Empfehlungsmodellen. Durch den Einsatz von massiv vernetzter Rechenleistung ermöglicht deep Learning komplexere Formen des maschinellen Lernens, die die Genauigkeit und Relevanz der Empfehlungen erhöhen können. Diese Technologie ermöglicht es, sich an die sich ständig ändernden Vorlieben und Verhaltensweisen der Benutzer anzupassen, wodurch die Benutzererfahrung weiter verbessert wird. Die Bedeutung von KI in Empfehlungsmodellen wird durch die Tatsache unterstrichen, dass ein erheblicher Anteil der Verkäufe und des Engagements in vielen Online-Unternehmen durch diese Technologie generiert wird. Die Investition in KI, insbesondere in deep Learning, trägt nicht nur zur Verbesserung der Kundenzufriedenheit bei, sondern bietet auch erhebliche wirtschaftliche Vorteile.

Darüber hinaus fördert die Open-Source-Natur vieler KI-Frameworks die Innovation und

ermöglicht es Entwicklern, die Empfehlungssysteme weiter zu verbessern. Dies trägt zur Entwicklung von Systemen bei, die mit weniger Daten genauere Vorhersagen treffen können, und zu Produkten, die die Kunden wahrscheinlicher anklicken und kaufen. Diese Technologien revolutionieren nicht nur die Art und Weise, wie Unternehmen mit ihren Kunden interagieren, sondern bieten auch eine Zukunft für die personalisierte Kundeninteraktion und das Unternehmenswachstum. Die kontinuierliche Forschung und Entwicklung in diesem Bereich verspricht, die Benutzererfahrung weiter zu verbessern und neue Möglichkeiten für die Geschäftswelt zu eröffnen [54].

2.4.3 Prozessautomatisierung und -optimierung

Mithilfe künstlicher Intelligenz können manuelle Aufgaben automatisiert werden, wodurch Mitarbeiter sich auf strategische Aufgaben konzentrieren können. Beispielsweise können KI-gesteuerte Chatbots Kundendienstanfragen bearbeiten, während Kundendienstmitarbeiter sich auf das Vorantreiben komplexerer Probleme konzentrieren können. Durch intelligente Planungs-, Buchhaltungs- und Projektmanagement-Tools kann ebenfalls Zeit eingespart und die Effizienz gesteigert werden. Es existieren mittlerweile verschiedene Tools, die durch intelligente Zeitplanung die Arbeitskraft bei der Planung des privaten und beruflichen Alltags unterstützen können. Durch Texterkennung und andere Teilgebiete der Computer Vision kann auch in der Buchhaltung eine massive Effizienzsteigerung beobachtet werden. Belege können automatisch kategorisiert, analysiert oder weiterverarbeitet werden. Im Projektmanagement kann insbesondere durch intelligente Planung der agile Entwicklungsprozess weiter unterstützt werden. So können User Storys automatisch geschätzt werden oder die Roadmap für einen Backlog automatisch geplant werden.

Weiterhin können KI-gesteuerte virtuelle Assistenten Mitarbeitern helfen, Aufgaben schneller zu erledigen, wodurch Zeit für wichtigeres Arbeiten frei wird. Automatisierte Berichts- und Analysetools können Echtzeitdaten und -einblicke liefern, die es Unternehmen ermöglichen, fundierte Entscheidungen zu treffen. Moderne Frameworks wie Langchain ermöglichen diese Entwicklung von intelligenten Business-Anwendungen. Weiterhin können KI-Algorithmen wesentlich größere Datenmengen analysieren und Muster und Trends identifizieren als es der Mensch könnte.

Zusammenfassend lässt sich sagen, dass KI-Automatisierung die Geschäftswelt verändern kann und ein neues Maß an Effizienz und Produktivität bringt. Durch die Automatisierung manueller Aufgaben, die Verbesserung der Datenanalyse und die Steigerung der Bottom-Line-Ergebnisse kann KI-Automatisierung Unternehmen dabei unterstützen, in der modernen Wirtschaft erfolgreich zu sein [1].

2.4.4 Softwareentwicklung

Die Integration von künstlicher Intelligenz (KI) in die Softwareentwicklung hat einen Paradigmenwechsel in der Art und Weise eingeleitet, wie Code geschrieben und verstanden wird. GitHub Copilot, ein KI-gestützter Programmierpartner, ist ein beispielhaftes Produkt dieser Entwicklung. Durch die Verwendung eines generativen KI-Modells, das auf öffentlich verfügbaren Quellcodes und natürlichen Sprachtexten trainiert wurde, bietet GitHub Co-

pilot Entwicklern sofortige Vorschläge für Codezeilen und ganze Funktionen. Die KI zieht Kontext aus Kommentaren und Code, um Vorschläge zu generieren, die den Entwicklungsprozess beschleunigen und die Qualität verbessern können. Diese Anwendung von KI in der Softwareentwicklung unterstreicht nicht nur die Vielseitigkeit und Leistungsfähigkeit von maschinellem Lernen, sondern zeigt auch, wie KI dazu beitragen kann, menschliche Fähigkeiten zu erweitern, repetitive Aufgaben zu automatisieren und die Entdeckung neuer Lösungswege zu fördern. Die kontinuierliche Entwicklung und Verfeinerung solcher KI-gestützten Werkzeuge verspricht, die Landschaft der Softwareentwicklung weiter zu transformieren und neue Möglichkeiten für Innovation und Effizienz zu eröffnen [28].

Automatisierung von Tests

Die Qualitätssicherung in der Softwareentwicklung ist von manuellen Methoden hin zu automatisierten Prozessen vorangeschritten. Mit der Einführung von KI in die Automatisierung von Tests hat sich die Qualitätssicherung weiterentwickelt. Die Verwendung von KI in der Softwareentwicklung bietet zahlreiche Vorteile. Erstens beschleunigt KI den gesamten Prozess, da keine manuellen Berechnungen oder wiederholten manuellen Tests erforderlich sind. Zweitens hat KI die Fähigkeit zu lernen und den Prozess im Laufe der Zeit zu verbessern. Darüber hinaus bietet KI erweiterte Analysemöglichkeiten, die es ermöglichen, schwierig zu erkennende Fehler zu finden und in umfassende Analysen zu korrigieren. Schließlich ermöglicht die KI-Optimierung eine Verbesserung der Testqualität und liefert bessere Softwarequalität.

Trotz der vielen Vorteile gibt es auch einige Herausforderungen bei der Verwendung von KI in der Automatisierung von Tests. Diese Systeme müssen auf einer Vielzahl von Szenarien und Datenmengen geschult werden und sie können Schwierigkeiten haben, komplexe oder nuancierte Ideen zu verstehen. Darüber hinaus kann KI anfällig für Bias sein, wenn die Trainingsdaten eine geringe Qualität aufweisen.

Die Integration von KI in die Automatisierung von Tests in der Qualitätssicherung hat den Prozess schneller, effizienter und intelligenter gemacht. Sie bietet sowohl erhebliche Vorteile als auch einige Herausforderungen. Die kontinuierliche Entwicklung und Verbesserung von KI-Technologien wird wahrscheinlich dazu beitragen, diese Herausforderungen zu überwinden und die Qualitätssicherung in der Softwareentwicklung weiter zu revolutionieren [60].

2.5 Aktuelle Tools

2.5.1 ChatGPT und Google Bard

ChatGPT, entwickelt von OpenAI, markiert einen Wendepunkt in der Welt der künstlichen Intelligenz (KI) und der natürlichen Sprachverarbeitung (NLP). Als eines der größten und leistungsfähigsten Sprachmodelle hat es die Fähigkeit, menschenähnliche Antworten auf eine breite Palette von Eingabeaufforderungen zu generieren. ChatGPT verwendet das kostenfreie GPT-3.5-Modell (Generative Pre-trained Transformer, Stand August 2023), bzw. GPT-4,

für zahlende Nutzer, das auf einem umfangreichen Datensatz aus verschiedenen Textquellen im Internet trainiert wurde. Neben ChatGPT hat auch Google den eigenen Chatbot „Bard“ veröffentlicht. Beide Tools haben sich als nützliches Werkzeug in Bereichen wie Übersetzung, Kundenservice und Inhaltsvermittlung erwiesen. Aber auch bei der Unterstützung in der Entwicklung spielen ChatGPT und Bard eine große Rolle [30] [9].

2.5.2 Generative KI für Bilder

Neben textbasierten generativen KI-Modellen haben auch generative KI-Modelle für visuelle Formate erhebliche Fortschritte gemacht, insbesondere in der Erzeugung von Bildern. Diese Technologien haben die Fähigkeit, realistische, Bilder aus einfachen Textaufforderungen zu erzeugen. Dall-E oder Midjourney sind Beispiele für KI-Modelle, die in der Lage sind, Bilder aus Prompts zu generieren. Beide haben Anwendungen in verschiedenen grafischen Bereichen gefunden, von Logo-Design bis zur digitalen Malerei und fotografischen Komposition. 3DALL-E ist eine Erweiterung von Dall-E, die in CAD-Software integriert ist und 2D-Bildinspirationen für 3D-Design generiert. Es ermöglicht Designern, Text- und Bildaufforderungen zu konstruieren, basierend auf dem, was sie modellieren [58] [40]. Die Qualität der Ergebnisse dieser generativen Modelle hat in einigen Kunstbereichen zum Hinterfragen zur Rolle des Menschen bei der Produktion von grafischem Inhalt geführt. Die Integration dieser Modelle in die Visualisierung bietet sowohl Herausforderungen als auch Möglichkeiten [58].

Generative KI für Bilder ist ein aufstrebendes und vielversprechendes Feld, das die Art und Weise, wie wir Bilder erstellen und interpretieren, revolutionieren könnte. Die Entwicklung von Tools wie Dall-E zeigt das Potenzial dieser Technologie, sowohl in künstlerischen als auch in technischen Anwendungen.

2.5.3 UIZard und Autodesigner

UIZard ist ein innovatives Tool, das die Automatisierung im Design von Benutzeroberflächen (UI) fördert. Mit dem Einsatz von deep Learning und künstlicher Intelligenz kann UIZard handgezeichnete Skizzen in funktionierende Codefragmente umwandeln. Der Autodesigner ist eine Erweiterung dieses Tools, die es ermöglicht, UI-Designs aus einfachen Textanweisungen zu generieren [11]. Der UIZard Autodesigner [65] ermöglicht es Benutzern, mehrschichtige Mockups zu erstellen, indem sie Projektideen in einfachen Prompts eingeben. Somit unterstützt er Projektmanager, Vermarkter, Start-up-Gründer und Entwickler dabei, ihre Ideen schneller in die Realität umzusetzen. Jedes Design, das Autodesigner generiert, ist vollständig bearbeitbar und anpassbar innerhalb von UIzards benutzerfreundlichem Editor.

Tools wie UIZard und Autodesigner haben das Potenzial, die Entwicklung von Benutzeroberflächen zu revolutionieren, indem sie Designern und Entwicklern ermöglichen, schnell Prototypen zu erstellen. Dies kann die Zeit und die Kosten für die Entwicklung erheblich reduzieren und gleichzeitig die Kreativität fördern. UIZard und Autodesigner repräsentieren einen Fortschritt in der Welt der UI-Entwicklung. Durch die Kombination von KI und

maschinellern Lernen bieten sie innovative Lösungen für die schnelle Prototyp-Erstellung und könnten die Art und Weise, wie Benutzeroberflächen entwickelt werden, grundlegend verändern.

2.5.4 Github Copilot

GitHub Copilot ist eine KI-gestützte Code-Erstellungshilfe, die von GitHub in Kooperation mit OpenAI entwickelt wurde. Es dient als Co-Programmierer, der Entwicklern Vorschläge für Code-Snippets bietet, basierend auf dem Kontext des aktuellen Repositories, an dem sie arbeiten. Die Technologie hinter Copilot nutzt ein Modell, das auf öffentlich verfügbarem Code trainiert wurde, und generiert diesen in einer probabilistischen Weise, wobei die Wahrscheinlichkeit, dass sie denselben Code wie ein Snippet aus dem Training erzeugt, gering ist. GitHub Copilot ist nicht nur ein Werkzeug zur Code-Generierung, sondern auch ein Partner, der die Fähigkeiten der Entwickler erweitert und ihre Produktivität steigert. Es hat das Potenzial, die Eintrittsbarrieren zu senken und mehr Menschen die Möglichkeit zu geben, die Softwareentwicklung zu erkunden [28].

2.5.5 Der GPT-Engineer

Der GPT-Engineer ist ein innovatives Open-Source-Projekt, das darauf abzielt, den Prozess der Code-Generierung vollständig zu automatisieren. Mit der Fähigkeit, auf Benutzeranweisungen zu reagieren, Fragen zur Klärung zu stellen und dann das gewünschte Projekt zu erstellen, steht es beispielhaft für die Möglichkeiten von KI in der Softwareentwicklung. Die Philosophie hinter dem GPT-Engineer ist es, einen Agenten zu schaffen, der einfach anzupassen, zu erweitern und zu lehren ist, wie der Code aussehen soll. Es generiert eine gesamte Codebasis basierend auf einem Prompt und ermöglicht eine schnelle Übergabe zwischen KI und Mensch. Die Vision des Projekts ist es, eine offene Plattform für Entwickler zu schaffen, um mit ihr zu experimentieren [47].

2.6 Verwandte Arbeiten

2.6.1 Über die Schulter geschaut

In dem Artikel „Über die Schulter geschaut“ wird ein tiefgehender Einblick in die fortschrittlichen Technologien der automatischen Programmierung von Websites und die Entwicklung von Copilot als Weiterentwicklung von GPT-3 gegeben. Die Arbeit beleuchtet insbesondere die Transformation von Layout Beschreibungen in JSX-Code, ein Prozess, der die Erstellung von Webseitenlayouts erheblich vereinfacht. JSX, eine Syntaxerweiterung für JavaScript, ermöglicht es Entwicklern, HTML-ähnliche Tags in ihren JavaScript-Code einzufügen, was die Entwicklung von Benutzeroberflächen erleichtert.

Copilot, als eine Weiterentwicklung von GPT-3, wurde speziell auf Kommentaren und Code trainiert (Tuning), um eine präzisere Generierung von Code zu ermöglichen. Diese spezialisierte Schulung ermöglicht es Copilot, Entwicklern bei der Programmierung zu as-

sistieren, indem es Vorschläge basierend auf dem Kontext des geschriebenen Codes macht. GitHub, das Unternehmen hinter Copilot, betont jedoch, dass Entwickler den von Copilot generierten Code nochmals korrekturlesen sollten. Dies unterstreicht die Tatsache, dass die Technologie sich hauptsächlich auf kleinere Codeabschnitte beschränkt, die mit Docstrings oder Kommentaren erzeugt werden, und nicht als Ersatz für menschliche Überprüfung und Intuition dienen kann. Die Erfolgsrate von Copilot, die richtige Lösung vorzuschlagen, liegt laut Artikel bei 43%. Obwohl dies ein bemerkenswerter Fortschritt in der automatischen Codegenerierung darstellt, weist es auch auf die Grenzen der Technologie hin und betont die Notwendigkeit menschlicher Überwachung und Intervention. Der Artikel bietet auch einen umfassenden Überblick über Grundbegriffe wie GPT, Zero-, One- und Fewshot Learning. GPT steht dabei für „Generative Pre-trained Transformer“ und ist ein large language Model mit sogenannter Transformer Architektur, welches auf maschinellem Lernen basiert. Zero-, One- und Fewshot Learning beziehen sich auf verschiedene Ansätze des maschinellen Lernens, bei denen Modelle mit keiner, einer oder sehr wenigen Beispielen trainiert werden.

Trotz dieser detaillierten Analyse fehlt es dem Artikel an Betrachtungen industriellen Maßstabs und einer tiefgehenden Untersuchung der wirklichen Automatisierung von Aufgaben. Dies wirft Fragen nach der praktischen Anwendbarkeit und Skalierbarkeit dieser Technologien in einem breiteren industriellen Kontext auf, insbesondere in Bezug auf die Integration in bestehende Arbeitsabläufe und Systeme. Insgesamt bietet der Artikel eine Perspektive auf die sich rasch entwickelnde Landschaft der automatischen Codegenerierung, betont jedoch die Notwendigkeit weiterer Forschung und Entwicklung, um die volle Tragweite und das Potenzial dieser Technologien zu realisieren. Die Balance zwischen Automatisierung und menschlicher Kontrolle, die Skalierbarkeit der Technologie und die ethischen Überlegungen bei der Implementierung sind Themen, die weiterer Untersuchung bedürfen, um einen ganzheitlichen Blick auf dieses sich ständig weiterentwickelnde Feld zu gewährleisten [25].

2.6.2 Yes, You Can Make an App Too: A Systematic Study of Prompt Engineering in the Automatic Generation of Mobile Applications from User Queries

In der Abhandlung „Yes, You Can Make an App Too: A Systematic Study of Prompt Engineering in the Automatic Generation of Mobile Applications from User Queries“ wird die Vision verfolgt, dass jeder, der eine Idee für eine App hat, auch die Möglichkeit haben sollte, diese in die Tat umzusetzen. Die Studie erkundet bestehende Ansätze wie Low-Code oder No-Code-Plattformen und hebt den MIT App Inventor hervor, der eine App mit einem Prompt erstellen soll, wodurch die technische Barriere entfernt wird.

Die Abhandlung beschreibt detailliert die Pipeline, mit der ein Prompt durch Few-Shot Learning mit Informationen angereichert wird, um Kontext zu schaffen, und untersucht die Fehler im Code mit zunehmender Länge des Prompts. Das Ziel ist es, der breiten Bevölkerung Zugang zur automatischen App-Generierung zu ermöglichen.

Allerdings ergibt sich aus der Arbeit die Schlussfolgerung, dass der Nutzer an das Tool gebunden ist, da lediglich der Mechanismus des Tools erklärt wird. Dies könnte die wahre Autonomie und das Verständnis des Benutzers für den gesamten Prozess begrenzen. Die Herausforderung besteht darin, bei der Herleitung von der Erstellung von Web-Apps und Seiten die Vorgehensweise durch öffentlich zugängliche Tools so darzustellen, dass die Toolchain nachgebildet werden kann. Dies würde eine tiefere Einsicht und Kontrolle über den Entwicklungsprozess ermöglichen. Insgesamt stellt die Abhandlung einen wichtigen Schritt in Richtung einer inklusiveren und zugänglicheren Technologielandschaft dar, hebt jedoch auch die Notwendigkeit hervor, über vorhandene Tools und Plattformen hinauszugehen, um eine echte Autonomie und Verständnis in der App-Entwicklung zu fördern [59].

2.6.3 Künstliche Intelligenz in der Softwareentwicklung

In der Abhandlung „Künstliche Intelligenz in der Softwareentwicklung“ wird ein umfassender Überblick über die Integration von künstlicher Intelligenz (KI) in den Bereich der Softwareentwicklung gegeben. Die Arbeit beleuchtet grundlegende Begriffe wie KI, natural language Processing (NLP) und natural language Recognition (NLR) und stellt deren Relevanz und Schnittpunkte mit der Softwareentwicklung dar. Angesichts der wachsenden Komplexität in der Softwareentwicklung wird betont, dass effizientere und automatisierte Prozesse unerlässlich sind. Daher wird die Notwendigkeit hervorgehoben, verstärkt auf KI-Technologien zu setzen, um diesen Anforderungen gerecht zu werden.

Ein zentrales Thema der Arbeit ist die noch unklare Definition der Umsetzungsszenarien von KI in der Softwareentwicklung. Während es bereits sehr spezifische KI-Systeme gibt, fehlen allgemeingültige Definitionen und Richtlinien, wie KI in den gesamten Softwareentwicklungsprozess integriert werden kann. Dieser Schnittpunkt zur Arbeit soll in zukünftigen Untersuchungen weiter vertieft und angeknüpft werden.

Ein weiterer wichtiger Aspekt, der in der Abhandlung hervorgehoben wird, ist die potenzielle Verwendung von KI zur Erkennung von Sicherheitsmängeln in Softwareanwendungen.

Obwohl KI das Potenzial hat, Sicherheitslücken zu identifizieren, ist die tatsächliche Fehlererkennung noch unzureichend. Die These der Arbeit besagt, dass es an ausreichenden Daten mit genau gekennzeichneten Sicherheitsmängeln für das Training der KI-Modelle mangelt. Dies unterstreicht die Notwendigkeit, umfangreiche und präzise Datenbanken zu erstellen, um die Effizienz von KI in der Erkennung von Software-Sicherheitsmängeln zu verbessern.

Insgesamt bietet die Abhandlung einen tiefen Einblick in die Herausforderungen und Möglichkeiten der Integration von KI in die Softwareentwicklung und legt den Grundstein für zukünftige Forschungen in diesem Bereich [10].

2.6.4 How Artificial Intelligence Can Improve Web Development and Testing

In der Arbeit „How Artificial Intelligence Can Improve Web Development and Testing“ wird eine gründliche Untersuchung der Rolle von künstlicher Intelligenz, maschinellem Lernen und deep Learning in der Webentwicklung vorgenommen. Die Autoren ordnen diese Technologien sorgfältig ein und erörtern, wie sie in verschiedenen Aspekten der Webentwicklung eingesetzt werden können.

Ein innovativer Ansatz, der in der Arbeit hervorgehoben wird, ist die Verwendung von KI im User Experience (UX) Design. Die Autoren beschreiben, wie KI Screenshots von sogenannten Scribbles in HTML-Wireframes übersetzen kann, was den Designprozess erheblich beschleunigt und vereinfacht. Darüber hinaus wird betont, dass KI auch im Bereich der Barrierefreiheit (Accessibility) helfen kann. Durch die automatische Übersetzung eines UI-Konzepts in Code können alternative Tags und andere barrierefreie Funktionen automatisch gesetzt werden, wodurch die Zugänglichkeit von Webanwendungen verbessert wird.

Ein weiterer wichtiger Aspekt der Arbeit ist die automatische Testgenerierung durch KI. Die Autoren argumentieren, dass diese Technologie die Sicherheit und Fehlerquote von Code verbessern kann, indem sie umfassende und präzise Tests durchführt, die menschliche Tester möglicherweise übersehen könnten.

Trotz der vielversprechenden Möglichkeiten, die KI in der Webentwicklung bietet, weist die Arbeit auch auf die Notwendigkeit hin, zwischen der Effizienz, die KI mit sich bringt, und den Kosten, die zum Trainieren der Modelle aufgebracht werden müssen, abzuwägen. Diese Abwägung unterstreicht die Komplexität der Implementierung von KI in der Webentwicklung und betont die Notwendigkeit, sowohl die technischen als auch die wirtschaftlichen Aspekte zu berücksichtigen.

Insgesamt bietet „How Artificial Intelligence Can Improve Web Development and Testing“ einen umfassenden und aufschlussreichen Überblick über die vielfältigen Möglichkeiten, wie KI die Webentwicklung revolutionieren kann. Die Arbeit legt einen klaren Weg für zukünftige Forschungen und Entwicklungen in diesem sich schnell entwickelnden Bereich dar [61].

2.6.5 AI for Agile development: a Meta-Analysis

In der Analyse „AI for Agile Development: a Meta-Analysis“ wird die zunehmende Bedeutung von künstlicher Intelligenz (KI) im Kontext der agilen Softwareentwicklung untersucht. Die Arbeit beginnt mit der Feststellung, dass Software heutzutage schneller veröffentlicht werden muss, um den sich rasch verändernden Marktbedingungen gerecht zu werden, wobei die Qualität nicht beeinträchtigt werden darf.

Die Autoren argumentieren, dass KI im Projektmanagement eine entscheidende Rolle spielen kann, indem sie eine Reihe von Funktionen unterstützt. Dazu gehören die Aufgabenverteilung, bei der KI die Stärken der Teammitglieder analysieren und Aufgaben entsprechend zuweisen kann, sowie die Code-Analyse und automatische Testfallgenerierung. Innerhalb dieses Bereichs wird die Priorisierung von Testfällen hervorgehoben, bei der KI entscheiden kann, welcher Teil des Codes inspiziert werden sollte. Weitere Anwendungen von KI in der agilen Entwicklung umfassen das Tracking der Teamleistung, die Priorisierung des Backlogs und das Schätzen von User Stories.

Trotz dieser vielversprechenden Anwendungen weist die Arbeit darauf hin, dass die Implementierung von KI im agilen Entwicklungsprozess nicht ohne Herausforderungen ist. Es wird betont, dass AI Model Management erforderlich ist, wofür technische Experten benötigt werden. Darüber hinaus gibt es Sicherheitsbedenken bei der Bereitstellung von Software, die deep Learning Networks und maschinelles Lernen verwendet, da die Daten ein hohes Bias aufweisen können. Dies unterstreicht die Notwendigkeit, sorgfältige Überlegungen und Kontrollen bei der Implementierung von KI in der Softwareentwicklung anzustellen.

Schließlich betont die Meta-Analyse, dass insgesamt mehr spezialisierte technische Experten für den Einsatz und die Integration von KI in die agile Softwareentwicklung benötigt werden. Die Arbeit bietet einen umfassenden Überblick über die Möglichkeiten und Herausforderungen von KI in der agilen Entwicklung und legt eine solide Grundlage für zukünftige Forschungen und Praktiken in diesem aufstrebenden und wichtigen Bereich [14].

2.6.6 A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT

Die Arbeit „A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT“ stellt eine Herangehensweise an die Interaktion mit large language Models (LLMs) wie ChatGPT vor. Sie führt das Konzept der Prompt Patterns ein, die als Alternative zu traditionellen Software-Patterns dienen und als wiederverwendbare Lösungen für spezifische Probleme fungieren. Die Autoren beschäftigen sich intensiv mit den vielfältigen Möglichkeiten, effektiv mit ChatGPT zu arbeiten, und teilen Prompt Patterns in verschiedene Kategorien ein. Zu diesen Kategorien gehören:

1. Input Semantics: Wie das LLM den Input versteht und interpretiert.
2. Output Customization: Hier nimmt das LLM eine bestimmte Rolle ein, um eine spezifische Ausgabe zu kreieren, wobei der Nutzer ein Template definiert, das das LLM mit Inhalt füllt.
3. Error Identification: Das LLM generiert eine Faktenliste zur Überprüfung und soll die eigene Antwort introspektieren, um Fehler zu erkennen.
4. Prompt Improvement: Das LLM schlägt eine bessere Variante der Frage vor oder generiert Unterfragen, die zunächst beantwortet werden müssen. Es kann auch die Frage umformulieren, wenn keine Antwort gefunden wird.

5. Interaction Category: Hier werden Rückfragen anstelle einer Antwort gestellt, oder das LLM generiert kontinuierlich eine Ausgabe, ohne dass der Nutzer den Prompt jedes Mal neu eingeben muss.
6. Context Control: Der Nutzer spezifiziert einen Kontext für die Ausgabe, um die Relevanz und Genauigkeit zu steuern.

Die Arbeit betont, dass solche Prompt Patterns die Ausgabequalität von LLMs drastisch verbessern können. Sie bieten eine strukturierte und effiziente Methode, um die Interaktion mit LLMs zu steuern und zu optimieren. Darüber hinaus stellt die Arbeit fest, dass diese Prompt Patterns auch als Grundlage für die methodische Umsetzung in anderen Forschungsarbeiten dienen können. Sie bieten ein solides Fundament, auf dem aufbauend spezifische Anwendungen und Techniken entwickelt werden können.

Insgesamt bietet „A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT“ einen Beitrag zur Weiterentwicklung der Interaktion mit LLMs. Durch die Einführung und Kategorisierung von Prompt Patterns trägt sie dazu bei, die Effizienz und Qualität der Kommunikation mit diesen fortschrittlichen Modellen zu steigern und öffnet neue Wege für die Forschung und Anwendung in diesem Bereich [82].

2.6.7 Interactive and Visual Prompt Engineering for Ad-hoc Task Adaptation with Large Language Models

Die Arbeit „Interactive and Visual Prompt Engineering for Ad-hoc Task Adaptation with Large Language Models“ konzentriert sich auf die Untersuchung unterschiedlicher Prompts, insbesondere auf eine Visualisierungsmethode, mit der verschiedene Prompts auf ihre Effizienz getestet werden können. Dieses innovative Konzept wird als PromptIDE bezeichnet. Damit wird es ermöglicht, anhand einer Quelle und möglichen Fragen und Antworten die Ausgaben zu bewerten und zu analysieren, wie gut ein Prompt funktioniert. Die Visualisierung spielt dabei eine Schlüsselrolle, da sie dem Endnutzer helfen kann, zu verstehen, wie Transformer die Sprache mittels natural language Processing (NLP) übersetzen und verarbeiten. Dies bietet eine intuitive und anschauliche Darstellung der komplexen Mechanismen, die in einem language Model ablaufen.

Ein zentraler Aspekt der Arbeit ist die Interaktion des Nutzers mit den Prompts. Der Nutzer sollte Feedback anhand der Daten zu den Prompts erhalten und die Möglichkeit haben, die Prompts basierend auf seinem Gebietswissen zu bearbeiten. Dies fördert eine engere Verbindung zwischen dem Nutzer und dem Modell und ermöglicht eine feinere Kontrolle und Anpassung der Prompts.

Darüber hinaus führt die Arbeit das Konzept der quantitativen Bewertung anhand von Prompt Scores im Tool ein. Diese Bewertung bietet eine messbare und objektive Methode, um die Effektivität eines Prompts zu beurteilen. Es ermöglicht eine systematische Analyse und Vergleich verschiedener Prompts und trägt dazu bei, die Qualität und Relevanz der

Ausgaben zu steigern.

Insgesamt bietet „Interactive and Visual Prompt Engineering for Ad-hoc Task Adaptation with large language Models“ einen bedeutenden Beitrag zur Weiterentwicklung der Prompt-Technik in der Interaktion mit großen Sprachmodellen. Durch die Einführung von PromptIDE und die Betonung der Visualisierung und Interaktion trägt die Arbeit dazu bei, die Verständlichkeit und Kontrolle über die Modelle zu erhöhen und öffnet neue Wege für die Forschung und Anwendung in diesem sich rasch entwickelnden Bereich [62].

2.7 Warum es weiterer Behandlung bedarf

Die rasanten Fortschritte in der künstlichen Intelligenz (KI) haben das Potenzial, die Webentwicklung grundlegend zu verändern. Von der Automatisierung von Tests bis zur Generierung von Code bieten KI-Technologien eine Fülle von Möglichkeiten, die Effizienz und Qualität der Webentwicklung zu steigern. Trotz dieser vielversprechenden Entwicklungen zeigt eine sorgfältige Analyse der bestehenden wissenschaftlichen Arbeiten auf diesem Gebiet, dass es noch erhebliche Lücken und Herausforderungen gibt, die angegangen werden müssen. Die folgenden Punkte beleuchten diese Lücken und begründen die Notwendigkeit einer eingehenden Untersuchung in dieser Bachelorarbeit, um ein vollständigeres Verständnis der Grenzen und Möglichkeiten von KI in der Webentwicklung zu erlangen:

1. Fehlende Toolchain für größere Projekte: Die bisherigen Arbeiten haben sich weitgehend auf spezifische Aspekte oder kleinere Anwendungen von KI in der Webentwicklung konzentriert. Es fehlt jedoch die klare Vorstellung einer Toolchain, die die Implementierung und Verwaltung größerer und komplexerer Projekte ermöglicht. Dies stellt eine erhebliche Lücke dar, da die Fähigkeit, KI in umfangreichen Projekten effektiv einzusetzen, entscheidend für die industrielle Anwendbarkeit ist.
2. Geringe Untersuchung der Grenzen: Obwohl Themen wie Testing, Patterns und Prompt Engineering in Arbeiten wie „A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT“ gut behandelt wurden, wurde die Untersuchung der Grenzen und Einschränkungen von KI in der Webentwicklung weitgehend vernachlässigt. Dies lässt eine wichtige Dimension unerforscht, die für ein vollständiges Verständnis der Möglichkeiten und Herausforderungen von KI in diesem Bereich unerlässlich ist. Die vorhandenen Arbeiten zielen mehr auf den effizienten und „richtigen“ Einsatz ab, nicht aber auf das Ausreizen der Grenzen.
3. Fehlender Ansatz für größere Projekte: Die Abwesenheit eines kohärenten Ansatzes, der sowohl Prompt Engineering als auch ein Framework für größere Projekte kombiniert, zeigt, dass die Skalierbarkeit und Anwendbarkeit von KI in der Webentwicklung noch nicht ausreichend adressiert wurden.
4. Fehlendes Grundlagenkapitel: Das Fehlen eines Grundlagenkapitels in vielen Arbeiten erschwert den Einstieg für Neulinge und begrenzt möglicherweise den Zugang zu die-

sen komplexen Themen. Dies könnte die Verbreitung und Akzeptanz von KI in der Webentwicklung hemmen.

5. Veraltete Informationen: Die Tatsache, dass viele Arbeiten veraltet sind und mehr von Perspektiven sprechen, unterstreicht die Notwendigkeit einer aktualisierten und zeitgemäßen Analyse der Rolle von KI in der Webentwicklung.
6. Fehlende klare Handlungsanweisungen für industriellen Standard: Die Abwesenheit klarer Handlungsanweisungen oder Leitlinien für den Bau von Projekten von industriellem Standard zeigt, dass die Forschung noch nicht ausreichend mit den praktischen und industriellen Anforderungen verknüpft ist.

Zusammenfassend zeigen die analysierten Arbeiten die Notwendigkeit auf, diese Lücken zu schließen und eine umfassendere und praxisorientierte Herangehensweise an die Rolle von KI in der Webentwicklung zu entwickeln. Diese Bachelorarbeit zielt darauf ab, die Lücken zu adressieren und einen Beitrag zur Weiterentwicklung dieses wichtigen und sich rasch entwickelnden Feldes zu leisten. Durch die Begründung und Untersuchung dieser spezifischen Bereiche bietet die Arbeit eine fundierte Grundlage für zukünftige Forschungen und Anwendungen und trägt dazu bei, die Verbindung zwischen theoretischer Forschung und praktischer Anwendung in der Webentwicklung mit KI zu stärken.

Kapitel 3

Grundlagen und Begriffsdefinitionen

3.1 Künstliche Intelligenz und Artificial Intelligence

Künstliche Intelligenz (KI) und Artificial Intelligence (AI) sind zwei Begriffe, die synonym verwendet werden und eine Technologie beschreiben, die in der Informatik und Datenverarbeitung eine immer größer werdende und zentrale Rolle spielt. Während „KI“ die deutsche Bezeichnung ist, wird der Begriff „AI“ im englischsprachigen Raum verwendet [66].

Die KI-Technologie ermöglicht es Computern, menschenähnliches Handeln nachzuahmen. Dies reicht von „einfachen“ Aufgaben wie Gruppierungsverfahren, anhand derer bestimmte statistische Vorhersagen getroffen werden können, bis hin zu komplexen Transformatoren, die den Anschein einer dem Menschen übergeordneten Intelligenz in bestimmten Gebieten wie der Textverarbeitung erwecken können. Die Herausforderung dabei ist, dass es keine eindeutige Definition für menschliche Intelligenz gibt [27]. Dies macht die Nachahmung menschlicher Intelligenz durch Maschinen zu einem komplexen Thema. Durch die fehlenden Definitionen wird immer häufiger in der Gesellschaft falsch eingeschätzt, welche Rolle durch KI übernommen werden kann und wird und wo sich aktuell noch deutliche Grenzen befinden [17].

Das Ziel der KI-Forschung ist es, menschliches Verhalten so genau wie möglich nachzuahmen oder sogar zu übertreffen [73]. Nach aktuellem Stand sind existierende Modelle, künstlicher Intelligenz allerdings mehr darauf ausgelegt, den Menschen in spezifischen Bereichen zu unterstützen und nicht darauf, ihn allgemein ersetzen zu wollen. Dabei wird nicht nur versucht, die kognitive Leistungsfähigkeit des Menschen zu replizieren, sondern es werden auch emotionale und soziale Aspekte in Betracht gezogen. Dies hat den Hintergrund, dass durch KI, insbesondere in Form von Chatbots, falsche bis hin zu vorgefärbten oder rassistischen Informationen in das Internet gelangen können [18]. Die Forschung in diesem Bereich ist dynamisch und entwickelt sich ständig weiter, wobei die ethischen Implikationen und die Auswirkungen auf die Gesellschaft aus diesem Grund immer stärker in den Mittelpunkt gerückt werden.

Die künstliche Intelligenz stellt eine herausfordernde Technologie dar. Die Nachahmung menschlicher Intelligenz durch Maschinen öffnet Türen zu neuen Möglichkeiten, wirft aber auch Fragen und Probleme auf, die in der Wissenschaft und Gesellschaft sorgfältig betrachtet werden müssen.

3.1.1 Starke und Schwache KI

In der Forschung und Entwicklung von künstlicher Intelligenz (KI) wird häufig zwischen zwei Hauptkategorien unterschieden: schwache KI (auch als „Narrow AI“ bezeichnet) [78] und Starke KI (auch als „General oder Strong AI“ bekannt) [78]. Im Folgenden wird kurz über die beiden Begriffe aufgeklärt.

Schwache KI (Narrow AI)

Schwache KI bezieht sich auf Computersysteme, die darauf ausgelegt sind, spezifische Aufgaben zu erledigen, ohne das volle Spektrum menschlicher Intelligenz nachzuahmen [83]. Diese Art von KI ist in der Regel auf bestimmte Bereiche wie die natürliche Sprachverarbeitung (NLP) oder die Computer Vision spezialisiert. Auch wenn es auf den ersten Blick nicht so erscheint, sind auch Chatbots in der Kategorie schwacher KIs einzuordnen. Tools wie ChatGPT lassen schnell den Eindruck erwecken, dass sie bereits einige Aufgaben besser umsetzen können, als es der Mensch könnte. Trotz der vielen Einsatzmöglichkeiten solcher Chatbots, kann die Technologie darauf heruntergebrochen werden, dass natürliche Sprache verarbeitet wird und die Ausgabe durch ein komplexes neuronales Netzwerk erzeugt wird, die am wahrscheinlichsten zu der ursprünglichen Anfrage passt [24]. Weitere Einsätze für schwache KIs befinden sich in der Gesichtserkennung oder selbstfahrenden Fahrzeugen, die durch Kameras, Sensoren und Algorithmen den Anschein einer Intelligenz erwecken können [83].

Starke KI (General AI)

Im Gegensatz dazu zielt starke KI darauf ab, jegliche Aufgaben eines Menschen zu übernehmen. Es handelt sich um ein verallgemeinertes Modell, das in der Theorie nicht vom Menschen unterscheidbar sein sollte. Die Idee der Starken KI geht über die bloße Nachahmung spezifischer menschlicher Fähigkeiten hinaus und strebt an, die gesamte Bandbreite menschlicher Intelligenz zu replizieren [81]. Bis heute ist es jedoch nicht bewiesen, ob eine solche Intelligenz tatsächlich technisch umsetzbar ist. Die Entwicklung einer starken KI stellt eine enorme wissenschaftliche und technologische Herausforderung dar und wirft auch ernsthafte ethische und philosophische Fragen auf [81]. Für weitere Informationen zu diesem Thema können sowohl der Turing-Test als auch das sogenannte „Chinesische Zimmer“ als Anhaltspunkte verwendet werden, welche oft als Beweise dafür genommen werden, dass eine solche generelle künstliche Intelligenz nicht in der Realität umsetzbar ist. Die Unterscheidung zwischen starker und schwacher KI ist fundamental für das Verständnis der Möglichkeiten und Grenzen der KI-Technologie. Während schwache KI bereits in vielen Bereichen unseres Lebens Anwendung findet, bleibt starke KI ein faszinierendes, aber noch unerreichbares Ziel. Die Fortschritte werden jedoch weiterhin die Art und Weise prägen, wie wir Technologie

nutzen und verstehen, und haben das Potenzial, unsere Gesellschaft in vielfältiger Weise zu beeinflussen.

3.2 Machine Learning und deep Learning

Maschinelles und tiefes Lernen (machine und deep Learning) sind Begriffe die ebenfalls oft miteinander verwechselt oder gleichgesetzt werden. Auch wenn die Teilgebiete miteinander verwandt sind und aufeinander aufbauen, handelt es sich dennoch um unterschiedliche Forschungsgebiete.

3.2.1 Machine Learning (ML)

Machine Learning (ML) ist ein zentrales Teilgebiet der künstlichen Intelligenz (KI). Es befasst sich mit der Entwicklung von Algorithmen und Datenstrukturen, die Computern ermöglichen, aus Daten zu lernen und Vorhersagen oder Entscheidungen zu treffen. Machine Learning stellt somit die Grundlage für deep Learning dar [76]. Mittlerweile haben sich verschiedene Arten, maschinellen Lernens entwickelt, die alle ihre Vor- und Nachteile mit sich bringen. Insbesondere im populären Bereich der natürlichen Sprachverarbeitung mit Tools wie ChatGPT, Google Bard und anderen Vertretern wird fast ausschließlich eine Mischung aus verschiedenen Formen eingesetzt, da die Masse an Daten schlichtweg zu groß ist, um mit reinem supervised Learning zu trainieren [75]. Auf diese Arten wird in den folgenden Abschnitten näher eingegangen.

3.2.2 Deep Learning (DL)

Deep Learning ist ein spezialisiertes Teilgebiet des machine Learning und basiert auf neuronalen Netzwerken. Es kann den menschlichen Eingriff drastisch reduzieren und macht das machine Learning wesentlich skalierbarer [74]. Häufig ist bei deep Learning von Deep Neural Networks die Rede. Ein neuronales Netzwerk wird dann als „Deep“ bezeichnet, wenn es mehr als drei Schichten besitzt, einschließlich Input- und Output-Layer. Diese Netzwerke sind in der Lage, komplexe Muster in Daten zu erkennen. Der grundlegende Aufbau eines neuronalen Netzwerks wird in einem späteren Kapitel noch einmal erläutert. Es ist jedoch anzunehmen, dass nahezu alle heutzutage entwickelten deep Learning Modelle auf Tiefen NN basieren [74].

Wie bereits beschreiben, hat deep Learning den großen Vorteil mit Daten ohne menschliches Eingreifen zu arbeiten, z.B. um diese zu clustern. Dies ermöglicht eine effizientere und automatisierte Analyse. Außerdem können Muster in Daten erkannt werden, die für den Menschen kaum bis gar nicht erkannt werden können [20].

3.2.3 Fazit

Sowohl machine Learning als auch deep Learning sind Schlüsseltechnologien in der modernen KI-Forschung und -Anwendung. Während ML eine breite Palette von Techniken und Ansätzen umfasst, konzentriert sich DL auf die Nutzung tiefer neuronaler Netzwerke. Beide

Gebiete tragen dazu bei, die Grenzen dessen, was mit KI erreicht werden kann, ständig zu erweitern, und bieten Möglichkeiten für Innovationen in vielen verschiedenen Bereichen.

3.3 Supervised, unsupervised und reinforcement Learning

Bei diesen Begriffen handelt es sich um Art und Weisen, wie ML-Modelle trainiert werden. Supervised Learning (SL) arbeitet mit sogenannten gelabelten Daten und erfordert menschliches Zutun. Es ist häufig akkurater als unsupervised Learning, benötigt jedoch mehr Zeit für die Datenaufbereitung [84]. Daher muss zunächst immer abgewägt werden, welche Art des Trainings für ein ML-Modell gewählt wird.

SL wird unter anderem zur Klassifizierung eingesetzt. Zum Beispiel kann durch die Zuordnung von diskreten Labels, z.B. Spam oder Nicht-Spam-Mail ein Spamfilter dazu trainiert werden, Spam-Mail zukünftig besser und schneller zu erkennen [20].

Unsupervised Learning (UL) oder unüberwachtes Lernen arbeitet hingegen mit nicht gelabelten Daten und kommt ohne menschliches Zutun aus. Dadurch ist es, wie im letzten Abschnitt beschreiben, wesentlich besser skalierbar. Mit UL können Regelmäßigkeiten entdeckt werden, die für den Menschen auf den ersten Blick nicht erkennbar sind, da ein Datensatz nicht „benannt“ werden muss [44]. Als Beispiel lässt sich hier die Kundensegmentierung nennen. Im großen Maße verwenden E-Commerce Anbieter dieses Verfahren, um Kunden in Kategorien einzuteilen und das Kaufverhalten zu analysieren. Dabei lässt sich unüberwachtes Lernen gut einsetzen, da die Zusammenhänge zwischen einzelnen Käufen und Produkten durch den Menschen oft nicht erkennbar sind [44].

Im Gegensatz zu überwachtem Lernen (supervised Learning) validiert das Reinforcement Learning nicht zwangsläufig jedes fehlerhafte Ergebnis. Das Modell befindet sich in einem Umfeld, welches nur unter bestimmten Bedingungen Rückmeldung gibt. Wenn das Ergebnis zu 95% der erwarteten Ausgabe entspricht, wären die Verbesserungen durch das Feedback zu marginal. Das bedeutet, dass das Modell nur dann ein Feedback erhält, wenn die Fehlerquote oder die Abweichung zu hoch ist [22]. Reinforcement Learning ist besonders nützlich bei vielen Daten oder wenn es schwierig ist, die Daten klar zu labeln und verbindet die Genauigkeit von überwachtem Lernen und die nicht überwachten Daten von unsupervised Learning. Allerdings ist Reinforcement Learning nur dann sinnvoll, wenn das zu lösende Problem recht komplex ist und mit UL oder SL nicht gelöst werden kann, da die Rechenkosten sehr hoch ausfallen [39].

Supervised, unsupervised und Reinforcement Learning sind drei Hauptkategorien des machine Learnings, die jeweils für unterschiedliche Ansätze und Anwendungsgebiete verwendet werden können. Während supervised Learning menschliches Eingreifen erfordert und sich auf klare Labels stützt, ermöglicht unsupervised Learning das Entdecken von Mustern ohne menschliche Hilfe. Reinforcement Learning bietet einen flexibleren Ansatz, der bei komple-

nen Datensätzen nützlich sein kann. Die Wahl des richtigen Ansatzes hängt somit von den spezifischen Anforderungen und Daten ab, mit denen gearbeitet wird.

3.4 Neuronale Netzwerke

Neuronale Netzwerke (NN) sind ein zentraler Bestandteil des deep Learnings und versuchen, die Arbeitsweise des menschlichen Gehirns nachzuahmen. Sie sind komplex aufgebaut und vielseitig einsetzbar. Im Folgenden werden die wichtigsten Aspekte von neuronalen Netzwerken erläutert [22].

3.4.1 Struktur und Aufbau

Ein NN besteht aus verschiedenen Schichten, darunter eine Input-Schicht, eine oder mehrere Hidden-Schichten und eine Output-Schicht. Jede Schicht enthält Neuronen, die miteinander verbunden sind. Die Input-Schicht erhält eine Eingabe, bzw. einen Datensatz. Die versteckten Schichten führen Berechnung durch. Die Output-Schicht gibt letztlich den Ausgabewert oder die Antwort aus [71]. Die Neuronen sind die grundlegenden Verarbeitungseinheiten eines NN und werden durch Kanten und Gewichte miteinander verbunden [26]. Gewichte bestimmen die Bedeutung eines Input-Features und spielen eine entscheidende Rolle bei der Minimierung des Fehlers in der Ausgabe. Ein Feature ist eine spezifische Eigenschaft eines Sachverhalts, wie z.B. die Marke eines Autos oder die durchschnittliche Herzrate eines Menschen [49]. Ein NN, welches durch eine lineare Regression abgebildet wird ($y = ax + b$), würde die Parameter a und b zu Beginn initialisieren und stetig anpassen [6].

3.4.2 Bias, Overfitting und Underfitting

Der Bias ist ein zusätzlicher Parameter im NN, der die Genauigkeit des Ergebnisses erhöhen kann. Er verschiebt die Funktion entlang der x-Achse ins positive oder negative, um die Daten besser abbilden zu können [33]. Im Beispiel der linearen Regression ($y = ax + b$) würde der Bias durch die Variable b abgebildet werden. Aus dieser Verschiebung kann es zu Over- und Underfitting kommen. Wenn der Bias zu hoch korrigiert wird, kann das NN den Zusammenhang zwischen Trainings- und Testdaten nicht klar referenzieren, was zu Underfitting führt [31]. Die Folge sind schlechte Ausgaben, die daraus resultieren, dass das Modell nicht stark genug trainiert wurde. Das Gegenteil wird durch Overfitting dargestellt. Wenn das Modell zu komplex wird und der Bias zu niedrig korrigiert wird, können zwar die Testdaten sehr genau abgegrenzt werden, allerdings führt das Verarbeiten neuer Daten zu schlechten und ungenauen Ausgaben. Die Anzahl der Hidden-Layer und Neuronen muss daher sorgfältig ausgewählt werden, um ein Gleichgewicht zwischen Effizienz und Genauigkeit zu finden [57].

3.4.3 Typen und Funktionen von neuronalen Netzwerken

Ein Typ von NN ist der Feed Forward Typ (3.1). Bei diesem Typ können die Daten nur von der Input-Schicht zur Output-Schicht fließen. Mithilfe von Backpropagation wird es

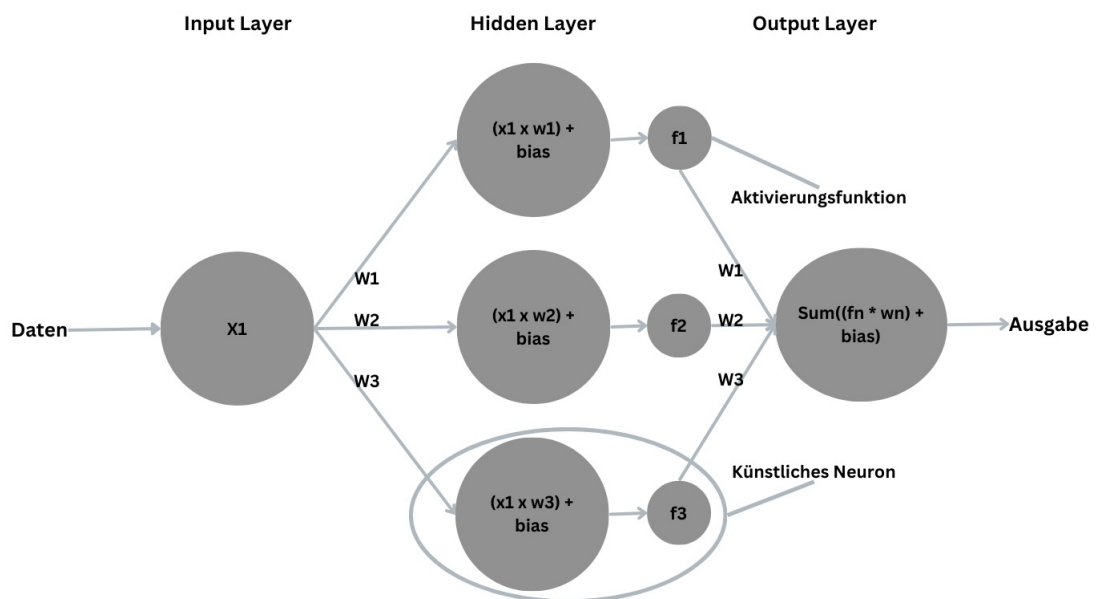


Abbildung 3.1: Schematische Darstellung eines 3-Schichtigen NN

ermöglicht, dass Daten nicht nur von links nach rechts, sondern auch von rechts nach links fließen. Dadurch können Gewichte und Parameter basierend auf der Ausgabe eines zukünftigen Neurons aktualisiert werden [35].

Neuronale Netzwerke sind ein komplexes Gebiet der künstlichen Intelligenz. Ihre Struktur, Funktionsweise und Anwendungen sind vielfältig und bieten ein enormes Potenzial für Innovationen in verschiedenen Bereichen. Die kontinuierliche Erforschung und Verbesserung von NN wird dazu beitragen, die Möglichkeiten der KI zu erweitern.

3.5 Generative KI

3.5.1 Foundation Models

Foundation Models (FM) sind ein aufstrebendes Gebiet innerhalb der generativen künstlichen Intelligenz. Ein FM wird unsupervised trainiert, wobei es sogenannte self-supervised Daten verwendet. Self-supervised Learning versucht, supervised Learning nachzuahmen, indem das Modell anhand von verarbeiteten Daten selbst entscheidet, wozu eine neue Eingabe gehört. Dabei werden die unsupervised Daten automatisch zu supervised Daten transferiert, indem das Modell den Daten selbst Labels zuweist [70].

Ein FM kann auch mit gelabelten Daten konfrontiert und getunt werden, um für spezifische Aufgaben wie Classification eingesetzt werden zu können. Dies eröffnet eine Reihe von Vorteilen, insbesondere für Unternehmen, die nicht die Größe besitzen, ein eigenes Foundationmodel trainieren zu können [55]. Durch die Fähigkeit, auf alle bereits gesehenen Daten

zuzugreifen, erhöht sich die Effizienz, Genauigkeit und Ausgabequalität. Dies erhöht auch die Wiederverwendbarkeit, indem das FM mit unternehmenseigenen Daten fine-getunt wird, um spezifische Aufgaben lösen zu können. Mit Fine-Tuning kann ein FM mit einer kleinen Menge an supervised Daten einen sehr spezifischen Task schnell ausführen.

Trotz dieser Vorteile gibt es auch einige Herausforderungen und Nachteile, die entstehen. Zum einen ist das Einführen vieler Daten sehr teuer und resultiert in hohen Trainings- und Betriebskosten, um erst einmal ein FM trainieren zu können. Zum anderen besteht insbesondere im Sprachbereich das Risiko, dass die KI nicht vertrauenswürdig ist und Falschinformationen enthält. Da das Modell in verschiedenen Tools wiederverwendet werden kann, können sich solche Fehler schnell verbreiten [55].

Insgesamt repräsentieren Foundation Models einen vielversprechenden Ansatz in der KI-Forschung, der sowohl einzigartige Möglichkeiten als auch spezifische Herausforderungen bietet. Die kontinuierliche Erforschung und Entwicklung von FM wird wahrscheinlich dazu beitragen, die Grenzen dessen, was mit KI erreicht werden kann, weiter zu verschieben, während gleichzeitig die Notwendigkeit besteht, ethische und praktische Überlegungen sorgfältig zu berücksichtigen.

3.5.2 Large Language Models (LLMs)

Large Language Models (LLM) bauen auf Foundation Models auf [29]. Im Gegensatz zu bisherigen Ansätzen, bei denen ein Modell für sehr spezifische Aufgaben immer wieder neu trainiert wurde, dient ein LLM als universelles Modell in der Sprachverarbeitung, das viele Aufgaben bewältigen oder als Grundlage für andere Modelle verwendet werden kann. Es basiert auf neuronalen Netzen mit sogenannter Transformer-Architektur und ist ein deep Learning-Modell, das natürliche Sprache verarbeitet (NLP).

Ein bekanntes Beispiel für ein LLM ist der Generative Pre-trained Transformer (GPT)[7], der die Fähigkeit besitzt, Text in einem menschenähnlichen Stil zu generieren. GPT und ähnliche Modelle haben die Entwicklung von Chatbots [72] ermöglicht, die NLP und KI verwenden, um den eingegebenen Text zu verstehen und auf natürliche Weise zu reagieren.

Die Transformer-Architektur, auf der solche Modelle basieren, ist ein deep Learning-Modell, das sich von traditionellen neuronalen Netzen unterscheidet [43]. Im Gegensatz zu herkömmlichen NN, die Daten in einer festgelegten Reihenfolge verarbeiten, haben Transformer die Fähigkeit, auf alle Neuronen zuzugreifen und den Input-Daten selbst eine Wichtigkeit zu verleihen. Sie verarbeiten Daten nicht zwangsläufig in der richtigen Reihenfolge, sondern versuchen stattdessen, den Kontext zu verstehen. Die Architektur besteht aus einem Encoder, der den Input verarbeitet, und einem Decoder, der den Output generiert. Dabei handelt es sich um eine Form von self-supervised Learning. Self-supervised learning verwendet keine gelabelten Daten und versucht die Strukturen in einem Datensatz selbstständig zu erkennen. [23] [53].

Insgesamt markieren LLMs, anhand des Beispiels von GPT mit der Transformer-Architektur einen Wendepunkt in der KI-Technologie, der die Art und Weise, wie wir mit Maschinen interagieren und wie sie lernen, grundlegend verändert hat. Sie bieten eine flexible und leistungsstarke Grundlage für eine Vielzahl von Anwendungen und setzen neue Maßstäbe für das, was mit KI erreicht werden kann.

3.5.3 Natural Language Processing (NLP)

Natural Language Processing (NLP) ist ein entscheidender Fortschritt in der KI, der es Computern ermöglicht, unstrukturierte menschliche Sprache zu verstehen und zu interpretieren [79]. Da die Art und Weise, wie Menschen sprechen, grundsätzlich unstrukturiert ist, besteht das Ziel von NLP darin, diese Sätze so zu strukturieren, dass ein Computer sie versteht. Der Prozess, bei dem unstrukturierte Sprache in strukturierte Sprache umgewandelt wird, wird als Natural Language Understanding bezeichnet, während der umgekehrte Prozess als Natural Language Generation (NLG) bekannt ist. NLP wird in Anwendungen wie machine Translation, Chatbots oder Virtual Assistants verwendet. Am Beispiel von ChatGPT zeigt sich, dass eine Eingabe eines Nutzers zunächst durch NLP in eine durch das LLM verarbeitbare Form überführt werden muss.

Nachdem die Eingabe verarbeitet wurde, kann durch NLG eine neue Ausgabe erzeugt werden, die wir wiederum in der Form von natürlicher Sprache verstehen können. Bei diesem Prozess handelt es sich im Grunde darum, Stück für Stück in einem Satz das Wort vorherzusagen, welches mit der höchsten Wahrscheinlichkeit als nächstes auftritt. Es handelt sich somit um statistische Wahrscheinlichkeiten, die mittlerweile so genau vorhergesagt werden können, dass in den meisten Fällen eine sehr kohärente Ausgabe erzeugt werden kann.

Immer wieder treten jedoch auch Fälle auf, in denen das Model eine Ausgabe erzeugt, die Falschinformationen oder missverständliche Texte beinhaltet. In diesem Fall spricht man von Halluzination (Halluzination). In der KI bezieht sich dieses Phänomen auf eine Ausgabe, die einem ausgedachten oder inkorrekten Sachverhalt folgt. Es gibt verschiedene Typen von Halluzinationen, die jeweils unterschiedliche Herausforderungen darstellen [68]:

1. Widerspruch im Satz: Hierbei handelt es sich um Sätze, die sich innerhalb der Ausgabe widersprechen, was zu Verwirrung und Missverständnissen führen kann.
2. Widerspruch zur Eingabe: Dies tritt auf, wenn die KI-Ausgabe nicht das wiedergibt, was tatsächlich angefragt wurde, und somit die Anfrage des Benutzers ignoriert.
3. Widerspruch in den Fakten: In diesem Fall generiert die KI falsche Fakten oder Informationen, die nicht mit der Realität übereinstimmen, was die Glaubwürdigkeit und Zuverlässigkeit der Ausgabe untergräbt.
4. Widerspruch durch irrelevante Fakten: Hierbei handelt es sich um Ausgaben, die nicht angefragte oder irrelevante Fakten enthalten, wodurch die Ausgabe weniger Informationsgehalt enthält.

Dieses Verhalten kann durch verschiedene Faktoren verursacht werden, einschließlich schlechter Datenqualität, unvollständiger Daten oder Overfitting. Es bleibt eine Herausforderung in der KI-Forschung und -Entwicklung, da es die Effektivität und Vertrauenswürdigkeit von KI-Systemen beeinträchtigen kann [48].

Zur Verbesserung von Halluzinationen können klare und spezifische Prompts durch Limitierung der möglichen Antworten verwendet werden. Dabei handelt es sich um sogenanntes „Shot-Prompting“, wobei die KI in dem Prompt null, bis mehrere verschiedene Beispiele erhält, anhand derer die gewünschte Ausgabe erkannt werden kann. Außerdem können „Temperaturanpassungen“ vorgenommen werden, wobei eine höhere Temperatur kreativere Ausgaben erzeugt, die möglicherweise Falschangaben enthalten. Niedrigere Temperaturen erzeugen hingegen vorhersehbarere Ausgaben [69]. Diese Prozesse werden noch einmal näher in den Hauptkapiteln beschrieben.

Insgesamt bieten NLP und die damit verbundenen Konzepte eine Möglichkeit, die Kommunikation zwischen Menschen und Maschinen zu verbessern und zu verfeinern. Gleichzeitig stellen sie Forscher vor Herausforderungen, wie sie die Genauigkeit und Zuverlässigkeit dieser Technologien sicherstellen können.

3.5.4 Tuning und CodeX

Tuning in der KI bezieht sich auf den Prozess, bei dem neue Parameter eingeführt werden, um einen spezifischen Task auszuüben [80]. Oftmals werden die großen LLMs wie GPT oder Palm von Google dazu verwendet, um auf Basis dieser Modelle, eigene zu trainieren. Dazu werden dann unternehmensinterne Daten verwendet, um zwar die bereits antrainierte Intelligenz des LLMs, aber dennoch die Informationen und Anwendungsgebiete im eigenen Kontext verwenden zu können.

Ein prominentes Beispiel für ein getuntes Modell ist CodeX von OpenAI [45]. CodeX ist spezifisch auf Code getuned und wurde mit natürlicher Sprache sowie Code (z.B. von öffentlichen GitHub Repositories) trainiert. Es handelt sich um ein „general-purpose“-Modell, das für eine Vielzahl von Aufgaben eingesetzt werden kann, einschließlich dem Transpilieren, Erklären und Refactoring von Code.

3.6 Wichtigkeit von Daten

3.6.1 Knowledge Graphs

Knowledge Graphs (KGs) haben in jüngerer Zeit sowohl in der Wissenschaft als auch in der Industrie erhebliche Aufmerksamkeit erregt. KGs bestehen aus Knoten, die über Kanten miteinander verbunden sind, und dienen dazu, komplexe Beziehungen und Semantiken in strukturierter Form darzustellen. Ein Beispiel wäre die Beziehung, dass Paris die Hauptstadt von Frankreich ist [34].

Darüber hinaus kann die Qualität und Glaubwürdigkeit in den Prozess der Konstruktion und Erweiterung von KGs integriert werden, insbesondere wenn sie aus gemischten Qualitätsressourcen wie sozialen Medien stammen [4].

In der Softwareentwicklung werden Vektor Datenbanken verwendet, um Informationen auf diese Art darzustellen. Diese Art von Datenbanken können anschließend hervorragend dazu verwendet werden, um LLMs zu tunen oder im Prozess des Prompt Engineerings einen strukturierten Kontext für den Prompt zu schaffen [52].

3.6.2 Schlechte Daten

Die Qualität der Daten ist entscheidend für die Leistung eines Modells. Irrelevante, unvollständige oder falsche Daten können zu schlechten Ausgaben führen, was durch den Ausdruck „Garbage in, garbage out“ [50] in diversen Artikeln verdeutlicht wird. Damit soll zum Ausdruck gebracht werden, dass das Modell nur so gut ist, wie die Daten, auf denen es trainiert wurde. Die Qualität und Vertrauenswürdigkeit der Daten sollte daher in den Prozess der Konstruktion und Erweiterung von LLMs integriert werden, um die Effektivität und Zuverlässigkeit der Systeme zu gewährleisten [4].

Kapitel 4

Verfahrensauswahl

4.1 Arbeiten mit Chatbots

4.1.1 Warum ChatGPT?

Neben ChatGPT gibt es weitere Chatbots, darunter Google Bard oder Microsoft Bing AI, um nur zwei Beispiele von großen Konzernen zu nennen. Mit der Premium-Version von ChatGPT wird als großer Vorteil gegenüber den Konkurrenten das Hochladen sowie das Generieren und Herunterladen von Dateien unterstützt. Dadurch ergibt sich ein großer Vorteil für diese Arbeit im Rahmen der Webanwendung. Google Bard ermöglicht zwar das Generieren von Code, kann zum aktuellen Zeitpunkt jedoch keine Dateien erzeugen oder lesen. Ebenso bietet auch Microsoft Bing AI diese Features noch nicht an.

OpenAI hat mit GitHub in Kooperation den GitHub Copilot auf Basis der GPT-3 Version entwickelt und trainiert. Zwar bietet ChatGPT als Applikation im Browser nicht die Möglichkeit, auf die lokale Entwicklungsumgebung zuzugreifen, allerdings wird durch die Verwendung im Browser das neuere Modell GPT-4 unterstützt, mit der zusätzlichen Möglichkeit, Dateien zu interpretieren und neue zu erstellen. Zum Stand dieser Arbeit wurde bereits Copilot-X veröffentlicht, welches ebenfalls auf GPT-4 basiert. Allerdings ist der Zugriff durch eine Beta aktuell begrenzt.

Des Weiteren kann gesagt werden, dass viele der öffentlich verfügbaren Tools auf OpenAI aufbauen und diese Modelle für Ihre Zwecke weiter fine-tunen (Siehe Sektion: 3.5.4). Daher sind viele der Tools lediglich eine Spezialisierung von ChatGPT für Code, Copywriting und weitere Aufgaben. Hinzu kommt, dass diese spezialisierten Werkzeuge zu Teilen noch auf GPT-3 oder 3.5 basieren und daher eine schlechtere Ausgabequalität besitzen können. Mit DevGPT [21] wird mittlerweile auch ein Entwicklungstool geboten, welches auf GPT-4 aufbaut und sich ebenfalls in die lokale Entwicklungsumgebung, bzw. das Repository integrieren lässt. Hier wird neben dem Vorteil des Kontextwissens jedoch nicht die Möglichkeit geboten, eine Konzeption oder Planung für ein größeres Projekt zu erzeugen und somit ein umfassenderes Wissen für den Kontextspeicher zugrunde zu legen.

Da mit ChatGPT eine Plattform geboten wird, die sowohl kostenfrei als auch gegen einen monatlichen Betrag mit mehr Leistung verwendet werden kann, ist das Tool die ideale Lösung, um reproduzierbare Ergebnisse für das Projekt im Rahmen dieser Arbeit zu erzielen.

4.1.2 Allgemeines zu ChatGPT

Die zu entwickelnde Webanwendung wird in dieser Bachelorarbeit mit dem Tool ChatGPT generiert. Dabei handelt es sich, wie in den Einleitungskapiteln bereits beschrieben, um ein Tool basierend auf einem deep Learning Modell, welches auf der sogenannten Transformer-Architektur eines neuronalen Netzwerks (NN) basiert (Siehe Kapitel: 3.4). Ein NN, welches diese Architektur verwendet, hat den Vorteil, dass eine Eingabe, wie ein Satz in natürlicher Sprache als Ganzes verarbeitet werden kann. Dadurch kann der Kontext des Satzes verstanden werden und gleichzeitig wird die Ausgabequalität erhöht, indem die Wörter des Satzes gewichtet werden [85].

Das Tool wurde von OpenAI entwickelt [16] und bietet ein User-Interface, über das mit dem GPT Modell ähnlich zu Messenger Diensten kommuniziert werden kann. Grundsätzlich werden Chatbots auf Basis der Modelle noch einmal fine-getunt mit dem Kontext, eines Chatbots, um auch als solcher zu agieren. Aktuell bietet OpenAI, zwei verschiedene Modelle, GPT-3.5 und für zahlende Nutzer GPT-4. Die Unterschiede zeigen sich unter anderem in der Komplexität des neuronalen Netzwerks. So steigt die Anzahl der Parameter im NN mit jeder Version an (siehe Abbildung 4.1).

Je höher die Komplexität, desto besser können die Transformer mit komplexeren Denkansätzen, Prompts oder Kreativität umgehen [56]. Mit erhöhter Komplexität steigt aber auch der „Wille“ des Modells, möglichst ausführlich auf sämtliche Eingaben zu antworten. Wenn der Nutzer eine bestimmte Ausgabe erwartet oder benötigt, muss deshalb immer

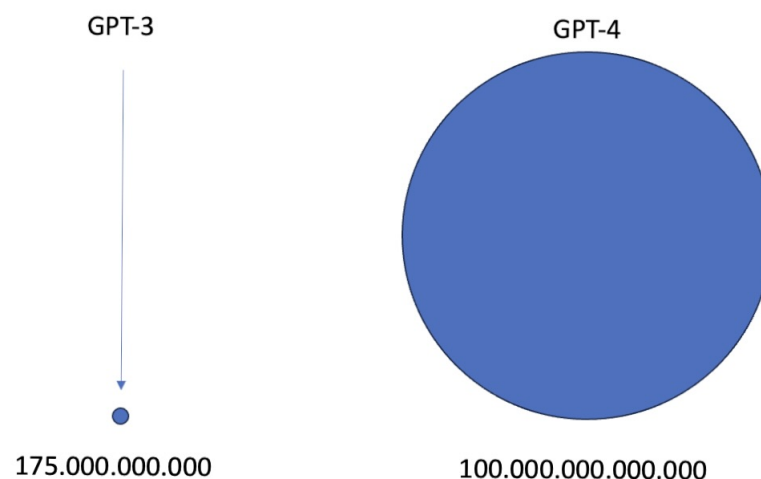


Abbildung 4.1: Größenvergleich zwischen GPT-3 und GPT-4 auf Basis der Anzahl der Parameter. In Anlehnung an Quelle: *The Ultimate Guide to GPT-4 Parameters* [41]

mehr Aufwand für den Prompt aufgebracht werden, um die gewünschte Ausgabe zu erzielen. Einen guten Einblick in das Verhaltensmuster von chatbasierten Sprachmodellen bietet der Playground von OpenAI [46]. Hier kann mit verschiedenen Modell-Einstellungen experimentiert oder ein Chatverlauf rekonstruiert werden, um einen fiktiven Kontext zu schaffen. Diese Einstellungen werden in der „klassischen“ Version von ChatGPT anhand des Prompts automatisch eingestellt. Je konkreter die Vorgaben des Nutzers sind, desto niedriger wird die Temperatur justiert.

4.1.3 Aufbau eines Chats

Ein Chat in ChatGPT ist grundsätzlich aus einem Eingabefenster und dem Nachrichtenverlauf aufgebaut. Abgeschickte Eingaben durch den Nutzer können im Nachhinein noch einmal bearbeitet werden, um eine alternative Ausgabe hervorzurufen. Dadurch wird eine hierarchische Baumstruktur gebildet, da alle folgenden Eingaben von der jeweils vorherigen abhängig sind. Somit können unterschiedliche „Pfade“ erzeugt werden, die jeweils auf einer vorherigen Eingabe basieren. Die Veränderung einer Eingabe bietet sich insbesondere dann an, wenn der Nutzer mit der Ausgabe bisher nicht zufriedengestellt ist. Dann können zusätzliche Informationen hinzugefügt oder die Wortwahl verändert werden, um eine Alternative Ausgabe zu erzeugen. ChatGPT baut dann den folgenden Verlauf auf der ausgewählten Variante auf. Weiterhin ist das „Wissen“ über den Nutzerkontext von ChatGPT immer auf den Chat bezogen [8]. Daher bietet es sich an, für jedes neue Thema oder jeden Anwendungsfall einen Chat zu öffnen, der sich mit länger werdendem Verlauf immer weiter auf das Thema spezialisiert. Andersherum bietet es sich auch an, leicht verwandte Themen in denselben Chat zu stellen, um auf dem bereits vorhandenen Kontextwissen aufzubauen und den Kontext nicht erneut anreichern zu müssen. Seit kurzem erlaubt das Tool auch, Vorgaben und Informationen über das allgemeine Anwendungsgebiet oder die Rolle des Nutzers anzugeben, um einen übergreifenden Kontext zu gestalten.

4.1.4 Funktionsweise und Verarbeitung

GPT-4 kann sich laut verschiedener Quellen 64.000 Wörter merken [41]. Dadurch wird auch die Empfehlung begründet, gleiche Themen, in einem Chat zu strukturieren. Ein optimal gefülltes Gedächtnis von ChatGPT kann daher besonders gute Ausgaben erzeugen, da zu diesem Zeitpunkt das Verhältnis zwischen dem antrainierten Wissen und dem Kontextwissen balanciert ist.

Um die Qualität und Bedeutung des Kontextspeichers auszutesten, kann ein Projekt, etwa eine SPA-Anwendung entwickelt werden. Wenn der Kontext zu Beginn erklärt wird, werden die ersten erzeugten Dateien noch Code enthalten, der nicht unbedingt den gewünschten Anforderungen entspricht. Das liegt schlichtweg daran, dass der Mensch viele Dinge intuitiv voraussetzt, die in KI-Modellen bisher nicht antrainiert sind. Der Mensch tendiert daher dazu, Informationen auszulassen, die für selbstverständlich gehalten werden. Diese ausgelassenen Informationen führen allerdings dazu, dass die Ausgaben zunächst von unseren Vorstellungen abweichen. Ab einem gewissen Punkt haben wir diese Vorstellungen ChatGPT mitgeteilt, wodurch das Gedächtnis immer weiter gefüllt wird und schlussfolgernd weniger Fehler ent-

stehen.

Umgekehrt ist es aber auch wichtig, dass ein gewisser Freiraum gelassen wird, da eine zu starke Eingrenzung den Fluss der Ausgabe unterbrechen kann. Betrachtet man eine Ausgabe im OpenAI Playground, bei der die Temperatureinstellung auf nahe null gestellt wird, erscheint diese möglicherweise auf den ersten Blick besser als eine kreativere Ausgabe. Für manche Anwendungsfälle kann eine solche Einstellung auch von Vorteil sein, zum Beispiel, wenn die Ausgabe anschließend durch ein anderes Tool weiterverarbeitet wird. Dann muss darauf verlass sein, dass die Vorgaben eingehalten und keine neuen Informationen eingeflochten werden.

Beim Entwickeln einer Anwendung ist es aber wichtig, dem Modell nicht die Kreativität zu nehmen. Eine Komponente, die einen Kalender in einem User-Interface abbilden soll, müsste sehr genau beschrieben werden, um bei einer niedrigen Temperatur eine gute Ausgabe zu erzeugen. Wird die Temperatur erhöht, können auch Ausgaben erzeugt werden, die insbesondere im Frontend-Development gewünscht sind, da sich die Darstellung erheblich verbessern kann. Im Backend sind hingegen die Anforderungen sehr genau definiert und beruhen nicht auf visuellem Design. Hier kann eine niedrige Temperatur gewünscht sein, um verschiedene REST-Routen immer wieder gleich zu generieren.

Wenn das Projekt sehr umfangreich ist, kann es auch vorkommen, dass die Kapazität des Gedächtnisses überschritten wird. Dann sinkt die Qualität der Ausgaben wieder, da ältere Dateien nicht mehr abgespeichert sind. Wenn eine solche Größe absehbar ist, dann lohnt es sich, das Projekt in verschiedene Chats aufzuteilen und Dokumente bereitzustellen, die den grundlegenden Kontext beinhalten. Wird die Kapazität überschritten, dann kann es zu falschen Ausgaben kommen, da nicht mehr erkenntlich ist, welche Änderung an einer Datei nun gespeichert wurde. So kann eine Komponente erzeugt werden, die im Verlauf mehrfach verändert wird. Wenn während einer der Veränderungen die Kapazität überschritten wird, dann ist im weiteren Verlauf nicht länger deutlich, welche Veränderung die letzte gespeicherte ist. In Folge kommt es dann bei einer Anpassung der Datei zu einer Ausgabe, die auf einer älteren Version der Datei basiert.

4.1.5 Vorgehensweise mit ChatGPT

Insbesondere wenn ein größeres Projekt mit ChatGPT erstellt werden soll, lohnt sich eine eingehende Analyse im Vorhinein, um einem gut strukturierten Top-Down Ansatz zu folgen. Top-Down meint eine Vorgehensweise, bei der die Anforderungen von sehr grob begonnen, immer spezifischer und konkreter werden.

Um eine Webanwendung zu generieren, kann daher zu Beginn der Kontext und der Anwendungsfall grob strukturiert werden. Im nächsten Schritt können gewünschte Funktionen angegeben werden, um anschließend Abhängigkeiten zu definieren, die zwischen den einzelnen Funktionen bestehen. Weiterhin kann in die Softwarearchitektur über Module und Komponenten eingestiegen werden, bis letztendlich einzelnen Dateien oder sogar Methoden beschrieben werden. Dieser Ansatz bietet den klaren Vorteil, dass immer wieder von Nutzer und GPT-Seite Rückfragen und weitere Überlegungen angestellt werden können, um den

Rahmen des Projekts möglichst genau zu definieren und keine Anforderungen unüberlegt zu lassen.

In der Version GPT-4 ist sogar die Erstellung von Dateien möglich. Um den Kontext zu „speichern“, kann ein Plan oder Dokument basierend auf dem Chatverlauf erstellt werden, der den Kontext zusammenfasst. Dieses Dokument ermöglicht es, nachfolgende Chats kontinuierlich mit dem gespeicherten Kontext anzureichern.

4.2 Konzeptionierung der Webanwendung

4.2.1 Warum eine Webanwendung?

Wie im Kapitel der verwandten Arbeiten beschrieben, gibt es bereits unzählige Arbeiten und Artikel, die sich mit dem Thema künstliche Intelligenz auseinandersetzen und einen Überblick über die am häufigsten verwendeten Begriffe geben (Siehe in der Sektion verwandte Arbeiten: 2.6). In diesen Arbeiten und Artikeln werden auch viele Tools vorgestellt, darunter Github Copilot, der GPTEngineer oder auch Dev-GPT [21]. Die meisten Tools helfen dabei einzelne Komponenten oder Zeilen von Code zu generieren, nicht aber ganze Webanwendungen.

Frameworks wie Langchain konzentrieren sich eher auf den Bereich der Erstellung intelligenter Anwendungen und nicht die intelligente Erstellung der Anwendungen. Mit dieser Bachelorarbeit soll daher beispielhaft gezeigt werden, ob sich die frei auf dem Markt verfügbaren Tools dazu einsetzen lassen, ganze Anwendungen durch eine strukturierte Herangehensweise erstellen lassen.

Weiterhin untersuchen viele Artikel bereits die Grenzen der Komplexität, die mit KI geschaffen werden kann. Im Rahmen dieser Arbeit, soll die Dimension der Größe und Skalierbarkeit näher untersucht werden, um Unternehmen und Webentwicklern aufzuzeigen, wo sich der aktuelle Stand der Technologie befindet und wohin sich die Technologie noch entwickeln muss, um ganze Projekte KI-basiert generieren zu lassen.

4.2.2 Problemstellung

Im Abschnitt „Funktionsweise und Verarbeitung“ 4.1.4 wurde beschrieben, dass moderne Sprachmodelle eine Art Gedächtnis und Speicher besitzen, um neue Antworten basierend auf dem Kontext des Chatverlaufs zu generieren. Dieser Umfang, Kontextinformationen zu verarbeiten wird zwar immer besser und kann durch die verschiedenen Prompt-Engineering Techniken auch durch den Nutzer erhöht werden, jedoch steigt mit dem Umfang des jeweiligen Projektes auch der Aufwand, um alle Dateien im Zwischenspeicher aktuell zu halten. Es sollte daher vermieden werden, Dateien nachträglich erneut zu bearbeiten. Das Ideal ist es, mithilfe eines explizit konstruierten Prompts, das Modul, die Datei oder die Komponente direkt zu erstellen. Verbesserungen sollten dann umgehend nachgetragen werden, wenn die Antwort noch nicht den Anforderungen entspricht. Somit wird vermieden, dass zwischen der Generierung und der Veränderung einer Datei, weitere Komponenten und Module erzeugt werden, die den Kontext verändern können.

4.2.3 Erwartungshaltung an Backend und Frontend

Bereits vor Beginn der Generierung wurde die Vermutung aufgestellt, dass das Backend wesentlich schneller generiert werden kann, da es durch die REST-Architektur in Verbindung mit dem MVC Pattern klare Richtlinien aufweist, auf denen die KI bereits trainiert wurde. Diese Richtlinien sind einfacher einzuhalten, da sie Mustern folgen, die sich leichter reproduzieren lassen, als es Frontend-Richtlinien würden.

Für das Frontend wurde vermutet, dass mehr Komplexitäten auftreten würden. Zum einen ist die Kommunikation zwischen einzelnen Komponenten und Dateien wesentlich höher, aber auch die visuelle Darstellung folgt nicht immer logischen Mustern. Aus diesem Grund soll bei der Generierung des Frontends ein noch stärkerer Fokus auf die feingranulare Aufteilung in Komponenten gelegt werden. Wo möglich wird die Kommunikation zwischen Komponenten in einen Store ausgelagert oder von der Komponente selbst gehandelt werden, um eine zu hohe Komplexität durch die Abhängigkeit zu vermeiden.

4.2.4 Ziel der Webanwendung

Ziel der Anwendung ist es, aufzuzeigen, was bereits mithilfe der verfügbaren Tools möglich ist. Es sollen Grenzen anhand einer größeren Anwendung, nicht anhand einzelner Komponenten genauer untersucht werden. Die Webanwendung selbst sollte ein Endergebnis aufzeigen, anhand dessen die Schwachstellen von chatbasierten KI-Modellen untersucht werden können. Daher muss ein Mittelweg zwischen einem zu einfachen und zu komplexen Anwendungsfall gefunden werden.

Eine TODO-App kommt daher nicht infrage, da die Grundidee und das Konzept mit hoher Wahrscheinlichkeit erstellt werden kann. Erst durch das Hinzufügen von weiteren Features würde in diesem Rahmen eine durchschnittlich komplexe Anwendung entstehen.

In verschiedenen Abstimmungsgesprächen ist in Kooperation mit dem unterstützenden Unternehmen die Idee für ein Event Management System entstanden. Es wurden verschiedene must-have-Features festgelegt, als auch nice-to-have Features, die anschließend entwickelt werden können. Die Anwendung wurde auf Grundlage des Szenarios bei dem kooperierenden Unternehmen aufgebaut. In diesem Szenario besteht die Unternehmensstruktur aus einer Arbeitskraft, die einer sogenannten Line zugeordnet ist. Eine Line wiederum ist einer Business-Unit zugeordnet. Für diese Bereiche sollen Events angelegt und Personen eingeladen werden können. In diesem Event soll dann von den Personen abgestimmt werden können, um einen passenden Termin zu finden.

4.2.5 Architektur & Softwaredesign

Öffentlich zugängliche Tools sind grundsätzlich auf kleinere Codeabschnitte ausgelegt. Das heißt, dass Sie eine hohe Ausgabequalität besitzen, wenn es darum geht, einzelne Methoden oder Dateien mit einheitlichem Inhalt zu erzeugen. Insbesondere repetitiver Code kann durch gut gewählte Prompts viel Arbeitszeit ersparen, um beispielsweise verschiedene REST Routen oder Pinia Stores [51] mit verschiedenen gettern und settern zu erstellen.

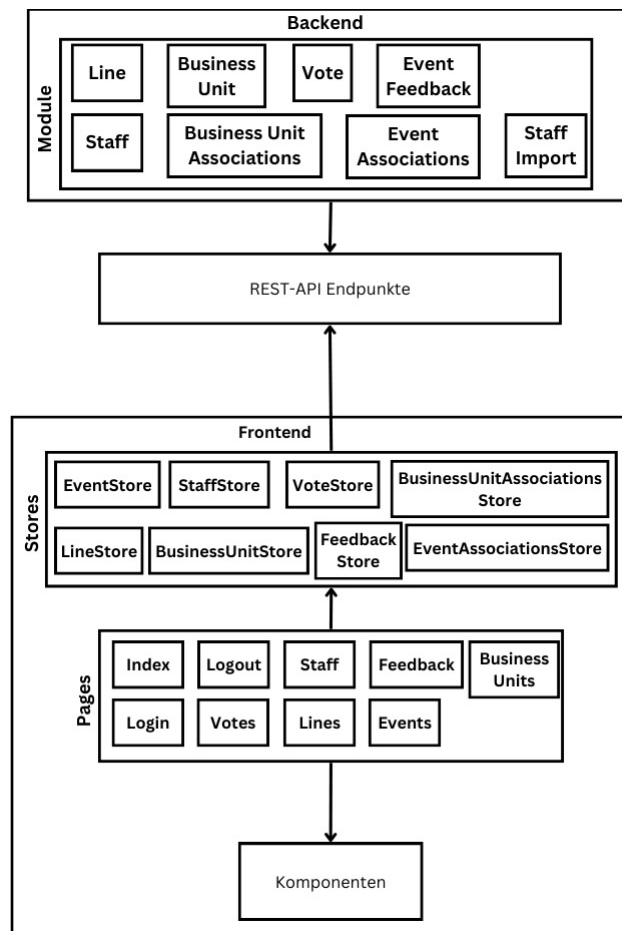


Abbildung 4.2: Schematische Darstellung der Architektur in Frontend und Backend

Damit auch von diesen Vorteilen weiterhin profitiert werden kann, müssen die Abhängigkeiten zwischen den einzelnen Dateien, Komponenten und Modulen so gering wie möglich ausfallen, sodass der Daten- und Kommunikationsfluss unidirektional ist. Genauer gesagt, stellt das Backend einen Endpunkt zur Verfügung, der von einem Store im Frontend abgefragt werden kann und ermöglicht, dass durch die zentrale Datenhaltung im Store verschiedene Komponenten und Views auf diese Daten zugreifen können. Weiterhin soll dadurch vermieden werden, dass durch Kommunikation zwischen Komponenten, die nicht über den Store fließt, die Komplexität und damit auch die Randbedingungen für neue Komponenten zu hoch werden und nicht mehr richtig durch die KI erfasst werden können.

Um das zu erreichen, wurde für die Webanwendung ein modularer Ansatz im Backend und ein Store zentrierter Ansatz im Frontend gewählt (siehe Abbildung 4.2). Ziel soll es sein, dass die Anwendung so aufgebaut ist, dass das Sprachmodell nicht zwangsläufig den Kontext der gesamten Webanwendung besitzen muss, sondern lediglich die Anforderungen für das

jeweilige Modul oder die Komponente. Abhängigkeiten wie etwa die Actions in den Stores können dann als zusätzliche Informationen mitgegeben werden. Die Stores und Module sollen somit den Großteil der Kommunikationsarbeit abnehmen, um Raum für die Komplexität durch das Design und die visuelle Darstellung im Frontend zu lassen.

Ein solches Modul soll hier einem Feature oder Bereich in der Anwendung gleich gesetzt werden. Ein Feature im Backend können dann die CRUD Endpunkte für Events sein. Im Frontend wird an dieses Feature anschließend durch einen Store angeknüpft und auf einer Seite dargestellt.

4.3 Prompt Based Entwicklung

4.3.1 Was ist ein Prompt?

Ein Prompt (= Aufforderung) ist im Grundsatz nichts anderes als eine Eingabe in ein chatbasiertes KI-Modell. Da die Ausgabe des Modells stark abhängig von der Qualität des Prompts ist, hat sich aus dem sogenannten Prompting das Prompt Based Engineering entwickelt. Laut dem Oxford Wörterbuch steht Engineering für:

„The activity of applying scientific knowledge to the design, building and control of machines, roads, bridges, electrical equipment, etc.“

Das bedeutet, dass sich Prompt Based Engineering damit beschäftigt, mithilfe des Wissens über künstlicher Intelligenz, chatbasierte KI-Modelle zu bauen und zu bedienen. Es existieren mittlerweile große Mengen an Ratgebern und Prompt-Sammlungen, um eine möglichst präzise und auf den Kontext zugeschnittene Ausgabe zu produzieren.

4.3.2 Ansätze von Prompt Engineering

Um auch bei der Konstruktion der Webanwendung im nächsten Kapitel Grundwissen zu besitzen, werden im Folgenden einige der bekanntesten Ansätze und Vorgehensweisen vermittelt.

1. Shot Prompting [\[37\]](#)

Unter dem Begriff „Shot Prompting“ verbirgt sich das Bereitstellen von Beispielen für einen Sachverhalt. Etabliert haben sich darunter, insbesondere die Varianten: Zero-, One- und Few-Shot Prompting. Dabei wird dem Modell entweder kein, ein oder mehrere Beispiele aufgezeigt, wie die ideale Ausgabe aussehen kann. Ähnlich wie bereits in der Funktionsweise von Chatbots beschrieben wird damit ein kleines Kurzzeitgedächtnis suggeriert, anhand dessen die KI den Kontext besser verstehen kann und erzeugt dann als Konsequenz eine qualitative Aussage.

Write a function in JavaScript that subtracts a number b from a number a.
Example function: `function add(a, b){ return a+b; }`

2. Denkprozess darstellen durch Anfügen von „Lets think step by step“

Mehrere Quellen [36] [38] berichten davon, dass das Anfügen des Satzes „Lets think step by step“ eine bessere Ausgabe erzeugt. Dies hat den Hintergrund, dass das Chatmodell durch die Aufforderung den Denkprozess darstellt und sich somit im Verlauf der Ausgabe die Wahrscheinlichkeit der richtigen Aussage erhöht. Insbesondere Transformer betrachten immer den Kontext einer Eingabe oder versuchen den Kontext vorherzusagen, anhand der wenigen Informationen, die durch den Nutzer eingegeben werden. Durch „Lets think step by step“ wird für jeden folgenden Satz der Kontext des letzten generierten Satzes mit aufgenommen. Daher ist die Schlussfolgerung immer basierend auf den generierten Überlegungen.

Write a function that takes a number n as an input and prints out the first n fibonacci numbers. Lets think step by step.

3. Chain of Thought (CoT) [67]

Chain-of-thought folgt einem ähnlichen Muster wie das shot-prompting. Hier wird allerdings nur die Denkweise für die gewünschte Ausgabe erläutert. Zu einer ersten Frage wird eine Antwort mit übergeben, die den Denkprozess inklusive des Ergebnisses genauer erklärt. Der dritte Bestandteil des Prompts enthält die eigentliche Frage. Durch diese Herangehensweise, wird das Denkmuster, in dem das Modell denken soll, vorgegeben.

Question: I have 5 lines of code, delete two lines, and write 6 lines. How many lines of code does the file consist of?

Answer: I start with 5 lines. Subtracting two leaves 3. Adding 6 lines makes it 9. The file consists of 9 lines of code.

Question: In a file, 4 variables are declared. 2 of them are deleted and 3 new ones are declared. How many variables are declared in the file?

4. Act like... [82]

Ein weiterer sehr häufig gewähltes Vorgehen ist das Mitgeben eines spezifischen Kontextes oder Berufsfelds durch eine Rollenzuweisung. Im Rahmen dieser Bachelorarbeit bietet es sich beim Generieren der Webanwendung an, etwa mithilfe des Satzes „Act as a Fullstack Webdeveloper“ Ausgaben zu erzeugen, die mehr Tiefe in der Ausgabequalität besitzen.

Act as a fullstack Webdeveloper. The goal is to develop a webapplication handling events for a Company. How should we start?

Kapitel 5

Generierung und Analyse

5.1 Umgesetzte Features

5.1.1 Login und Logout

Da zu Beginn des Experiments noch nicht abgeschätzt werden konnte, welcher Aufwand für die Webapp betrieben werden muss, wurde sich dafür entschieden, den Login zunächst nur anzutäuschen und beispielhaft zu implementieren. Um sich einzuloggen, muss der Nutzer die ObjectId eines Staff eingeben. Dann wird überprüft, ob die ObjectId im Backend existiert und falls ja, im localStorage abgespeichert, um den Nutzer eingeloggt zu lassen. Beim Ausloggen wird die Id aus dem localStorage gelöscht und dem Nutzer wird nur noch die Loginseite angezeigt.

5.1.2 Events

Das Kernmerkmal der Webanwendung besteht in der Event-Erstellung. Wie in der Architektur skizziert, wurden die grundlegenden CRUD-Operationen erfolgreich umgesetzt. Darüber hinaus wurden spezielle Hilfsrouten entwickelt, die das Aktualisieren von Mitarbeitern innerhalb eines Events ermöglichen. Diese Methoden erlauben nicht nur das Hinzufügen oder Entfernen einzelner Mitarbeiter, sondern auch das Aktualisieren von Mitarbeitern einer gesamten Line oder Business-Unit. Im Frontend manifestiert sich diese Funktionalität auf der CreateSeite (5.1). Diese Create Seite wird wiederverwendet, wenn von der Übersichtsseite, ein Event ausgewählt wird, um es zu aktualisieren. Komplettiert werden die CRUD Operationen im Frontend durch das Löschen von Events. In der Datenbankschicht werden die Events mit Ihrer Object Id und alle anderen Keys als Strings, bzw. das Datum als Date abgespeichert (5.2).

Dashboard Events Staff Lines Business Units Feedback Votes Logout

Create Event

Name

Date

dd.mm.yyyy

Location

Add Staff By:

Manual

Prause
Grace
Alice
David
Alice

Create

Abbildung 5.1: Frontend View zur Erstellung eines Events (Screenshot aus der Webapp)

```
_id: ObjectId('651d68cdd2f6c12c4592b342')
name: "Herbstparty"
date: 2023-10-19T00:00:00.000+00:00
location: "Beispielstadt"
staff: Array (1)
  0: ObjectId('65168b521b110bb042526129')
__v: 0
```

Abbildung 5.2: Datenbankobjekt eines Event in der Datenbank

5.1.3 Staff

Zu einem Event lassen sich Mitarbeiter, Lines oder ganze Business Units hinzufügen. Auch hier wurden die grundlegenden CRUD Operationen im Backend implementiert. Dem zugehörig wurde ein weiteres Modul erstellt, welches folgende Aufgaben zusätzlich übernimmt:

- Alle lines einer Business Unit abfragen
- Alle Mitarbeiter einer line abfragen
- Alle Mitarbeiter einer Business Unit abfragen
- Die Line und Business Unit zu der ein Mitarbeiter zugeordnet ist, abfragen

Mit diesem Hilfsmodul kann die Create Seite für einen Mitarbeiter einfacher erstellt werden. Um einen Mitarbeiter zu erstellen, werden einige personenbezogene Daten angegeben und eine Business-Unit ausgewählt, in der sich der Mitarbeiter befindet (siehe Abbildung 5.3). Wenn die Business-Unit ausgewählt wurde, kann weiterhin eine Line ausgewählt werden, zu der der Mitarbeiter zugeordnet wird. Wenn die Daten aktualisiert werden sollen, kann

Dashboard Events Staff Lines Business Units Feedback Votes Logout

Create Staff

Name

Prenom

Number

Email

Notes

Business Unit

Line

Create

Abbildung 5.3: Frontend View zur Erstellung eines Staff (Screenshot aus der Webapp)

auch hier, ein Mitarbeiter ausgewählt werden, um die Create Seite mit den entsprechenden Daten zu befüllen und somit wiederverwendbar zu machen. Auf der Übersichtsseite können ebenfalls Mitarbeiter gelöscht werden.

Als weiteres Feature gibt es die Möglichkeit, mehrere Mitarbeiter auf einmal zu importieren (siehe Abbildung 5.4). Dazu kann eine csv Datei ausgewählt werden, in der die Daten der verschiedenen Mitarbeiter tabellarisch abgespeichert werden (siehe Abbildung 5.5). Bei der Generierung wurde hier darauf geachtet, dass für die Business-Unit und Line zu der ein Mitarbeiter zugeordnet ist, nicht die unbekannte Object Id angegeben werden muss, sondern der Klurname als String angegeben werden kann. Im REST-Endpunkt, der den Import im Backend verarbeitet, wird dann zu den Klarnamen die entsprechende Object Id herausgesucht, um die Mitarbeiter korrekt abzuspeichern.

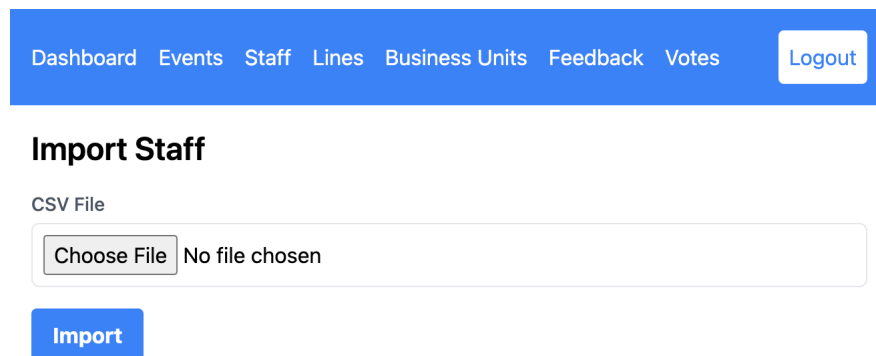
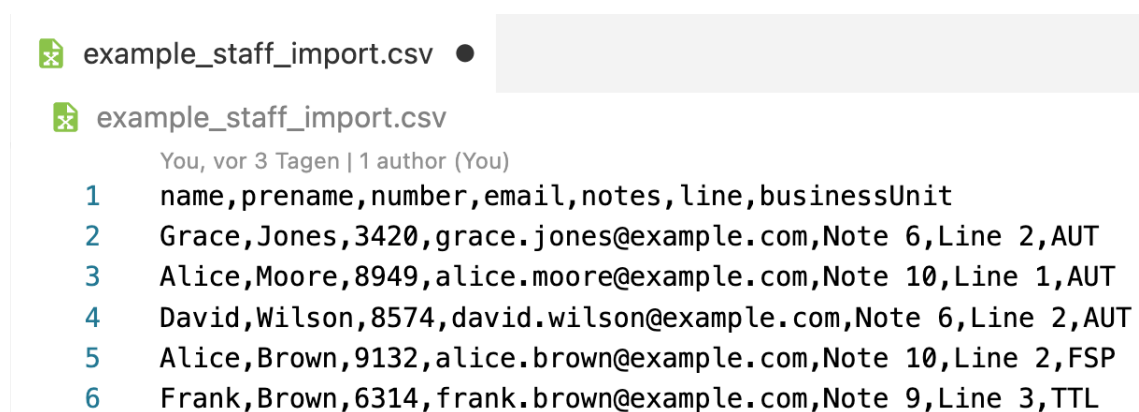


Abbildung 5.4: Frontend View zum Importieren von Staff (Screenshot aus der Webapp)



```
example_staff_import.csv
You, vor 3 Tagen | 1 author (You)
1 name,prename,number,email,notes,line,businessUnit
2 Grace,Jones,3420,grace.jones@example.com,Note 6,Line 2,AUT
3 Alice,Moore,8949,alice.moore@example.com,Note 10,Line 1,AUT
4 David,Wilson,8574,david.wilson@example.com,Note 6,Line 2,AUT
5 Alice,Brown,9132,alice.brown@example.com,Note 10,Line 2,FSP
6 Frank,Brown,6314,frank.brown@example.com,Note 9,Line 3,TTL
```

Abbildung 5.5: CSV Datei zum Importieren von Staff

5.1.4 Lines und Business Units

Die Seiten für Lines und Business Units sind analog zu den Seiten für Events und Staff aufgebaut (siehe Abbildung 5.6). Auf einer Übersichtsseite werden sämtliche Lines angezeigt, sowie die dazugehörige Business-Unit. Hier gibt es allerdings nur die Möglichkeit, Lines hinzuzufügen. Es wurde bewusst, darauf verzichtet, Operationen für das Löschen und Aktualisieren von Lines und Business Units ebenfalls zu implementieren, da es sich hierbei um grundlegende Unternehmensstrukturen handelt, die oftmals initial angelegt und nachträglich nicht mehr verändert werden.

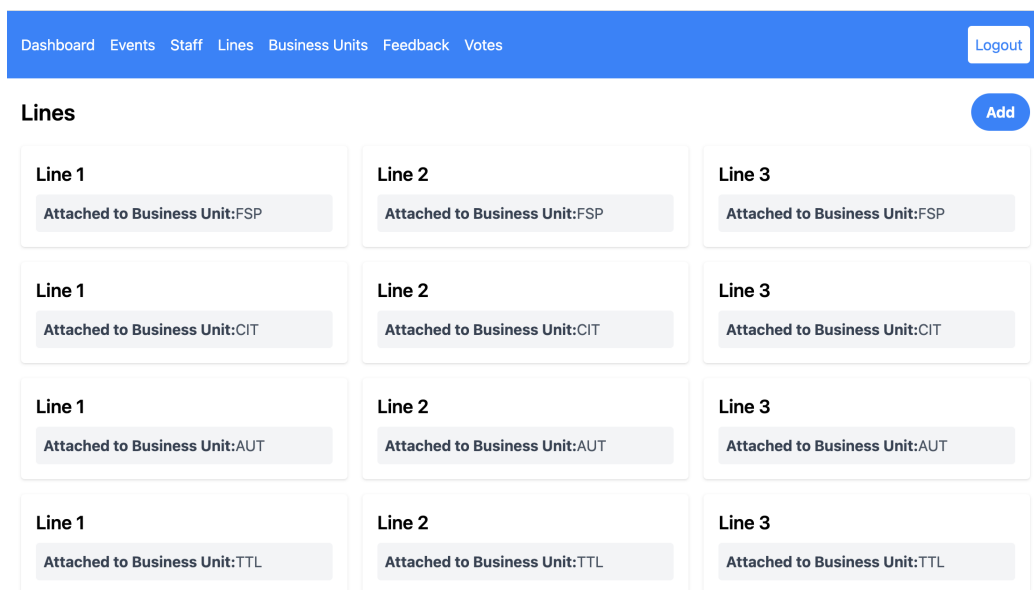


Abbildung 5.6: Frontend View für die Übersicht aller Lines (Screenshot aus der Webapp)

Auf der Create Seite der Lines wird ein Name für die Line angegeben und eine dazugehörige Business-Unit ausgewählt. Die Create Seite für Business Units besteht aus einem Namensfeld und einem Feld für eine Beschreibung.

5.1.5 Feedback

Auf der Feedback-Seite können Mitarbeiter ein Event bewerten (siehe Abbildung 5.7). Dabei kann ein Rating und ein Kommentar abgegeben werden. Auf der Übersichtsseite können die Bewertungen dann anonymisiert eingesehen werden. In der Datenbankschicht wird ein Feedback mit der StaffId sowie dem Rating und dem Kommentar abgespeichert.

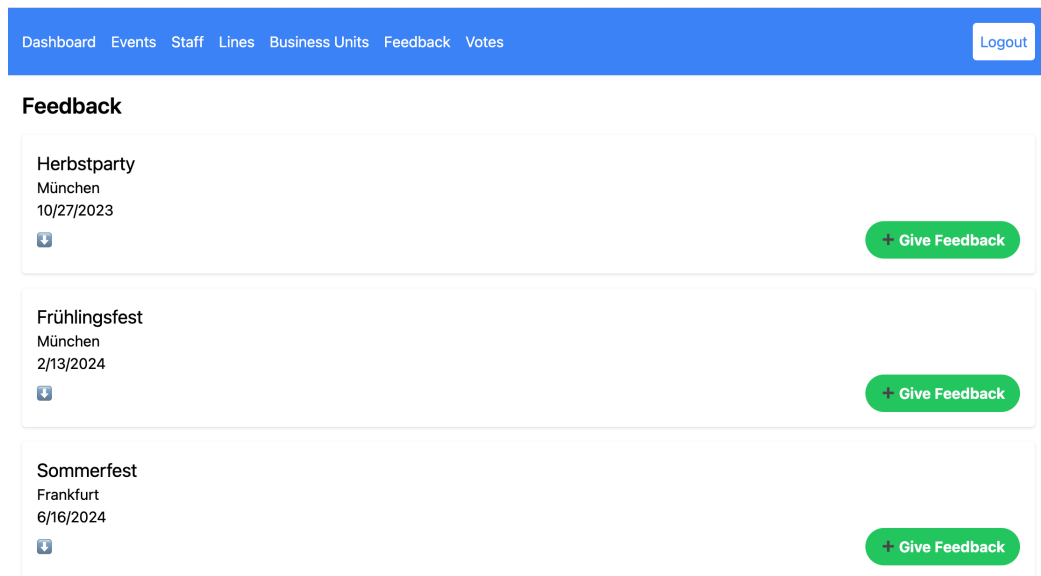


Abbildung 5.7: Frontend View für das Einsehen von Bewertungen zu einem Event (Screenshot aus der Webapp)

5.1.6 Votes

Weiterhin kann für ein Event abgestimmt werden, um eine Datumsfindung zu ermöglichen (siehe Abbildung 5.8). Hier werden die Events angezeigt, mit der Möglichkeit, zuzusagen, abzusagen oder mit Vorbehalt zuzusagen.

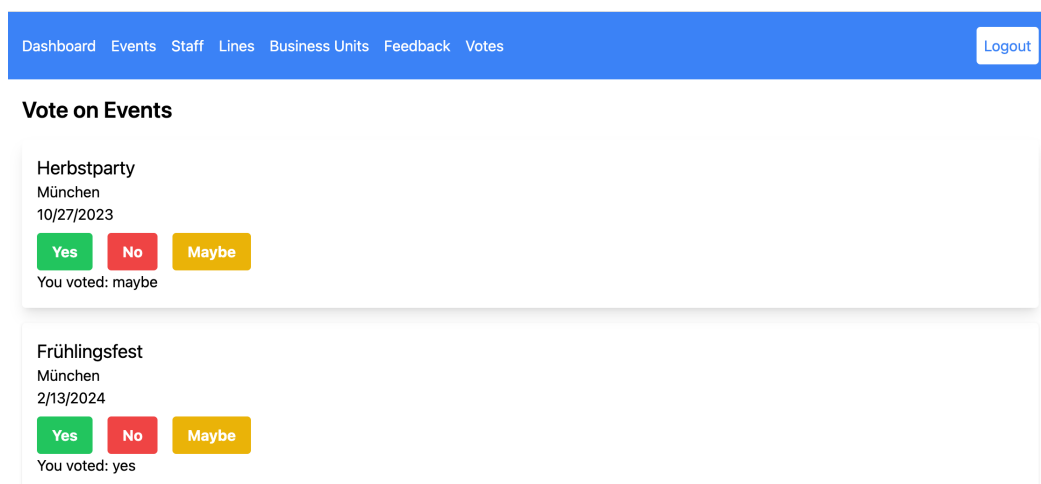


Abbildung 5.8: Frontend View für das Abstimmen für Events (Screenshot aus der Webapp)

5.1.7 Dashboard

Aufgrund der erhöhten Komplexität durch die Personalisierung einer Webanwendung, ist das Dashboard als einzige Seite mit personalisierten Inhalten, die auf den Mitarbeiter zugeschnitten sind, erstellt worden (siehe Abbildung 5.9). Auf dieser Seite werden verschiedene Komponenten angezeigt:

1. Willkommensnachricht
2. Die Line und Business Unit, zu der der Mitarbeiter zugeordnet ist
3. Events, die in der Vergangenheit liegen und für die noch kein Feedback abgegeben wurde
4. Events, zu denen der Mitarbeiter eingeladen ist und noch nicht abgestimmt hat
5. Übersicht über anstehende Events

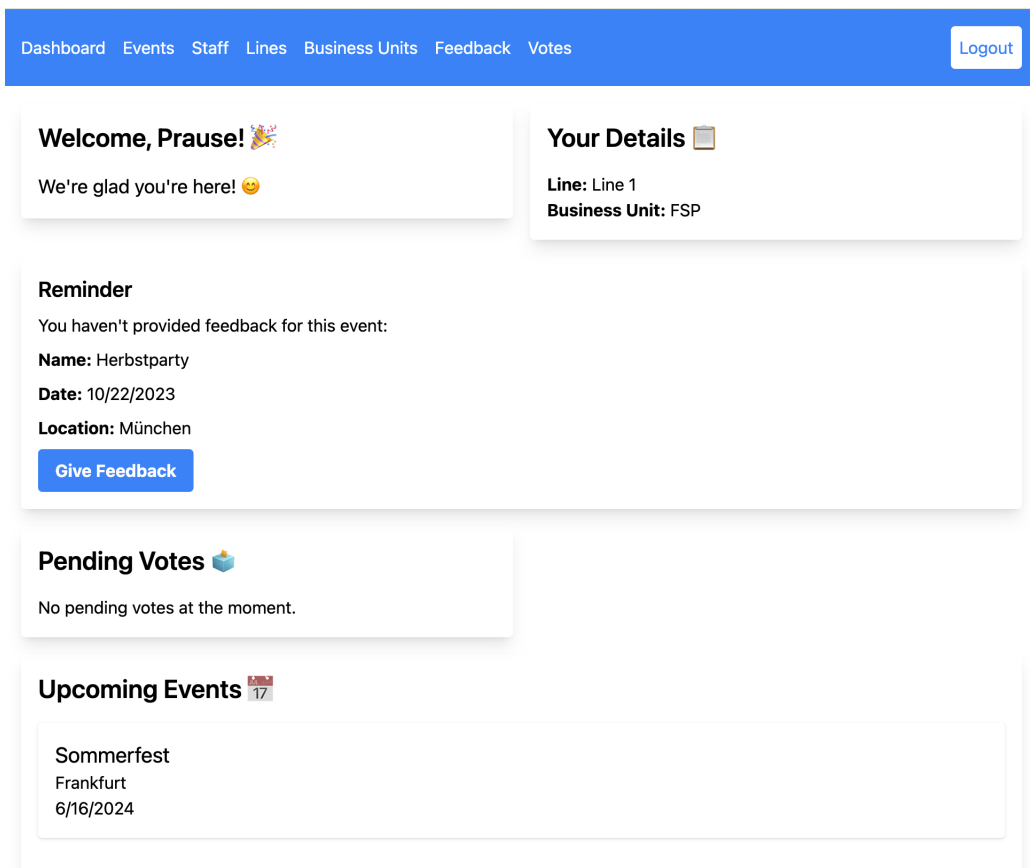


Abbildung 5.9: Frontend View für das Dashboard (Screenshot aus der Webapp)

5.2 Leitfaden der Generierung

5.2.1 Definition of Coding

Für die Generierung der Webanwendung wurde in Abstimmung mit dem unterstützenden Unternehmen festgelegt, welche Kriterien erfüllt sein müssen, sodass die Regel „No Coding“ nicht verletzt wird. Ziel ist es, dass sämtlicher Code von ChatGPT selbst generiert wird und kein Eingriff durch manuelles Schreiben von Code erfolgen muss. Zunächst muss unterschieden werden zwischen dem Schreiben und dem Erstellen von Dateien. Da ChatGPT grundsätzlich keinen Zugriff auf die lokale Entwicklungsumgebung besitzt, soll es erlaubt sein, Dateien und Ordner händisch zu erstellen. Außerdem muss zwischen Konfiguration des Projekts und Coding unterschieden werden. Unter Konfiguration zählt etwa das Anlegen des Projekts über ein Command Line Interface (CLI) oder das Anpassen von Dateipfaden, die den Zeitaspekt massiv beeinträchtigen würden, wenn die aktuelle Ordner- und Dateistruktur mit jedem Prompt mitgeschickt werden müsste. Das bedeutet zum einen, dass ein händischer Eingriff von vorneherein vorgesehen ist, aber die reine Logik und Implementierung dennoch mithilfe künstlicher Intelligenz umgesetzt werden soll.

5.2.2 Vorgehen

Die Webanwendung wurde mithilfe der verschiedenen Techniken, die bereit vorgestellt wurden, generiert (siehe folgende Sektion: 4.3). Im Rahmen dieses Kapitels sollen daher die Erfahrungswerte geteilt und analysiert werden. Da das Kontextwissen von ChatGPT maximal 64.000 Wörter umfasst, wurde von Beginn an darauf geachtet, dass die Dateien mit einem Mal, bzw. weiteren Rückfragen direkt im Anschluss generiert werden. Um das Gedächtnis mit einigen Informationen zu Beginn zu befüllen, wurden die Features mit ChatGPT konzipiert. Zunächst wurde durch das Umreißen der Features festgelegt, in welcher Reihenfolge die Webapp implementiert werden soll. Die Ausgabe umfasste dabei folgende Pipeline:

- Step 1** Project Setup
- Step 2** Database Schema Design
- Step 3** Backend - NestJS
- Step 4** Frontend - NuxtJS
- ...

Im Einklang mit dem festgelegten Schema begann die Generierung der Webanwendung mit der Entwicklung des Backends und des Datenbank-Designs. Nachdem die Mehrheit der Endpunkte definiert war, schwenkte der Fokus auf das Frontend. Um sicherzustellen, dass der Kontextspeicher aktuell bleibt, diente eine grobe Übersicht, die auf den bereits entwickelten Backend-Endpunkten basierte, als Referenzdokument für die Feature-Liste.

Für die Frontend-Entwicklung wurde auf Basis dieses Dokuments ein Entwicklungsplan generiert. Dieser Plan unterteilte sich in zwei Abschnitte: eine Auflistung der gewünschten

Features und einen darauf aufbauenden Entwicklungsleitfaden, der die Implementierungsreihenfolge der Features, Dateien und Komponenten festlegt.

In der initialen Frontend-Entwicklung lag der Schwerpunkt auf den Features. Das bedeutet, dass Querschnitte durch die verschiedenen Schichten der Anwendung gezogen und generiert wurden. Dieses Vorgehen zeigte allerdings Schwächen, da rasch der Überblick über die noch zu implementierenden Features verloren ging. Auch stellte der Wechsel in den Bestandteilen, etwa zwischen Store und Vue Dateien, der Applikation ein Problem dar. In einem zweiten Anlauf orientierte sich die Entwicklung daher an den verschiedenen „Schichten“ des Frontends.

Schicht 1 Stores

Schicht 2 Pages

Schicht 3 Components

Um die Kontexte der beiden Entwicklungsversuche sauber zu trennen, wurde ein separater Chat initiiert. Dieser konnte erneut durch den „Frontend Development Plan“ angereichert werden, was einen schnellen Wiedereinstieg in den Entwicklungsprozess ermöglichte.

Dieser strukturierte Ansatz verbesserte die Generierungsreihenfolge, insbesondere weil die verschiedenen Schichten in der Regel nicht mehr nachträglich modifiziert werden müssen. Die Stores agieren als Schnittstellen für alle REST-API Endpunkte. Die nächste Schicht besteht aus den einzelnen Seiten, die in der Navigationsleiste repräsentiert sind und die Features durch ihre Komponenten abbilden. Unter Verwendung von NuxtJS wird für jede im „Pages“-Ordner erstellte Datei automatisch eine Route generiert. Dies ermöglicht eine nahezu direkte Übertragung der Stores auf die Seitenschicht, illustriert am Beispiel der Staff-Seiten.

```
/pages
  /staff
    [id].vue
    import.vue
    index.vue
    /create
      [[id]].vue
```

In der Regel entspricht eine Seite einer Action im Store und damit einem Endpunkt im Backend. Die dritte Schicht, bestehend aus Komponenten, bietet zusätzliche Features, die entweder die Dateigröße einer Seite aufblähen würden oder wiederverwendbar sind. Als Beispiele dienen hier die Komponenten für Fehler- und Erfolgsmeldungen sowie die einzelnen Dashboard-Komponenten.

Während der Frontend-Generierung ließen sich kontinuierlich neue Routen zum Backend integrieren. Die schichtbasierte Struktur legte hierbei die Reihenfolge für das Hinzufügen einer neuen Route fest. Nach der Implementierung eines Endpunkts im Backend wurde die entsprechende Methode im Store hinzugefügt und konnte von der neu generierten Seite verwendet werden. Diese Architektur gewährleistet eine einfache Umsetzung von Änderungen in den einzelnen Schichten. Dabei wurde stets darauf geachtet, dass Änderungen an bestehenden Seiten minimale Auswirkungen auf den Kontextspeicher haben.

5.3 Generierung der Features

Um eine möglichst qualitative Ausgabe durch ein chatbasiertes KI-Modell zu erhalten, darf der Prompt nicht zu spezifisch, aber auch nicht zu allgemein sein. Dadurch wird gewährleistet, dass die Ausgabe eine gewisse Kreativität enthält, die in den meisten Fällen durch den gegebenen Spielraum viele nicht spezifizierte Aspekte abdeckt. Wenn im Backend eine REST-Route generiert werden soll, wären folgende Level von Spezifität möglich:

Level 1 Du bist ein Backend-Webentwickler. Bitte generiere einen REST-Endpunkt in NestJS, über den alle Events abgefragt werden können

Level 2 Du bist ein Backend-Webentwickler. Bitte generiere einen REST-Endpunkt in NestJS, über den alle Events abgefragt werden können. Bitte achte darauf, Fehler abzufangen und entsprechende Server Codes zurückzugeben.

Level 3 Du bist ein Backend-Webentwickler. Bitte generiere einen GET Endpunkt mit Hilfe von REST in NestJS, über den alle Events abgefragt werden können. Bitte achte darauf, einen Statuscode 404 zurückzugeben, wenn kein Event gefunden werden konnte und einen Statuscode 200, wenn der Aufruf erfolgreich war

Alle drei Prompts können eine Problematik mit sich bringen. Mit dem Level 1 Prompt besteht das Problem, dass unter Umständen gar keine Fehler gemeldet werden und wenn ein Fehler auftritt, dieser im Frontend nicht lesbar ist. Der Level 2 Prompt wird aufgrund des gelassenen Freiraums Servercodes zurücksenden, gegebenenfalls aber einige Fälle nicht abdecken. Wenn der Prompt noch spezifischer werden würde, etwa mit dem Level 3 Prompt ist die Wahrscheinlichkeit hoch, dass nur die Servercodes zurückgegeben werden, die auch im Prompt definiert sind. Die Kreativität wird somit im dritten Prompt stark gehemmt und verhindert, dass intuitive Aspekte beim Entwickeln nicht durch das KI-Modell betrachtet werden.

Im Rahmen des Projekts gibt es zwar festgelegte must-have Features, jedoch wurde die Ausgestaltung der Features bewusst nicht festgelegt. Das hat den genannten Hintergrund, dass zu spezifische Anforderungen die Kreativität des KI-Modells hemmen würden und die Ausgabequalität dementsprechend sinkt. Um die Webanwendung zu generieren, kann im allgemeinen gesagt werden, dass die Planung mit Level 1 Prompts ausgestaltet wird, um auch Aspekte abzudecken, die von menschlicher Seite im Vorhinein nicht abgedeckt wurden.

Komponenten, Seiten, Endpunkt, usw. werden grundsätzlich mithilfe von Level 2 Prompts generiert. Level 3 Prompts hingegen kommen dann zum Einsatz, wenn es sich um bestimmte Teile der Anwendung handelt, über die die KI kein Wissen besitzt. Das können Feinheiten einer Config-Datei in einer neuen Framework-Version sein, auf die die KI nicht getrimmt ist. Weiterhin können Level 3 Prompts dafür verwendet werden, Fehler zu beheben oder Features zu generieren, die nach mehrfachen Versuchen mit Level 2 Prompts gescheitert sind.

5.4 Bewertung der Codequalität

5.4.1 Benennung im Code

Zunächst kann positiv angemerkt werden, dass die Routen, Methoden oder Variablen sprechend benannt wurden. Der Code kann aufgrund dieser Gegebenheit also gut gewartet und verstanden werden. Als Beispiel dient folgende Methode aus dem Event Service im Backend (Abbildung 5.10) und folgender Skript-Teil aus der Komponente (Abbildung 5.11) „UpcomingEvents.vue“, die im Dashboard Anwendung findet. Dennoch muss angemerkt

```
62 | async addStaffFromLine(eventId: string, lineId: string): Promise<Event> {  
63 |     const staffMembers = await this.staffModel.find({ line: lineId });  
64 |     const staffIds = staffMembers.map((staff) => staff._id);  
65 |     return this.eventModel.findByIdAndUpdate(  
66 |         eventId, { $addToSet: { staff: staffIds } }, { new: true }  
67 |     ).exec();  
68 | }
```

Abbildung 5.10: Backend Methode aus dem Event Service

werden, dass es Aspekte gibt, die nicht abgedeckt werden. Darunter zählt das Implementieren der REST-Endpunkte in der Mehrzahl. Demnach sollte der Endpunkt für Events unter *localhost : 3000/events* nicht unter *localhost : 3000/event* zu finden sein.

```
21 <script>
22 import { ref, onMounted, computed } from 'vue';
23 import { useEventAssociationStore } from '~/store/eventAssociationStore';
24 import { useUserStore } from '~/store/userStore';
25
26 export default {
27   setup() {
28     const upcomingEvents = ref([]);
29     const eventAssociationStore = useEventAssociationStore();
30     const userStore = useUserStore();
31
32     const userId = computed(() => userStore.userId);
33
34     onMounted(async () => {
35       await userStore.loadUserId();
36       await eventAssociationStore.fetchEventsForStaff(userId.value);
37       const allEvents = eventAssociationStore.eventsForStaff;
38
39       upcomingEvents.value = allEvents.filter(event => new Date(event.date) > new Date());
40     });
41
42     const formatDate = (date) => {
43       return new Date(date).toLocaleDateString();
44     };
45
46     return {
47       upcomingEvents,
48       formatDate,
49     };
50   },
51 };
52 </script>
```

Abbildung 5.11: Frontend Skript aus der Komponente UpcomingEvents.vue

5.4.2 Errorhandling

Da grundsätzlich darauf verzichtet werden soll, Code nachträglich zu bearbeiten, um eine möglichst klare Speicherung im Kontext zu behalten, gibt es viele Dateien, die das Thema Errorhandling vernachlässigen. Insbesondere im Backend werden grundsätzlich keine Fehlermeldungen generiert, wenn dies nicht dementsprechend spezifiziert wurde. Grundsätzlich kann aber angenommen werden, dass das Errorhandling ein Hauptbestandteil der Softwareentwicklung und des sicheren und stabilen Codes darstellt und daher erwartet werden kann, dass KI-Systeme zukünftig auf solche Faktoren besser getrimmt werden.

5.4.3 Weiteres

Weiterhin kann gesagt werden, dass die Generierung der Stores neben der Generierung des Backends am reibungslosesten verlief. Auf den Effizienzfaktor wird im kommenden Kapitel noch näher eingegangen. Trotz dessen ist die Ausgestaltung der sogenannten Actions im Store nicht intuitiv für den menschlichen Entwickler. Als Beispiel dient folgender Ausschnitt aus dem lineStore im Frontend (5.12):

```
3 import { defineStore } from 'pinia';
4 import axios from 'axios';
5
6 export const useLineStore = defineStore({
7   id: 'line',
8   state: () => ({
9     lines: [],
10    lineById: null,
11  }),
12  actions: {
13    > async fetchLines() { ...
21  },
22    async fetchLineById(id) {
23      try {
24        const response = await axios.get(`http://localhost:3001/api/line/${id}`);
25        this.lineById = response.data;
26        return null;
27      } catch (error) {
28        return 'An error occurred while fetching line data by ID.';
29      }
30    },
31    > async createLine(lineData) { ...
39  },
40  },
41  });
```

Abbildung 5.12: Ausschnitt aus der Datei lineStore.js im Frontend

Hier wird gezeigt, dass die Methode „fetchLineById“ den Response Wert in einer State Property abspeichert, anstatt ihn als Rückgabewert der Action zu definieren. Durch diesen Zwischenschritt werden auch Performance Aspekte vernachlässigt.

Bei der Generierung der Seiten und Komponenten ist durch diese Art des Zugriffs auf etwa eine Line das Problem entstanden, dass ChatGPT in den Komponenten versucht auf den Rückgabewert der Methoden zuzugreifen, nicht etwa auf die State Property. Dies zeigt auch, dass zum einen der Kontextspeicher nicht mehr auf die ursprünglich generierte Store Datei zugreifen konnte und zum anderen, dass der Wissensstand und die Intuitivität in der Ausgabe der KI noch stark schwankt.

Kapitel 6

Auswertung

6.1 Datenaufbereitung

Da es sich bei ChatGPT um einen Chatbot handelt, dessen Ausgaben nicht einem Muster folgen oder immer in demselben Ausgabeformat sind, muss zunächst ein Weg gefunden werden, wie das Vorgehen beim Entwickeln in ein zu bewertendes Schema überführt werden kann. Zum einen sind die Eingaben durch den Nutzer bis zu einem gewissen Grad „willkürlich“ und zum anderen kann die Ausgabe immer Sätze oder Wörter beinhalten, die nicht zur eigentlichen Ausgabe gehören, sondern gewissermaßen einen Overhead darstellen. Häufig werden in der Ausgabe Floskeln verwendet, wie etwa: „Certainly, you can add a PUT route to update a vote based on...“ und vielen weiteren. Diese zusätzlichen Sätze erhöhen die Zeit, die benötigt wird, bis die Ausgabe erst einmal generiert ist und weiterhin bis die Ausgabe auf das wesentliche reduziert wurde, um beispielsweise den generierten Code kopieren zu können. Zu den festen Faktoren, aus denen objektiv bewertbare Aussagen gezogen werden können, gehören:

- Ziel erreicht?
- Ausgabe Qualität, gemessen an der Funktionalität des generierten Codes
- benötigte Rückfragen
- Veränderungen des Prompts
- Prompt Länge
- Antwort Länge

Anhand dieser Faktoren können im Anschluss verschiedenste Aussagen getroffen werden, mit denen sich der Nutzen und die Qualität in der Verwendung bewerten lassen. Über die Frage, ob das Ziel erreicht wurde, soll abgebildet werden, ob die Ausgabe durch das Modell die Anforderungen des Prompts deckt. Dabei dürfen Rückfragen gestellt oder der Prompt verändert werden. Es geht also bei der Erreichung des Ziels darum, ob das Modell durch

Prompts und Rückfragen, eine Ausgabe generieren kann, die den Anforderungen entspricht.

6.1.1 Der Effizienzfaktor

Im Rahmen der Bewertung wurde eine Formel für die Berechnung eines direkt ablesbaren Effizienzfaktors aufgestellt. Dieser Effizienzfaktor ist abhängig von den anderen Faktoren und bildet diese zusammengefasst auf einer Skala von 0 bis 1 ab. Die Formel lautet wie folgt:

$$\begin{aligned} & \frac{1}{5} \times \left(\frac{\text{Antwort Länge}}{\text{Prompt Länge} + \text{Antwort Länge}} \right. \\ & + \text{Ziel erreicht} \\ & + \frac{1}{1 + \text{benötigte Rückfragen}} \\ & + \frac{\text{Ausgabequalität}}{5} \\ & \left. + \frac{1}{1 + \text{Prompt Veränderungen}} \right) \end{aligned}$$

Die Formel besteht aus 5 Teilen. Im ersten Teil ($\frac{\text{Antwort Länge}}{\text{Prompt Länge} + \text{Antwort Länge}}$) wird das Verhältnis zwischen den eingegebenen und ausgegebenen Zeichen berechnet und auf eine Zahl zwischen 0 und 1 reduziert. Dieser Teil dient dazu, bewerten zu können, wie gut das Verhältnis zwischen den manuell eingetippten und den dadurch produzierten Zeichen ist. Somit kann gesagt werden, ob sich die Eingabe lohnt, oder, wenn der Term gegen 0 geht, das manuelle Schreiben grundsätzlich schneller wäre. Mit dem zweiten Term wird 0 oder 1 aufaddiert, je nachdem, ob das Ziel erreicht wurde oder nicht. Wenn das Ziel nicht erreicht wurde, muss entsprechend der Effizienzfaktor angepasst werden. Da aber häufig Ausgaben erzielt werden, die ein sehr gutes Verhältnis zwischen Ein- und Ausgabe und den anderen Termen besitzen und lediglich einige Fehler im Code haben, darf dieser Term nicht zu stark gewichtet werden. Der dritte Term berücksichtigt erneut die Zeit, die benötigt wird, wenn Rückfragen gestellt werden müssen. Auch hier wird wieder eine Ausgabe zwischen 0 und 1 erzeugt. Mit dem vierten Term wird die Ausgabequalität auf einer Skala von 1 bis 5 wie folgt bewertet:

- 1 funktioniert nicht, Antwort beinhaltet Fehler oder Rückfragen > 5
- 2 funktioniert teilweise, es sind bei weitem nicht alle Anforderungen abgedeckt und er beinhaltet Fehler
- 3 funktioniert und die Grundanforderungen laufen, beinhaltet aber teilweise Fehler
- 4 funktioniert, aber nicht alle Anforderungen abgedeckt
- 5 funktioniert und alle Anforderungen sind abgedeckt

Der fünfte und letzte Term berücksichtigt die benötigten Promptveränderungen. Die Veränderungen unterscheiden sich zu den Rückfragen, da der ursprüngliche Prompt verändert und

erneut gestellt wird. Es wird somit nicht auf die Ausgabe des Modells eingegangen, sondern eine gänzlich neue versucht zu erzeugen. Durch dieses Vorgehen entsteht wieder ein großer Zeitaufwand für den Nutzer, bzw. Entwickler.

6.1.2 Übertrag in Airtable

Airtable ist ein SaaS Produkt, welches verschiedene Lösungen in einer APP zur Verfügung stellt. Dazu gehört das Erstellen von komplexen Datenbanktabellen, Automatisierungen oder dem Design von Interfaces. Im ersten Schritt können hier Daten frei gestaltet und angelegt werden. Das Interface bietet die Möglichkeit, verschiedene Charts, KPIs oder Listen anhand der abgelegten Daten darzustellen. Nachfolgend ist ein Ausschnitt aus den übertragenen Chats in die Datenbank (6.1):

	Id	Ziel	Prompt	Antwort	Ziel erreicht?	Antwort Qualität
1	111	zweites Modul erstellen	please provide me with th...	# Generating the code fil...	✓	★★★★★
2	112	Drittes Modul erstellen	please provide me with th...	Certainly, I'll create a basi...	✓	★★★★★
3	113	Beispiel Datei für den Imp...	provide me with an exam...	# Create an example CSV...	✓	★★★★★
4	114	Testen des Kontakt impor...	how would the url look lik...	The error suggests that t...	✓	★★★★
5	115	viertes Modul erstellen	please provide me with th...	Certainly, I'll provide you ...	✓	★★★★
6	116	fünftes Modul erstellen	please provide me with th...	Certainly, I'll provide you ...	✓	★★★★★
7	117	Dokumentation erstellen i...	Is there a way to create a ...	Certainly, here's a more d...	✓	★★★★

Abbildung 6.1: Ausschnitt aus der Tabelle, die zur Auswertung der Daten verwendet wird (zur besseren Darstellung wurden nicht alle Felder abgebildet).

6.2 KPIs und Charts

Anhand der Faktoren können im Weiteren verschiedene aussagekräftige Darstellungen generiert werden, mithilfe derer der Nutzen für Webentwickler im Rahmen eines größeren Webprojektes bewertet werden kann. Ziel soll es sein, mithilfe der Auswertungen einen Anhaltspunkt für Webentwickler zu bieten und die Erfahrungen durch die Generierung zu teilen. So kann den Unternehmen aufgezeigt werden, in welchen Bereichen der Einsatz von KI in der Entwicklung sich besonders effizient darstellt.

6.2.1 Prompts je Kategorie

Das folgende Balkendiagramm (siehe Abbildung 6.2) bildet ab, wie viele Prompts insgesamt benötigt wurden, um den jeweiligen Bestandteil der Webapp zu generieren. Deutlich wird, dass das Frontend mit 151 Prompts den größten Umfang und Aufwand in der Generierung besitzt. Das Backend mit 45 Prompts hingegen ist in der Entwicklung mithilfe von KI überschaubar. Hier muss in Betracht gezogen werden, dass es sich um eine Fullstack Webanwendung mit vollständigem Frontend und Backend handelt. Daraus lässt sich bereits ableiten, in welchem Einsatzgebiet besonders schnell ein Projekt entwickelt werden kann.

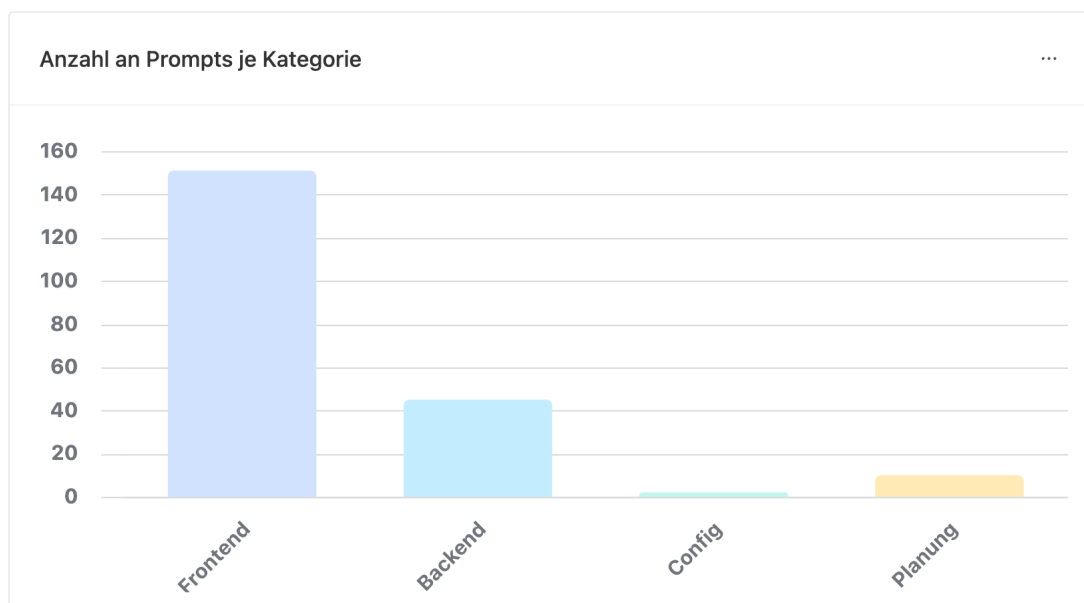


Abbildung 6.2: Darstellung, wie viele Prompts je Kategorie benötigt wurden

6.2.2 Lines of Code je Prompt

Um die Aussagekraft der Grafik 6.2 zu erhöhen, müssen die generierten Zeilen an Code mit in Betracht gezogen werden (siehe Abbildung 6.3). So besteht das Frontend aus insgesamt 2439 Zeilen Code und ist damit 1400 Zeilen größer, als das Backend, welches 1051 Zeilen Code umfasst. Im Verhältnis fällt allerdings auf, dass für das Backend mit 23 Zeilen Code pro Prompt durchschnittlich 7 Zeilen Code mehr pro Prompt generiert werden konnten als im Frontend mit 16 Zeilen Code pro Prompt. Die Effizienz rein auf das Verhältnis zwischen Prompts und Zeilen Code reduziert, ist für das Backend daher um den Faktor 1,4 höher. Je größer das Projekt, desto stärker kann dieser Faktor ins Gewicht fallen.

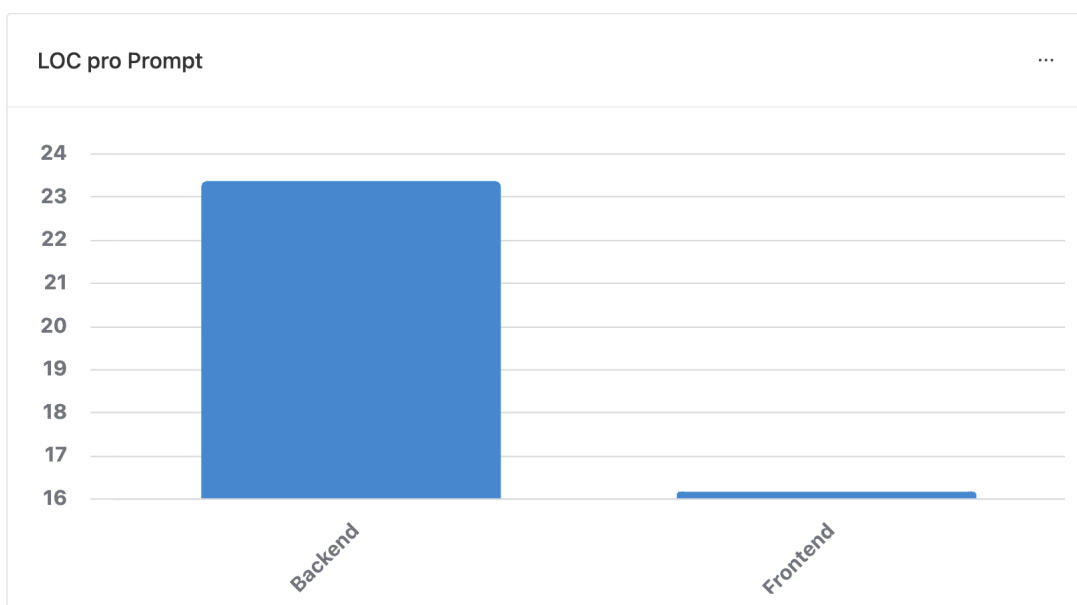


Abbildung 6.3: Darstellung, wie viele Zeilen Code mit jedem Prompt im Durchschnitt generiert wurden

6.2.3 Effizienz Faktor je Kategorie

In der Sektion „Der Effizienzfaktor“ (6.1.1) wurde beschrieben, wie sich dieser zusammensetzt und berechnet. Im Folgenden wird dargestellt (siehe Abbildung 6.4), wie dieser in den einzelnen Kategorien ausfällt. Hier zeigt sich wieder, dass das Backend mit einem Faktor von 0.92 den höchsten Effizienzfaktor nach der Planung mit 0.97 besitzt. Grundsätzlich ist die Planung mit Level 1 Prompts (siehe 5.3) die Phase mit den geringsten Anforderungen. Hier steht das Brainstorming im Fokus sowie das Betrachten unterschiedlicher Blickwinkel und Herangehensweisen. Somit wird das Ziel nahezu immer erreicht und das Verhältnis zwischen Prompt- und Ausgabelänge ist besonders hoch, da die Kreativität des KI-Modells kaum gehemmt wird.

Im Backend ist die Anzahl unterschiedlicher Herangehensweisen eingeschränkter. Wenn vor-

gegeben wird, dass ein REST-Endpoint in NestJS mit MongoDB als Datenbank implementiert werden soll, gibt die Architektur des Frameworks und der Technologien bereits stark vor, wie ein solcher Endpoint zu implementieren ist. Es lässt sich auch vermuten, dass aus diesem Grund der Trainingsdatensatz von ChatGPT nicht so viele verschiedene Ansätze aufweist, wie es im Frontend Wege gibt, eine Seite auszugestalten. Das Frontend ist mit einem Effizienzfaktor von 0.86 immer noch effizient in der Generierung, reiht sich allerdings hinter dem Backend ein. Wenn nun der Effizienzfaktor noch in ein Verhältnis zu den generierten Zeilen Code je Prompt gesetzt wird, dann zeigt sich die Diskrepanz zwischen Frontend und Backend noch einmal deutlicher.

Wird der Effizienzfaktor mit den Zeilen Code je Prompt multipliziert ergibt sich für das Frontend folgender Faktor: $16 \times 0.86 = 13.76$. Für das Backend hingegen ergibt sich ein Faktor von 21.16. Im Verhältnis hat somit das Backend deutlich den Effizienzvorteil in der Generierung.

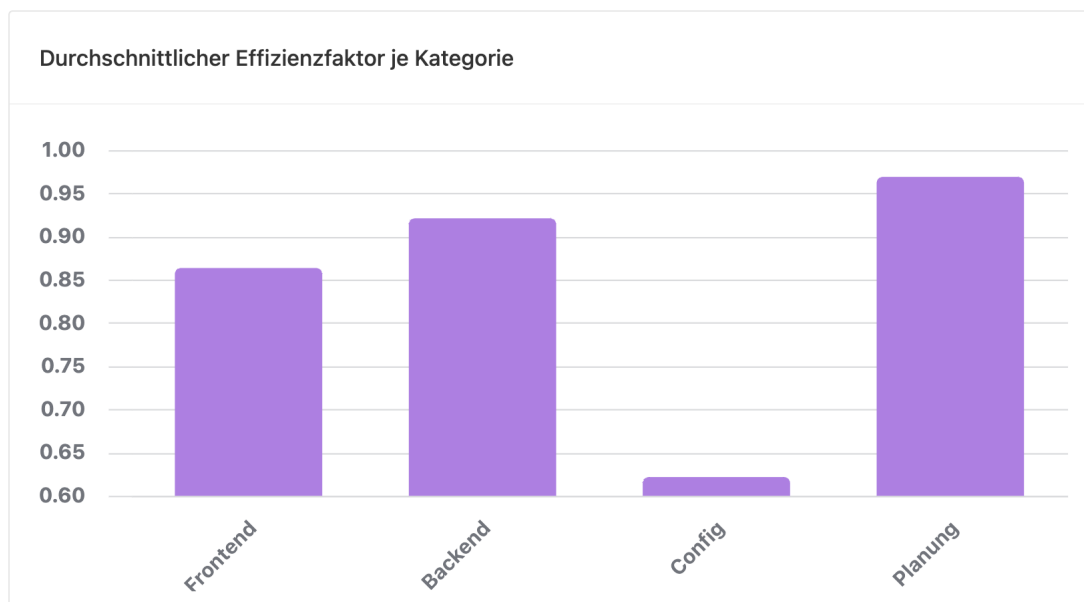


Abbildung 6.4: Darstellung des Effizienzfaktors je Kategorie

6.2.4 Effizienzfaktorverlauf bei gestellten Rückfragen

Ein weiterer wichtiger Aspekt ist die Anzahl der gestellten Rückfragen. Die nachfolgende Grafik (siehe Abbildung 6.5) zeigt sehr deutlich, dass der Effizienzfaktor stark sinkt, je mehr Rückfragen gestellt werden müssen. Das zeigt, dass ein gut gestaltetes Kontextwissen oder Prompt einen großen Einfluss auf die Effizienz in der Generierung von Frontend oder Backend besitzt. Zu Beginn des Projekts wurde der Chat mit dem Kontext und den Zielen der Webanwendung angereichert. Durch diese Anreicherung wurden von ChatGPT Phasen vorgeschlagen, auf die im Chat immer wieder durch den Nutzer zurückgegriffen werden kann

oder häufig auch ohne weitere Anweisung durch ChatGPT vorgeschlagen wird. Dadurch genügt in einigen Fällen ein einfaches „Ja“ als Prompt, um mit der Generierung der nächsten Phase oder Datei fortzufahren. Dadurch steigert sich zum einen das Verhältnis zwischen Prompt- und Ausgabelänge und zum anderen zeigt ChatGPT dadurch, dass die Anforderungen noch klar definiert sind. Die Verbindung aus dem Kontextwissen und einem einfachen „Ja“ ergibt in Summe einen Prompt mit einer hohen Effizienz. Selbst bei 1-2 gestellten Rückfragen befindet sich der Effizienzfaktor noch im sehr guten Bereich, oberhalb von 0.8.

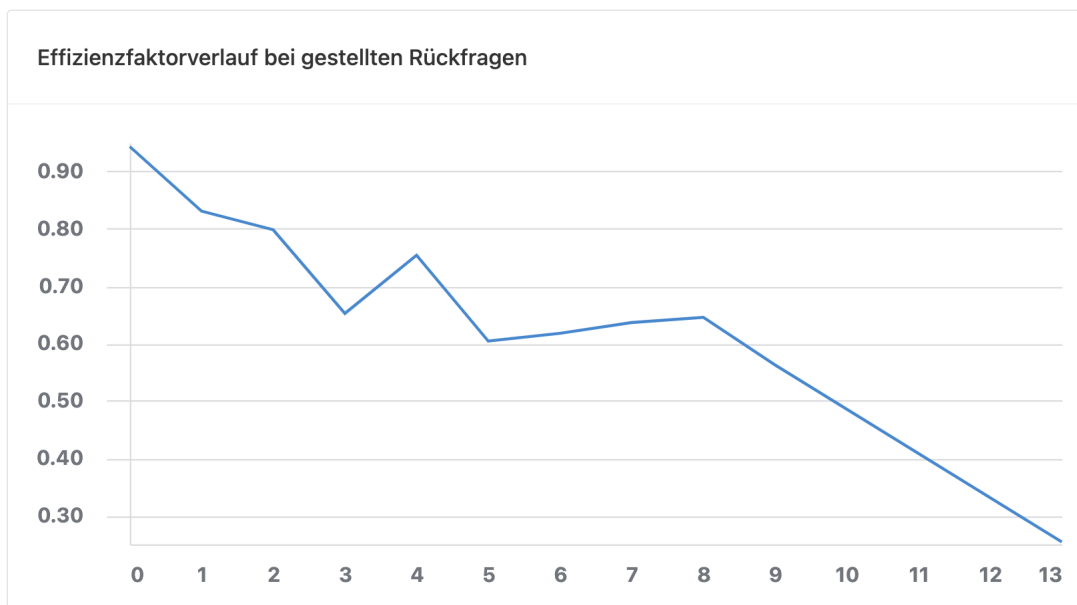


Abbildung 6.5: Darstellung des Effizienzfaktorverlaufs bei gestellten Rückfragen

6.2.5 Verwendung von Prompt Bricks

In der Sektion „Ansätze von Prompt Engineering“ (4.3.2) wurde bereits dargestellt, dass es unterschiedliche Ansätze gibt, mit denen ein Prompt ausgestaltet werden kann. Diese Ansätze können durchaus auch miteinander kombiniert werden. Im Rahmen der Auswertung des Projekts habe ich diese Ansätze als „Prompt Bricks“ bezeichnet, um auch hier eine Auswertung treffen zu können. Die Grafik 6.6 stellt die prozentuale Nutzung der einzelnen Bausteine dar. Dabei zeigt sich, dass am häufigsten der Brick „I cannot write any code myself...“ verwendet wurde. Das hat den Hintergrund, dass das Ziel in dieser Arbeit keine Zeile Code zu schreiben einen hohen Stellenwert besitzt.

6. AUSWERTUNG

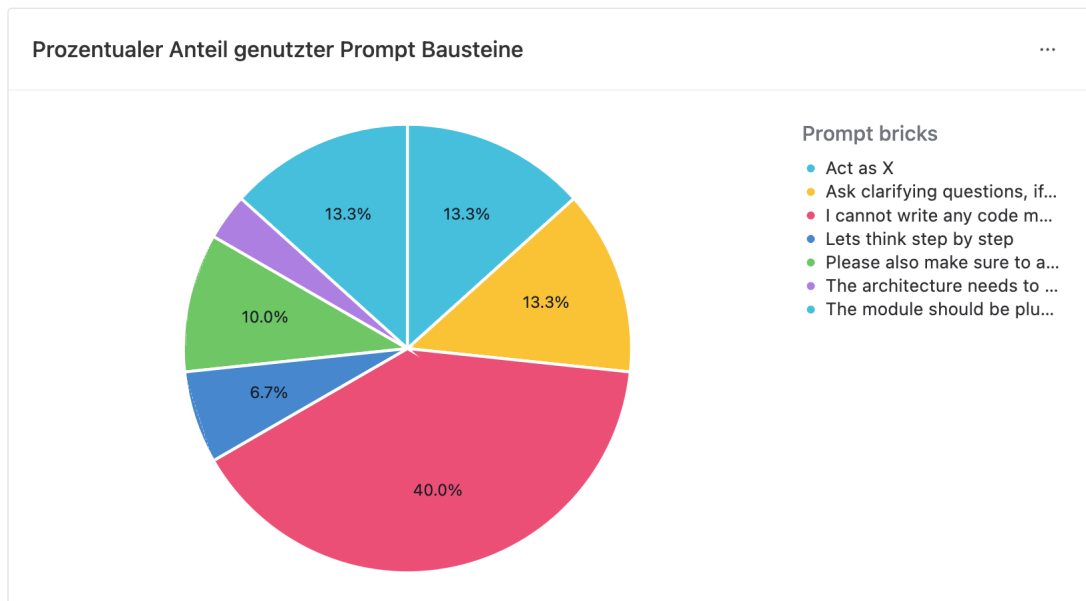


Abbildung 6.6: Darstellung der Nutzung von Prompt Bricks

6.2.6 Verwendung von Prompt Bricks

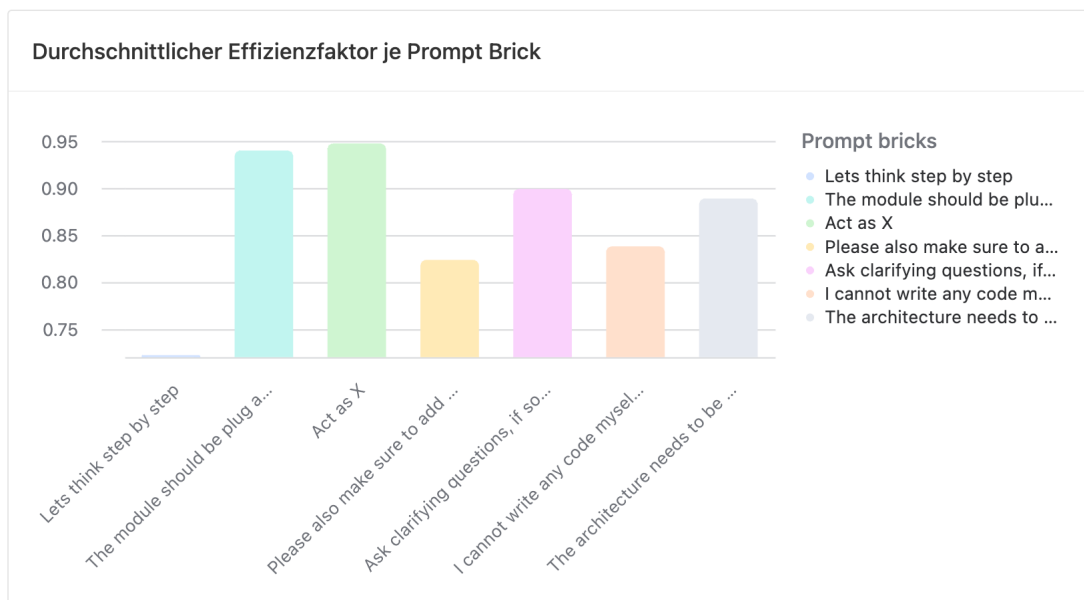


Abbildung 6.7: Darstellung des Effizienzfaktors je Brick

In der Grafik (siehe Abbildung 6.7) wird der durchschnittliche Effizienzfaktor je Prompt

Brick dargestellt. Dabei fällt auf, dass mithilfe von „Lets think step by step“ nur ein unterdurchschnittlicher Effizienzfaktor sowohl für das Frontend als auch für das Backend erzielt werden kann. Der Faktor von 0.72 lässt sich darauf zurückführen, dass mit GPT-4 ein sehr fortschrittliches KI-Modell dem Nutzer zur Verfügung gestellt wird, dass bereits selbstständig sehr gut zusammenhängend denken kann. Durch die Verwendung dieses Satzes wird die Kreativität eingeschränkt und Komplexität in einen Prompt gebracht, der unter Umständen durch das Modell einfacher zu lösen wäre.

Eine zweite wichtige Auswertung ist der durchschnittliche Faktor unter Verwendung des Bricks „I cannot write any code myself...“. Im Vergleich mit dem durchschnittlichen Faktor im Frontend von 0.86, verschlechtert sich der Faktor um 0.02 Punkte. Das lässt sich darauf zurückführen, dass Prompt Bricks insbesondere dann verwendet werden, wenn der Kontext nicht klar gesteckt ist oder es sich um eine komplexere Aufgabe oder Datei, die zu generieren ist, handelt.

Insgesamt ist sichtbar, dass die Bausteine „The module should be plug and play“ sowie „Act as X“ den höchsten Effizienzfaktor aufweisen. Begründen lässt sich das, durch die Tatsache, dass bei ersterem bereits der Baustein „I cannot write any code myself...“ indirekt inkludiert ist. Bei letzterem steigt im Allgemeinen die Ausgabequalität durch die Anforderung an das Modell, eine professionelle Ausgabe zu erzeugen.

6.2.7 Rückfragen bei Prompt Bricks

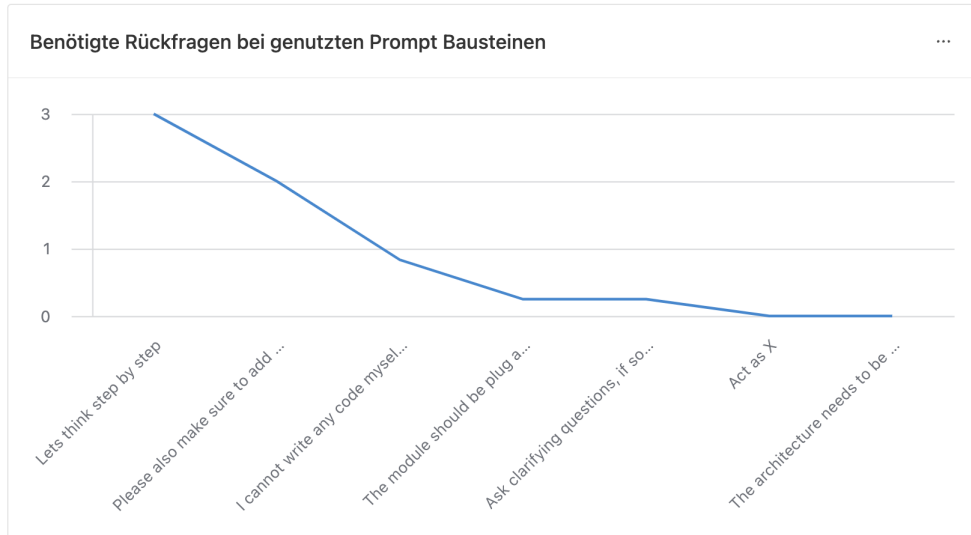


Abbildung 6.8: Darstellung der Rückfragen je Prompt Brick

Die Analysen bestätigen sich durch die Grafik 6.8. Mit dem Prompt Brick „Lets think step by step“ wird das Modell dazu angehalten, die Ausgaben aufzuteilen. Dadurch werden mehr Rückfragen benötigt, wodurch der Effizienzfaktor sinkt und mehr Zeit investiert werden muss, um die gewünschte Ausgabe zu erzielen.

Weiterhin bestätigt sich, dass die Rückfragen der beiden Bausteine, die den höchsten Effizienzfaktor besitzen, („The module should be plug and play“ und „Act as X“) stark gegen null gehen und die Annahme bestätigen, dass es sich in Verbindung mit GPT-4 um Bausteine handelt, die durchaus die Ausgabequalität verbessern.

6.2.8 Wurde ein Prompt Brick verwendet

Insgesamt ist der Effizienzfaktor unter Verwendung von Prompt Bricks mit einem Wert von 0.86 niedriger, als der Prompt ohne die Verwendung eines Bausteins mit einem Wert von 0.88 (siehe Abbildung 6.9). Einerseits wird der Faktor durch die Untersuchung der Bricks, die letztendlich einen niedrigen Faktor aufweisen, negativ beeinflusst. Andererseits zeigt sich dadurch deutlich, dass ein gut angereichertes Kontextwissen wichtiger ist, als die Verwendung eines Prompt Bricks. Die Qualität der Ausgabe in ChatGPT kann durch ein gut konfiguriertes Gedächtnis erheblich gesteigert werden. Auch hier muss in Betracht gezogen werden, dass die Fortschrittlichkeit von GPT-4 keine Spezifität im Prompt voraussetzt, wie am Beispiel des Bricks „Lets think step by step“ zu sehen ist.

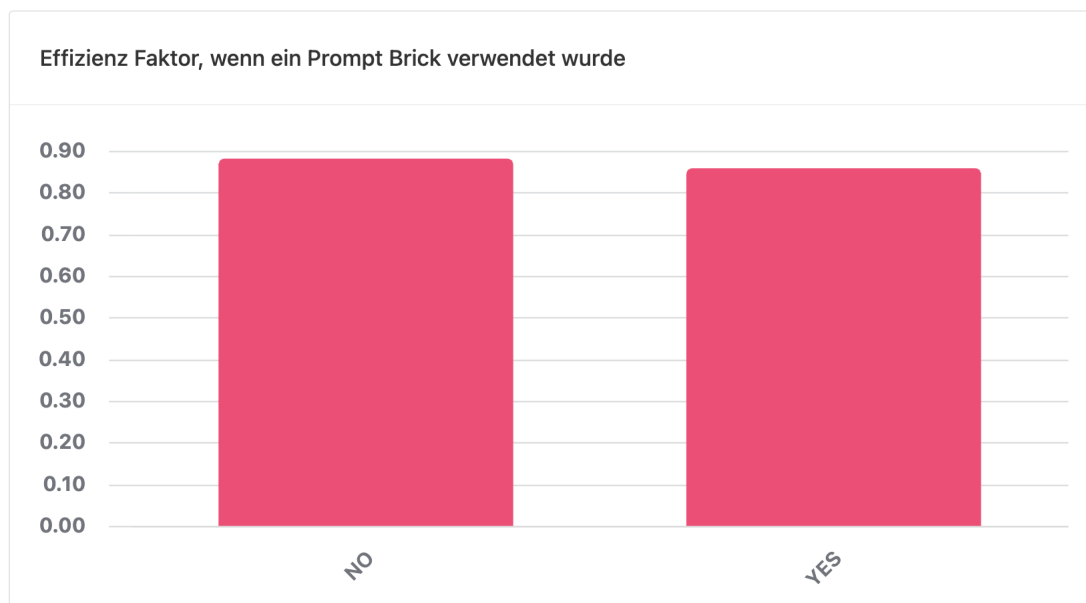


Abbildung 6.9: Darstellung des Effizienzfaktors, wenn ein Prompt Brick verwendet wurde

6.3 Aufgetretene Grenzen

6.3.1 Zeitersparnis

Die durchschnittliche Zeit, die aufgebracht werden muss, um einen Prompt zu entwickeln, kann schwer gemessen werden. Jedoch sollte betrachtet werden, dass trotz des hohen Effizienzfaktors oder der generierten Zeilen an Code pro Prompt, neben dem Konstruieren

des Prompts einiges an Zeit aufgebracht werden muss. Dazu gehört zum einen die Aufbereitung des Ergebnisses. Das bedeutet, dass die generierte Antwort zunächst in die lokale Entwicklungsumgebung überführt werden muss. Zum anderen kann ein fehlender oder nicht optimal aufbereiteter Kontextspeicher einen Overhead verursachen, der in der manuellen Entwicklung nicht auftritt.

6.3.2 Kontextspeicher

Ein optimal befüllter Kontextspeicher kann viel Zeit sparen, da die Qualität des Prompts nicht alleine ausschlaggebend für die Ausgabequalität ist. Der Kontextspeicher muss allerdings zunächst befüllt werden und ist in seiner Kapazität begrenzt. Dieser Aspekt ist insbesondere für die industrielle Softwareentwicklung von großer Bedeutung. Es kann nicht vorausgesetzt werden, dass die Webanwendung vollständig durch KI-generiert wird. Vielmehr verlangt es hier einen Ansatz, der einen punktuellen Einsatz von Chatbots oder Entwicklungshelfern erlaubt. Voraussetzung hierfür ist aber ein Zugriff auf die lokale Entwicklungsumgebung und somit auf den Kontext der Anwendung sowie live-Zugriff auf sämtliche Datei-Änderungen.

6.3.3 Integration in die industrielle Softwareentwicklung

Durch die Abschirmung des Projekts im Rahmen dieser Bachelorarbeit wurde in gewissem Maße ein geschütztes Umfeld geboten. Das heißt, es gibt einen Entwickler, der ohne äußere Ansprüche die Webanwendung nach im Vorhinein gesetzten Anforderungen generiert. Es gibt somit wenig nachträgliche Änderungen der Anforderungen.

Anhand der gesammelten Erfahrungen kann gesagt werden, dass zunächst ein Problem entsteht, wenn der Entwickler eine Datei manuell bearbeitet, ohne dies im Kontext von ChatGPT anzugeben. Wird also etwas an einer Datei manuell verändert und ChatGPT soll nachträglich ein Feature zu einer Komponente oder Datei hinzufügen, so ist der Kontextspeicher nicht länger aktuell und es kommt zu Ungenauigkeiten und einer schlechteren Ausgabequalität. Dieses Problem muss in weiteren Versuchen genauer betrachtet werden, da die Integration in ein professionelles Umfeld es voraussetzt, dass Dateien verändert werden können, ohne dass dadurch der Effizienzfaktor zu stark sinkt. Im Zuge der schnelllebigen Entwicklung von KI-Tools werden immer mehr Programme veröffentlicht, die auch Zugriff auf die lokale Entwicklungsumgebung haben. Zu diesen Tools gehört GitHub Copilot oder DevGPT. Das Problem: die Tools sind noch spezifischer auf das Generieren von Code ausgelegt. Darüber hinaus beruhen die Tools zu Teilen noch auf alten GPT-Versionen. Daher verändert sich hier der Umgang im Prompting. Bausteine wie „Lets think step by step“ können daher im Kontext von Copilot mit GPT-3 wieder hilfreich werden. Weiterhin ist der Copilot nicht dazu in der Lage, Dateien zu generieren oder in das Repository einzufügen. Das könnte sich mit der Integration durch den Zugriff auf die Kommandozeile mit Copilot-X ändern. Zum aktuellen Zeitpunkt ist das Tool allerdings noch nicht frei verfügbar.

6.3.4 Veraltete Technologieversionen

Ein weiteres Problem besteht unabhängig von der Version des LLMs. Die Tools sind immer auf einem Stand der Daten zu einem bestimmten Zeitpunkt trainiert. Das heißt, dass Tools, die Zugriffe auf das Internet besitzen, dennoch für bestimmte Aufgaben auf ihr antrainiertes Wissen zurückgreifen. Dadurch werden Ausgaben erzeugt, die auf älteren Versionen der Technik beruhen. Im Rahmen des Projekts wurden sowohl im Frontend als auch im Backend Codeausgaben erzeugt, die auf älteren Versionen von Nest- bzw. NuxtJS basieren. Das zeigt insbesondere der Effizienzfaktor von 0.62 im Rahmen der Konfiguration.

6.3.5 Fortbildung der Entwickler

Aus diesem Grund ist es nach wie vor wichtig, dass die Entwickler in der Technologie geschult werden und auch auf dem aktuellen Stand sind, um veraltete Informationen und damit möglicherweise Sicherheitsrisiken oder Performanceprobleme im Code zu vermeiden. Foren wie Stack Overflow sind daher nach wie vor von hoher Bedeutung, um dem Entwickler einen Anhaltspunkt zu bieten, wie Probleme in aktuellen Technologien implementiert werden.

6.3.6 Fazit

Abschließend stellt sich immer die Frage, wie viel Zeit gespart werden kann, wenn künstliche Intelligenz auf diese Art und Weise in den Entwicklungsprozess integriert wird. Eine erste Einschätzung wird durch den Effizienzfaktor gegeben. Sobald allerdings Dateien nachträglich bearbeitet werden, wird der Effizienzfaktor stark sinken, da das Verhältnis aus Prompt- und Ausgabelänge aus der Balance gerät, wenn die neuen Dateien mitgeschickt werden müssen. Weiterhin beeinflussen alte Technologieversionen den Effizienzfaktor, da in diesem Fall Level 3 Prompts benötigt werden, um zur gewünschten Ausgabe zu gelangen.

Die Webanwendung hat gezeigt, dass ganze Webanwendungen ohne manuelles Coden entwickelt werden können. Insbesondere in professionellen Umfeldern, die eine starke Skalierbarkeit und Integrationsfähigkeit in Verbindung mit mehr-köpfigen Teams als Grundanforderungen haben, wird sich der Nutzen von künstlicher Intelligenz allerdings mehr auf das Generieren eines Grundgerüsts beschränken.

Kapitel 7

Ergebnisse

7.1 Ausgangssituation

In der Ausgangssituation wurde zunächst die Frage gestellt, ob es grundsätzlich möglich ist, eine Webanwendung alleine durch chatbasierte KI Systeme zu erstellen. Insbesondere sollte mit diesem experimentellen Vorgehen herausgefunden werden, wo die aktuellen Grenzen für Webanwender liegen. Wo liegen die Grenzen bei Tools und was muss in der Technologie noch weiterentwickelt werden, damit künstliche Intelligenz nahtlos in größere Projekte integriert werden kann oder ob dies bereits möglich ist.

Weiterhin wurden Vermutungen aufgestellt, für welchen Bereich der Anwendung der größte Nutzen auftritt, bzw. in welchem Bereich sich die Effizienz von Webentwicklern durch den Einsatz von KI stark verbessern lässt. Durch die klaren Richtlinien, Vorgaben und weniger vielfältigen Wege ein Backend zu erstellen, wurde hier der Vorteil in der Effizienz vermutet. Hier handelt es sich um Code, der vom Grundsatz her häufiger repetitiv ist.

Außerdem sollen die Grenzen durch das Entwickeln eines Projekts in einem größeren Maßstab betrachtet werden. Vorhandene Arbeiten untersuchen die Tools an sich oder das Generieren von kleineren Codebausteinen und Dateien, nicht aber von Projekten. In den Arbeiten wird sich auch meist auf einen Aspekt der Softwareentwicklung fokussiert. Das kann das KI-basierte Erstellen von Tests oder der Einsatz im Projektmanagement sein.

7.2 Lösungsansatz

Um die genannten Aspekte eingehend zu untersuchen, wurde zunächst ein Grundverständnis in den Bereichen der künstlichen Intelligenz sowie des maschinellen und tiefen Lernens geschaffen. Dieses Wissen ermöglichte ein besseres Verständnis für das Verhalten von ChatGPT und anderen chatbasierten KI-Modellen. Es erlaubte auch die frühzeitige Identifizierung von Problemen, die in der Methodik und Planung berücksichtigt werden konnten.

Zu diesen erkannten Herausforderungen zählten beispielsweise die Gedächtnisbegrenzung von ChatGPT sowie die spezifischen Anforderungen an die Formulierung von Prompts, um gewünschte Ausgaben zu erzielen.

Es wurde deutlich, wie wichtig eine umfassende Planung der Softwarearchitektur im Vorfeld

ist, um nachträgliche Änderungen an Dateien zu minimieren. Eine durchdachte Architektur, die den Großteil der gewünschten Features abdeckt, erhöhte nicht nur die Effizienz, sondern reduzierte auch die Notwendigkeit, den Kontextspeicher mit Dateiänderungen zu belasten. Die sorgfältige Planung der Softwarearchitektur erlaubte auch eine präzise Abschätzung des Umfangs der Anwendung sowie des Kontextspeicherbedarfs in Bezug auf ChatGPT. Es konnten Vorhersagen getroffen werden, wie viele Chats benötigt würden, um eine optimale Qualität der Ausgaben zu erzielen.

Bei der Architekturplanung wurde konsequent darauf geachtet, dass die Kommunikation zwischen Modulen oder Komponenten minimiert wurde. Jegliche Kommunikation wurde idealerweise ausgelagert, um Abhängigkeiten so gering wie möglich zu halten. Dies konnte durch den Einsatz von Stores oder weiteren APIs erreicht werden.

7.3 Evaluation

Nach der Generierung wurde festgestellt, dass die Grenzen und Möglichkeiten von Webentwicklern aktuell stark davon abhängig sind, was entwickelt werden soll. Wie im Kapitel Auswertung (6) bereits dargestellt, weißt die Generierung von Code für das Backend einen höheren Effizienzfaktor als im Frontend auf. Hier lohnt es sich auch zu betrachten, dass die durchschnittlichen Zeilen Code pro Prompt im Backend bei 23 liegen. Bewertet man diese 23 rein zeitlich, so lässt sich ganz grundsätzlich sagen, dass es sich bei einer klar strukturierten Webanwendung durchaus lohnt, einzelne Dateien oder auch ganze Module zu generieren. Insbesondere berücksichtigt die KI bei einem Level 2 Prompt (5.3) viele Aspekte, die automatisch mit generiert werden und einen gewissen Zeitaufwand durch das manuelle Tippen bedeuten würden.

Sowohl im Frontend als auch Backend konnten sehr gute Ausgaben erzielt werden, was durch den Effizienzfaktor von 0.86 im Frontend und 0.92 im Backend sichtbar wird. Wenn die Architektur gut ausgestaltet ist, besteht der Vorteil, dass die Darstellung der Architektur lediglich mithilfe eines angereicherten Kontextspeichers des jeweiligen Chatbots verschriftlicht werden muss, um einiges an manueller Schreiarbeit zu sparen. Allerdings muss auch hier berücksichtigt werden, welche Aspekte im jeweiligen Projekt am stärksten gewichtet sind. Wenn es essenziell ist, dass die Version des genutzten Frameworks immer die aktuelle ist, muss mehr Aufwand betrieben werden, um den generierten Code zu überprüfen. Ist dies weniger im Fokus, so kann ein gutes Grundgerüst geschaffen werden, welches anschließend leicht durch den Entwickler auf die eigenen Bedürfnisse oder Vorgaben angepasst werden kann.

Zukünftig wird es wichtig sein, dass KI-basierte Entwicklungstools auf den modernsten Modellen basieren, um Aufwand im Prompting einzusparen, da weniger Prompt Bricks (6.2.5) verwendet werden müssen. Wenn sich in diesem Zusammenhang auch die Größe des Kontextspeichers weiterentwickelt, wird die Ausgabequalität im Verlauf des Projekts zwangsläufig mit ansteigen. Sobald diese Grenze überschritten oder stark erweitert werden kann, werden mehr Dateiänderungen oder ältere Prompts abgespeichert. Somit wird eine Art

Fine-Tuning ohne allzu großes technisches Vorwissen ermöglicht, um sowohl die Komplexität von Prompts zu verringern als auch die Ausgabe intuitiv zu verbessern.

Darüber hinaus fehlt es nach wie vor an Tools, die sich nicht nur auf das Brainstormen oder rein auf das Programmieren beschränken. Mit GitHub Copilot-X und dem Zugriff auf die Kommandozeile könnten hier starke Fortschritte gemacht werden. Für die Inhaltserstellung muss allerdings nach wie vor auf ChatGPT, Dall-e oder ähnliche Tools zurückgegriffen werden. Hier würde ein „All-in-one-Tool“ große Zeitersparnisse mit sich bringen.

Weiterhin müssen zukünftig Modelle, die speziell für das Coden entwickelt oder getunt werden, darauf trainiert werden, Designaspekte im Frontend zu berücksichtigen. Hier zeigen sich aktuell noch starke Grenzen auf. Durch die Subjektivität im Frontend und visuellen Design sind aktuelle Modelle noch nicht darauf ausgelegt, anregende und nutzerfreundliche Designs zu erstellen oder Vorgaben intuitiv zu berücksichtigen. Hier bedarf es in der Zukunft einer Symbiose aus Tools wie dem UIZard Autodesigner und ChatGPT, die Zugriff auf das Design haben, um dieses direkt in Code zu übersetzen.

7.4 Vergleich zum traditionellen Entwicklungsansatz

Im weiteren Verlauf wird untersucht, welchen Einfluss KI-gestützte und manuelle Entwicklungsansätze auf verschiedene Aspekte der Entwicklung haben. Ziel ist es, eine fundierte Bewertung darüber abgeben zu können, unter welchen Umständen der Einsatz von KI-Tools für Unternehmen oder Entwickler vorteilhaft ist.

7.4.1 Zeitfaktor

Unter den Aspekt Zeit fällt nicht nur das blanke Schreiben von Code, sondern auch der Aufwand in die Konzeptionierung und Suchen von Lösungsansätzen. Durch den Einsatz von KI wird vom Entwickler verlangt, die Zeitinvestition in den Bereich der Konzeptionierung und weg von der eigentlichen Schreibarbeit zu verschieben. Zum einen wird also Zeit gespart, wenn man erneut die durchschnittlichen 23 Zeilen Code pro Prompt betrachtet. Allerdings benötigt der Entwickler auch einiges an Zeit, um den Prompt zu entwickeln, das Kontextwissen des Chatsystems aufzubereiten und die Anwendung zu konzeptionieren. Diese Aspekte dürfen nicht außer Acht gelassen werden.

7.4.2 Coding Richtlinien

Durch vorgegebene Coding Richtlinien oder andere formale Anforderungen, die der Code erfüllen muss, erhöht sich die Spezifität des Prompts. Zum einen verschiebt sich durch die genaueren Anforderungen das Verhältnis zwischen Prompt- und Ausgabelänge hin zur Eingabe und zum anderen verschlechtert sich die Kreativität des Modells durch die klaren Vorgaben. Solange die aktuellen Modelle noch nicht exzessiv auf verschiedene Coding-Richtlinien

trainiert wurden, die für die meisten Unternehmen von großer Bedeutung sind, um wartbaren und lesbaren Code zu schreiben, wird das manuelle Schreiben von Code unter diesem Blickwinkel im Vorteil bleiben.

7.4.3 Komplexität der Features

Weiterhin ist ChatGPT im Vorteil, wenn es sich um Probleme handelt, die häufig auftreten oder bereits von vielen Entwicklern gelöst wurden. Diese Probleme sind dann Bestandteil des Trainingsdatensatzes und somit in der Kernkompetenz der KI. Wenn ein Feature nun aber sehr spezifische Anforderungen besitzt oder von gängigen Vorgehensweisen abweicht, dann rückt der Entwickler wieder in den Vordergrund, um den Code entsprechend der Vorgaben anzupassen.

7.4.4 Updates und Skalierbarkeit

Des Weiteren wurde bereits mehrfach darauf hingewiesen, dass nachträgliche Änderungen stets mit einer Verschlechterung des Effizienzfaktors einhergehen. Das bedeutet auch, dass Bugs und Updates rein durch künstliche Intelligenz nur ineffizient zu beheben, bzw. umzusetzen sind. Das Verändern von Code verbleibt daher in der Hand des Entwicklers, bis der Kontextspeicher massiv ansteigt oder das Finetuning für einzelne Anwendungen anstatt einem Tuning auf Unternehmensebene von der Kosten- und Zeiteffizienz praktikabel wird.

Kapitel 8

Zusammenfassung

8.1 Retrospektive

Im Allgemeinen verlief die Entwicklung der Webanwendung sehr positiv. Die folgende Grafik (8.1) veranschaulicht noch einmal den Effizienzfaktor Verlauf während des gesamten Projekts:

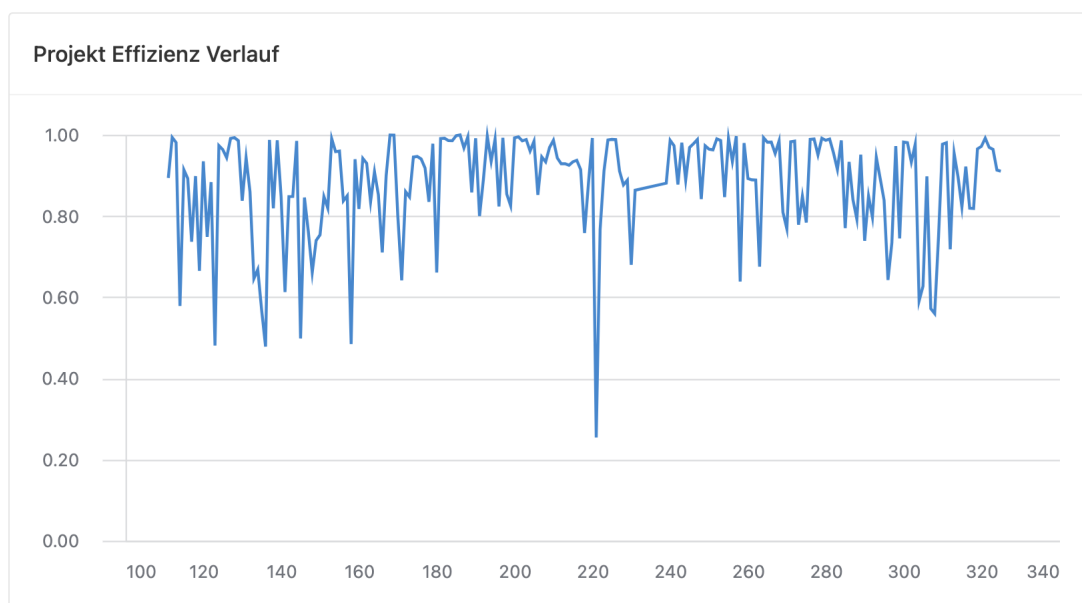


Abbildung 8.1: Darstellung des Effizienzfaktors über das gesamte Projekt

Die Grafik zeigt trotz vieler Kursschwankungen, dass sich die meisten Prompts im guten bis sehr guten Bereich zwischen 0.75 und 1 bewegen. Trotzdem kann gesagt werden, dass das Frontend wesentlich langwieriger in der Generierung ist. Hier spielen viele subjektive Eindrücke in der Generierung eine Rolle. Dadurch, dass das Frontend immer ein visuelles Produkt ist, können die Anforderungen zu Beginn auch nicht so klar gesteckt werden, wie

im Backend. Zwar kann zu Beginn ein Wireframe oder Prototyp generiert werden, jedoch ist es nach aktuellem Stand der Technik kaum möglich einen solchen Prototyp so zu verschriftlichen, dass ein textbasiertes KI-Modell eine Ausgabe erzeugt, die den visuellen Ansprüchen der Vorlage entsprechen. Die Logik im Frontend kann jedoch vergleichbar schnell zum Backend generiert werden. Das Problem stellt hier die grafische Schnittstelle zwischen Code und Nutzer dar.

Das Backend hingegen hat durch die rein logische Abbildung von Abläufen keine visuellen Anforderungen. Hier spielen Aspekte der Sicherheit, Performance, usw. eine übergeordnete Rolle. Um ein Grundgerüst zu erstellen, bedarf es lediglich eines Kontexts über die Technologie und die Anforderungen. Das Abfangen von Fehlern oder weitere Performance Verbesserungen können nachträglich durch den Entwickler schnell implementiert werden, da der repetitive Teil des Codes bereits durch das Modell generiert wurde.

Allgemein kann sowohl für das Backend als auch für das Frontend gesagt werden, dass die Ausgabequalität stark von der Größe einer Datei abhängig ist. Je größer die Datei, desto höher ist das Risiko, dass der Kontextspeicher nicht mehr alle Details auswerten kann, um später Änderungen vorzunehmen. Ziel ist es, Dateien so atomar wie möglich zu halten. Eine gute Softwarearchitektur ist daher unerlässlich und der Grundbaustein für das KI-basierte Entwickeln.

Neben der eigentlichen Entwicklungsarbeit fallen auch einige Planungs- und Konfigurationsaufgaben an. Die Planung verläuft im Durchschnitt mit einem Effizienzfaktor von 0.97 und ist damit mit Abstand am effizientesten. Für das Brainstorming und die Generierung einer Vorgehensweise für ein Projekt kann also sehr gut ein chatbasiertes KI-Modell hinzugezogen werden. Im Kontrast dazu steht die Konfiguration. Da dieser Bestandteil eines Projekts stark versionsabhängig ist, und die Prompts extrem spezifisch sein müssen, um eine den Anforderungen entsprechende Ausgabe zu erzielen, wird nur ein Effizienzfaktor von 0.62 erzielt.

Zusammenfassend lässt sich sagen, dass man grundsätzlich eine Empfehlung für den Einsatz von KI in der Webentwicklung und der Softwareentwicklung aussprechen kann. Durch eine gute Architektur kann schnell ein Grundgerüst erzeugt werden, welches in der Symbiose mit Entwicklern das Projektvorgehen stark beschleunigen kann. Dennoch wird der Entwickler aktuell immer wieder an Grenzen stoßen, welche durch manuelle Arbeit schneller behoben werden als mithilfe von künstlicher Intelligenz. Hier muss durch gesammelte Erfahrungen ein guter Mittelweg gefunden werden, in welchen Bereichen und Technologien sich der Einsatz lohnt und wo nach wie vor ein manueller Ansatz gewählt werden sollte.

8.2 Ausblick

8.2.1 Technologieperspektive

Wie aufgezeigt, existieren je nach Anwendungskontext unterschiedlich ausgeprägte Limitationen. Um diese Grenzen weiter zu senken und die Einstiegsbarriere für den Einsatz von KI

im Entwicklungsprozess zu verringern, bedarf es verstärkter Anstrengungen in Forschung und Technologieentwicklung. Angesichts der aktuellen Fokussierung auf künstliche Intelligenz im technologischen Bereich ist es wahrscheinlich, dass solche Fortschritte in absehbarer Zeit erzielt werden können.

Zunächst muss der Kontextspeicher stark verbessert werden. Die Tools müssen die Fähigkeit besitzen, dass Sie nicht länger Informationen in einem Kurzzeitgedächtnis, sondern einem Langzeitgedächtnis speichern. Wichtig ist dabei zu berücksichtigen, dass ein Langzeitgedächtnis bereits durch Finetuning implementiert werden kann. Hier ist mehr die Rede davon, einen Kontextspeicher auf Projektebene zu etablieren, nicht auf Unternehmensebene, wie es mithilfe von Finetuning geschieht. Es müssen also Dateien und Informationen für die gesamte Zeitdauer des Projekts persistent gespeichert werden können, sodass das KI-Modell einen Speicher hat, auf den zuverlässig und ohne weiteres technisches Know-how zugegriffen werden kann. Wenn dieser Umschwung vom Finetuning hin zum projektbezogenen Langzeit-Kontextspeicher stattfindet, dann ist bereits ein erstes Hindernis für den Einsatz in der Entwicklung genommen.

Weiterhin muss dieser Kontextspeicher in Verbindung mit einem Tool eingesetzt werden, welches Zugriff auf die lokale Entwicklungsumgebung oder besser noch, das Repository hat. Diese Tools existieren, wie bereits erwähnt, basieren aber auf älteren Versionen von LLMs oder sind sehr spezifisch auf das Generieren von Code ausgelegt und erlauben es nicht, stärker auf Planung und Konfiguration einzugehen, um den Kontextspeicher zu verbessern.

Außerdem besteht ein weiteres Hindernis aufgrund der teilweise veralteten Informationen zum Veröffentlichungszeitpunkt des LLMs. Wenn ein Zugriff auf aktuelle Dokumentationen und Foren zur Laufzeit, also während der Generierung einer Ausgabe stattfinden kann, wird die Ausgabequalität stark erhöht. Es müssen weniger Rückfragen gestellt oder Prompts spezifiziert und verändert werden. Stattdessen könnte lediglich die aktuelle oder gewünschte Version der Technologie angegeben werden, um die Ausgabe auf die Anforderungen anzupassen. Zum aktuellen Stand sind aber Probleme der Kategorie Konfiguration oder aktuelle APIs von Frameworks nur schwer in die Problemlösung zu integrieren.

8.2.2 Entwicklerperspektive

Mit dem technologischen Fortschritt verändert sich auch die Entwicklungsbranche kontinuierlich. In Zukunft wird die Schulung von Entwicklern in Konzeption und Softwarearchitektur immer wichtiger. Die Rolle der Softwarearchitekten gewinnt an Bedeutung, insbesondere wenn bestehende Hindernisse abgebaut werden. In diesem Kontext wird der Entwickler weniger Problemlöser als vielmehr Konzeptersteller sein.

Trotz rasanter technologischer Entwicklungen werden weiterhin Nischenprodukte und neue Technologien von menschlichen Entwicklern geschaffen werden müssen. Large Language Models sind noch nicht so weit fortgeschritten, dass sie diese spezialisierten Aufgaben übernehmen können.

8. ZUSAMMENFASSUNG

Darüber hinaus bleibt die Community-Unterstützung durch Foren und Videoplattformen essenziell, um Entwicklern während des Einsatzes von KI alternative Lösungsansätze und Erklärungen zu bieten.

Zusammenfassend lässt sich sagen, dass eine effiziente Symbiose aus menschlichen Entwicklern, künstlicher Intelligenz und weiteren Ressourcen den Entwicklungsprozess verbessern kann. Diese Balance wird sich je nach technologischer Entwicklung stetig verändern. Bis zur Erreichung einer starken KI, sofern dies möglich ist, werden für Webentwickler weiterhin Grenzen bestehen, trotz der neuen Möglichkeiten durch Technologie.

Glossar

AI Artificial intelligence. 18, 23, 24, 33

API Application programming interface. 75

CAD Computer-aided design. 13

CLI Command-line interface. 50

CRUD Create Read Update Delete. 40, 43, 45

csv Comma-separated values. 46

DL Deep learning. 26

FM Foundation model. 28, 29

GPT Generative pre-trained transformer. 1, 13, 14, 15, 29, 30, 31, 33, 34, 35, 37, 65, 66, 67

HTML HyperText Markup Language. 14, 17

JSX JavaScript XML. 14

KG Knowledge graph. 32

KI Künstliche Intelligenz. 1, 2, 3, 4, 7, 8, 9, 10, 11, 12, 13, 14, 16, 17, 18, 20, 21, 23, 24, 25, 26, 28, 29, 30, 31, 35, 37, 38, 39, 40, 52, 53, 54, 55, 60, 61, 65, 67, 69, 70, 71, 72, 74, 75

KPI Key performance indicator. 59

LLM Large language model. 18, 19, 30, 32, 68, 75

ML Machine learning. 25, 26

MVC Model View Controller. 38

- NLG** Natural language generation. 30
- NLP** Natural language processing. 12, 16, 19, 24, 29, 30, 31
- NN** Neural network. 25, 27, 28, 30, 34
- REST** Representational state transfer. 36, 38, 46, 52, 53, 62
- SaaS** Software as a Service. 59
- SL** Supervised learning. 26
- SPA** Single-page application. 35
- UI** User Interface. 2, 13, 14, 17
- UL** Unsupervised learning. 26
- UX** User Experience. 2, 3, 17

Bibliography

- [1] (20) *The Benefits of AI and Automation for Businesses: Improve Efficiency, Productivity, and Results* — LinkedIn. URL: <https://www.linkedin.com/pulse/benefits-ai-automation-businesses-improve-efficiency-results/> (visited on 10/10/2023).
- [2] *8 Ways Real Businesses Are Using AI for Content Creation* — Zapier. URL: <https://zapier.com/blog/ai-content-creation/> (visited on 08/21/2023).
- [3] *9 Ways We Use AI in Our Products*. Google. Jan. 19, 2023. URL: <https://blog.google/technology/ai/9-ways-we-use-ai-in-our-products/> (visited on 10/09/2023).
- [4] Bilal Abu-Salih et al. "Relational Learning Analysis of Social Politics Using Knowledge Graph Embedding". In: *Data Min Knowl Disc* 35.4 (July 1, 2021), pp. 1497–1536. ISSN: 1573-756X. DOI: 10.1007/s10618-021-00760-w. URL: <https://doi.org/10.1007/s10618-021-00760-w> (visited on 08/22/2023).
- [5] *AI Design Tool* — AI UI Design — Uizard. URL: <https://uizard.io/ai-design/> (visited on 10/09/2023).
- [6] Valentina Alto. *Neural Networks: Parameters, Hyperparameters and Optimization Strategies*. Medium. July 6, 2019. URL: <https://towardsdatascience.com/neural-networks-parameters-hyperparameters-and-optimization-strategies-3f0842fac0a5> (visited on 08/18/2023).
- [7] anitakivindyo. *What Are Generative Pre-trained Transformers (GPTs)?* Medium. Feb. 16, 2023. URL: <https://medium.com/@anitakivindyo/what-are-generative-pre-trained-transformers-gpts-b37a8ad94400> (visited on 08/18/2023).
- [8] Habibullah Awan. *How Does ChatGPT Handle Context and Continuity in Conversation?* Medium. Feb. 6, 2023. URL: <https://medium.com/@habibullahawan/how-does-chatgpt-handle-context-and-continuity-in-conversation-a41d7ae19514> (visited on 10/15/2023).
- [9] *Bard – Chat-based AI Tool from Google, Powered by PaLM 2*. URL: <https://bard.google.com> (visited on 10/10/2023).
- [10] Marco Barenkamp. "Künstliche Intelligenz in der Softwareentwicklung". In: (2020). URL: <http://dx.doi.org/10.1365/s35764-020-00235-5> (visited on 08/21/2023).

- [11] Tony Beltramelli. *Pix2code: Generating Code from a Graphical User Interface Screenshot*. Sept. 19, 2017. DOI: [10.48550/arXiv.1705.07962](https://doi.org/10.48550/arXiv.1705.07962). arXiv: [1705.07962](https://arxiv.org/abs/1705.07962) [cs]. URL: <http://arxiv.org/abs/1705.07962> (visited on 08/18/2023). preprint.
- [12] *Big Data in Practice: How 45 Successful Companies Used Big Data Analytics to Deliver Extraordinary Results* — Wiley. Wiley.com. URL: <https://www.wiley.com/en-gb/Big+Data+in+Practice%3A+How+45+Successful+Companies+Used+Big+Data+Analytics+to+Deliver+Extraordinary+Results-p-9781119231387> (visited on 08/21/2023).
- [13] brisant.de. *Chat GPT: Wie funktioniert der KI Chat Bot von Open AI und was kann die App mit künstlicher Intelligenz?* URL: <https://www.brisant.de/ratgeber/chat-gpt-216.html> (visited on 10/09/2023).
- [14] Beatriz Cabrero-Daniel. *AI for Agile Development: A Meta-Analysis*. Comment: 8 pages, 2 figures, 24th International Conference on Agile Software Development. AI-Assisted Agile Software Development Research Workshop. May 14, 2023. DOI: [10.48550/arXiv.2305.08093](https://doi.org/10.48550/arXiv.2305.08093). arXiv: [2305.08093](https://arxiv.org/abs/2305.08093) [cs]. URL: <http://arxiv.org/abs/2305.08093> (visited on 08/21/2023). preprint.
- [15] Aylin Caliskan, Joanna J. Bryson, and Arvind Narayanan. "Semantics Derived Automatically from Language Corpora Contain Human-like Biases". In: *Science* 356.6334 (Apr. 14, 2017), pp. 183–186. DOI: [10.1126/science.aal4230](https://doi.org/10.1126/science.aal4230). URL: <https://www.science.org/doi/10.1126/science.aal4230> (visited on 08/21/2023).
- [16] *ChatGPT*. URL: <https://openai.com/chatgpt> (visited on 10/09/2023).
- [17] Dennis Consorte. *Why People Fear Generative AI — and What to Do About It*. Entrepreneur. May 22, 2023. URL: <https://www.entrepreneur.com/science-technology/why-people-fear-generative-ai-and-what-to-do-about-it/451757> (visited on 10/09/2023).
- [18] David De Cremer and Garry Kasparov. "AI Should Augment Human Intelligence, Not Replace It". In: *Harvard Business Review* (Mar. 18, 2021). ISSN: 0017-8012. URL: <https://hbr.org/2021/03/ai-should-augment-human-intelligence-not-replace-it> (visited on 10/09/2023).
- [19] *DALL·E 2*. URL: <https://openai.com/dall-e-2> (visited on 10/09/2023).
- [20] Julianna Delua. *Supervised vs. Unsupervised Learning: What's the Difference?* IBM Blog. Mar. 12, 2021. URL: <https://www.ibm.com/blog/supervised-vs-unsupervised-learning/> (visited on 10/09/2023).
- [21] *DevGPT - Start Developing The Easy Way with AI-Generated Code*. URL: <https://www.devgpt.com/> (visited on 10/14/2023).
- [22] M. Tim Jones Published 4 Dezember 2017. *Models for Machine Learning*. IBM Developer. URL: <https://developer.ibm.com/articles/cc-models-machine-learning/#reinforcement-learning> (visited on 08/17/2023).

-
- [23] E2Analyst. *GPT-4: Everything You Want to Know about OpenAI's New AI Model*. Predict. Mar. 15, 2023. URL: <https://medium.com/predict/gpt-4-everything-you-want-to-know-about-openais-new-ai-model-a5977b42e495> (visited on 10/21/2023).
- [24] Amanda Fetter. *The Difference between Weak AI and Strong AI*. VoiceHQ. Jan. 24, 2023. URL: <https://medium.com/voicehq/the-difference-between-weak-ai-and-strong-ai-2166e8b2ecdb> (visited on 10/09/2023).
- [25] Dr Bernd Finkbeiner and Frederik Schmitt. "Über die Schulter geschaut". In: ().
- [26] Kuruva Satya Ganesh. *What's The Role Of Weights And Bias In a Neural Network?* Medium. Sept. 7, 2022. URL: <https://towardsdatascience.com/whats-the-role-of-weights-and-bias-in-a-neural-network-4cf7e9888a0f> (visited on 08/18/2023).
- [27] Lakemeyer Gerhard. *Künstliche Intelligenz*. kas.de. URL: <https://www.kas.de/de/analysen-und-argumente/detail/-/content/kuenstliche-intelligenz> (visited on 08/18/2023).
- [28] *GitHub Copilot · Your AI Pair Programmer*. GitHub. URL: <https://github.com/features/copilot> (visited on 08/21/2023).
- [29] Manish Goyal. *What Is Generative AI, What Are Foundation Models, and Why Do They Matter?* IBM Blog. Mar. 8, 2023. URL: <https://www.ibm.com/blog/what-is-generative-ai-what-are-foundation-models-and-why-do-they-matter/> (visited on 08/17/2023).
- [30] Md Asraful Haque. "A Brief Analysis of "ChatGPT" – A Revolutionary Tool Designed by OpenAI". In: *EAI Endorsed Transactions on AI and Robotics* 1 (2022). ISSN: 2790-7511. DOI: 10.4108/airo.v1i1.2983. URL: <https://publications.eai.eu/index.php/airo/article/view/2983> (visited on 08/18/2023).
- [31] Sanskar Hasija. *HyperParameter Tuning: Fixing High Bias(Underfitting) in Neural Networks*. MLearning.ai. July 18, 2021. URL: <https://medium.com/mllearning-ai/hyperparameter-tuning-fixing-high-bias-underfitting-in-neural-networks-5184ead3cbed> (visited on 08/18/2023).
- [32] *How Amazon Uses AI to Prevent Damaged Products from Arriving on Your Doorstep*. US About Amazon. July 7, 2023. URL: <https://www.aboutamazon.com/news/amazon-ai/how-amazon-uses-ai-to-prevent-damaged-products> (visited on 10/09/2023).
- [33] *Importance of Neural Network Bias and How to Add It*. URL: <https://www.turing.com/kb/necessity-of-bias-in-neural-networks> (visited on 08/18/2023).
- [34] *Knowledge Graphs*. Fraunhofer-Institut für Intelligente Analyse- und Informationssysteme IAIS. URL: <https://www.iais.fraunhofer.de/de/geschaeftsfelder/speech-technologies/knowledge-graphs.html> (visited on 08/17/2023).

- [35] Kiprono Elijah Koech. *How Does Back-Propagation Work in Neural Networks?* Medium. Aug. 11, 2022. URL: <https://towardsdatascience.com/how-does-back-propagation-work-in-neural-networks-with-worked-example-bc59dfb97f48> (visited on 08/18/2023).
- [36] Takeshi Kojima et al. *Large Language Models Are Zero-Shot Reasoners*. Comment: Accepted to NeurIPS2022. Our code is available at https://github.com/kojima-takeshi188/zero_shot_cot. Jan. 29, 2023. arXiv: 2205.11916 [cs]. URL: <http://arxiv.org/abs/2205.11916> (visited on 10/15/2023). preprint.
- [37] *Learn Prompting: Your Guide to Communicating with AI*. URL: https://learnprompting.org/docs/basics/few_shot (visited on 10/15/2023).
- [38] *Learn Prompting: Your Guide to Communicating with AI*. URL: https://learnprompting.org/docs/intermediate/zero_shot_cot (visited on 10/15/2023).
- [39] Great Learning. *Reinforcement Learning*. Great Learning Blog: Free Resources what Matters to shape your Career! Oct. 29, 2019. URL: <https://www.mygreatlearning.com/blog/reinforcement-machine-learning/> (visited on 09/13/2023).
- [40] Vivian Liu et al. *3DALL-E: Integrating Text-to-Image AI in 3D Design Workflows*. July 31, 2023. arXiv: 2210.11603 [cs]. URL: <http://arxiv.org/abs/2210.11603> (visited on 08/21/2023). preprint.
- [41] Mohammed Lubbad. *The Ultimate Guide to GPT-4 Parameters: Everything You Need to Know about NLP's Game-Changer*. Medium. Oct. 6, 2023. URL: <https://medium.com/@mlubbad/the-ultimate-guide-to-gpt-4-parameters-everything-you-need-to-know-about-nlps-game-changer-109b8767855a> (visited on 10/15/2023).
- [42] Bernard Marr. *A Short History Of ChatGPT: How We Got To Where We Are Today*. Forbes. URL: <https://www.forbes.com/sites/bernardmarr/2023/05/19/a-short-history-of-chatgpt-how-we-got-to-where-we-are-today/> (visited on 10/09/2023).
- [43] Rick Merritt. *What Is a Transformer Model?* NVIDIA Blog. Mar. 25, 2022. URL: <https://blogs.nvidia.com/blog/2022/03/25/what-is-a-transformer-model/> (visited on 08/18/2023).
- [44] Nidhi. *Unsupervised Learning: Discovering Hidden Patterns and Insights*. Medium. July 9, 2023. URL: <https://artificialcorner.com/unsupervised-learning-discovering-hidden-patterns-and-insights-cd3353aa7a5d> (visited on 10/09/2023).
- [45] *OpenAI Codex*. URL: <https://openai.com/blog/openai-codex> (visited on 08/17/2023).
- [46] *OpenAI Platform*. URL: <https://platform.openai.com> (visited on 10/14/2023).
- [47] Anton Osika. *GPT Engineer*. Aug. 18, 2023. URL: <https://github.com/AntonOsika/gpt-engineer> (visited on 08/18/2023).

-
- [48] Joanna Peña-Bickley. *AI Hallucinations*. Medium. July 9, 2023. URL: <https://joannapenabickley.medium.com/ai-hallucinations-e79e20d1aebb> (visited on 08/18/2023).
- [49] Matthew Stewart PhD. *Simple Introduction to Neural Networks*. Medium. July 29, 2020. URL: <https://towardsdatascience.com/simple-introduction-to-neural-networks-ac1d7c3d7a2c> (visited on 08/18/2023).
- [50] Ron Ozminkowski PhD. *Garbage In, Garbage Out*. Medium. Nov. 16, 2021. URL: <https://towardsdatascience.com/garbage-in-garbage-out-721b5b299bc1> (visited on 08/17/2023).
- [51] *Pinia — The Intuitive Store for Vue.js*. URL: <https://pinia.vuejs.org> (visited on 10/24/2023).
- [52] Dominik Polzer. *All You Need to Know about Vector Databases and How to Use Them to Augment Your LLM Apps*. Medium. Sept. 18, 2023. URL: <https://towardsdatascience.com/all-you-need-to-know-about-vector-databases-and-how-to-use-them-to-augment-your-llm-apps-596f39adfedb> (visited on 10/09/2023).
- [53] Divyanshu Raj. *Semi-Supervised Approach for Transformers*. Medium. May 12, 2022. URL: <https://towardsdatascience.com/semi-supervised-approach-for-transformers-38722f1b5ce3> (visited on 08/18/2023).
- [54] Dr Mussaad M. Al-Razouki. *The Long Tail of Artificial Intelligence*. Medium. Dec. 18, 2018. URL: <https://mussaad.medium.com/the-long-tail-of-artificial-intelligence-fba8a192cb09> (visited on 10/09/2023).
- [55] *Reflections on Foundation Models*. Stanford HAI. URL: <https://hai.stanford.edu/news/reflections-foundation-models> (visited on 08/18/2023).
- [56] Mordechai Rorvig. *Computer Scientists Prove Why Bigger Neural Networks Do Better*. Quanta Magazine. URL: <https://www.quantamagazine.org/computer-scientists-prove-why-bigger-neural-networks-do-better-20220210/> (visited on 10/15/2023).
- [57] Gaurav Rajesh Sahani. *Elucidating Bias, Variance, Under-fitting, and Over-fitting*. Analytics Vidhya. Oct. 29, 2020. URL: <https://medium.com/analytics-vidhya/elucidating-bias-variance-under-fitting-and-over-fitting-273846621622> (visited on 08/18/2023).
- [58] V. Schetinger et al. "Doom or Deliciousness: Challenges and Opportunities for Visualization in the Age of Generative Models". In: *Computer Graphics Forum* 42.3 (2023), pp. 423–435. ISSN: 1467-8659. DOI: 10.1111/cgf.14841. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.14841> (visited on 08/21/2023).
- [59] Jasmine Shone. "Yes, You Can Make an App Too: A Systematic Study of Prompt Engineering in the Automatic Generation of Mobile Applications from User Queries". In: ().

- [60] Margarita Simonova. *Council Post: How AI Is Changing Automation Testing In Quality Assurance*. Forbes. URL: <https://www.forbes.com/sites/forbestechcouncil/2023/05/25/how-ai-is-changing-automation-testing-in-quality-assurance/> (visited on 08/21/2023).
- [61] Andrea Stocco. "How Artificial Intelligence Can Improve Web Development and Testing". In: *Proceedings of the Conference Companion of the 3rd International Conference on Art, Science, and Engineering of Programming*. Programming '19: Companion of the 3rd International Conference on Art, Science, and Engineering of Programming. Genova Italy: ACM, Apr. 2019, pp. 1–4. ISBN: 978-1-4503-6257-3. DOI: 10.1145/3328433.3328447. URL: <https://dl.acm.org/doi/10.1145/3328433.3328447> (visited on 08/21/2023).
- [62] Hendrik Strobelt et al. *Interactive and Visual Prompt Engineering for Ad-hoc Task Adaptation with Large Language Models*. Comment: 9 pages content, 2 pages references. Aug. 16, 2022. DOI: 10.48550/arXiv.2208.07852. arXiv: 2208.07852 [cs]. URL: <http://arxiv.org/abs/2208.07852> (visited on 08/21/2023). preprint.
- [63] Christian Szegedy et al. *Intriguing Properties of Neural Networks*. Feb. 19, 2014. DOI: 10.48550/arXiv.1312.6199. arXiv: 1312.6199 [cs]. URL: <http://arxiv.org/abs/1312.6199> (visited on 08/21/2023). preprint.
- [64] techslang. *What Is Prompt-Based Learning? — Techslang*. Techslang — Tech Explained in Simple Terms. June 1, 2023. URL: <https://www.techslang.com/definition/what-is-prompt-based-learning/> (visited on 10/09/2023).
- [65] *Uizard Autodesigner — AI UI Design Generator — Uizard*. URL: <https://uizard.io/autodesigner/> (visited on 08/18/2023).
- [66] *Unterschied zwischen AI und KI einfach erklärt • Kobold AI*. Apr. 15, 2022. URL: <https://www.kobold.ai/ai-vs-ki/> (visited on 08/18/2023).
- [67] Jason Wei et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. Jan. 10, 2023. arXiv: 2201.11903 [cs]. URL: <http://arxiv.org/abs/2201.11903> (visited on 10/15/2023). preprint.
- [68] *What Are AI Hallucinations and Why Are They a Problem? TechTarget*. WhatIs.com. URL: <https://www.techtarget.com/whatis/definition/AI-hallucination> (visited on 08/18/2023).
- [69] *What Are AI Hallucinations—and How Do You Prevent Them? — Zapier*. URL: <https://zapier.com/blog/ai-hallucinations/> (visited on 08/18/2023).
- [70] *What Are Foundation Models? IBM Research Blog*. Feb. 9, 2021. URL: <https://research.ibm.com/blog/what-are-foundation-models> (visited on 08/17/2023).
- [71] *What Are Neural Networks? — IBM*. URL: <https://www.ibm.com/topics/neural-networks> (visited on 10/09/2023).
- [72] *What Is a Chatbot? — IBM*. URL: <https://www.ibm.com/topics/chatbots> (visited on 08/18/2023).

-
- [73] *What Is Artificial Intelligence (AI)?* Google Cloud. URL: <https://cloud.google.com/learn/what-is-artificial-intelligence> (visited on 08/18/2023).
- [74] *What Is Deep Learning?* — IBM. URL: <https://www.ibm.com/topics/deep-learning> (visited on 10/09/2023).
- [75] *What Is GPT AI? - Generative Pre-Trained Transformers Explained* - AWS. Amazon Web Services, Inc. URL: <https://aws.amazon.com/what-is/gpt/> (visited on 10/09/2023).
- [76] *What Is Machine Learning?* — IBM. URL: <https://www.ibm.com/topics/machine-learning> (visited on 08/17/2023).
- [77] *What Is Microsoft's Approach to AI?* — Microsoft Source. Source. URL: <https://news.microsoft.com/source/features/ai/microsoft-approach-to-ai/> (visited on 10/09/2023).
- [78] *What Is Narrow AI (Weak AI)?* — Definition from TechTarget. Enterprise AI. URL: <https://www.techtarget.com/searchenterpriseai/definition/narrow-AI-weak-AI> (visited on 10/09/2023).
- [79] *What Is Natural Language Processing?* — IBM. URL: <https://www.ibm.com/topics/natural-language-processing> (visited on 08/17/2023).
- [80] *What Is Prompt Tuning?* IBM Research Blog. Feb. 9, 2021. URL: <https://research.ibm.com/blog/what-is-ai-prompt-tuning> (visited on 08/17/2023).
- [81] *What Is Strong AI?* — IBM. URL: <https://www.ibm.com/topics/strong-ai> (visited on 08/17/2023).
- [82] Jules White et al. *A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT*. Feb. 21, 2023. DOI: 10.48550/arXiv.2302.11382. arXiv: 2302.11382 [cs]. URL: <http://arxiv.org/abs/2302.11382> (visited on 08/21/2023). preprint.
- [83] Hochschule Würzburg-Schweinfurt- Hochschule für angewandte Wissenschaften. *Technische Hochschule Würzburg-Schweinfurt*. University Würzburg-Schweinfurt - Default Homepage with Accesskey 0. URL: <https://ki.thws.de/en/about/strong-vs-weak-ai-a-definition/> (visited on 10/09/2023).
- [84] Hao Zhang, Kai Guan, and Fei Jin. *Zhang, H. and Jin, F. (2023). Study on Sample Noise Immunity of Dense Matching Deep Learning Supervised Methods*. May 23, 2023. URL: <https://doi.org/10.1117/12.2680850>.
- [85] Enes Zvornicanin. *How Does ChatGPT Work?* — Baeldung on Computer Science. July 2, 2023. URL: <https://www.baeldung.com/cs/chatgpt-model> (visited on 10/14/2023).

